Dev2 구현 학습 가이드

2026-05-29

Contents

학습 노트 인덱스 7
Task 1~2 — 분석 노드 구현 (run_tools) 7
Task 3 — vector_search + llm_judge 노드 8
Task 4 — store 노드 (DB 저장) 9
Task 5 — LangGraph StateGraph 조립 (graph.py) 10
Task 6 — FastAPI 엔드포인트 (POST /scan + Insights API + PATCH dismiss) 10
Task 6 — FastAPI 엔드포인트 (POST /scan + Insights API + PATCH dismiss) 12
복습 순서 추천 13
Dev 2 구현 가이드 — 비전공자도 이해하는 버전 13
우리가 만드는 게 뭐야? 13
전체 흐름 한눈에 보기 13
Task 1: 주방 세팅 (Docker + 환경 세팅) 14
Task 2: 서류 양식 만들기 (Pydantic 스키마 + DB 테이블) 14
Task 3: 언어별 맞춤 검사관 팀 + 자동 배정 시스템 16
Task 4-a: 검사관 지휘자 만들기 (run_tools 노드) 18
Task 4-b: AI에게 지시서 잘 쓰기 (프롬프트 엔지니어링) 19
Task 4-c: AI 판사 만들기 (llm_judge 노드) 20
Task 4-d: 과거 사례 검색 (vector_search 노드 — RAG) 20
Task 4-e: AI 채점 능력 측정 (LLM Evaluation Harness) 21
Task 4-f: 결과 저장하기 (store 노드) 21
Task 5: 부품 조립하기 (LangGraph StateGraph) 22
Task 6: 창구 만들기 (FastAPI 엔드포인트) 23
Task 7: 자동화 스케줄러 + 연결 자동화 (Airflow + n8n) 24
Task 8: 전체 확인 (테스트 + E2E) 24
전체 Task 요약표 25
이해도 확인 퀴즈 26
변경 이력 27
subprocess 학습 노트 27
한 줄 요약 27
1. 비유 27
2. 개념 28

3. 왜 쓰나 28
4. 코드 (Dev 2) 28
복습 퀴즈 29
asyncio.gather 학습 노트 29
한 줄 요약 30
1. 비유 30
2. 개념 30
3. 왜 쓰나 30
4. 코드 (Dev 2) 30
복습 퀴즈 31
학습 노트: subprocess + asyncio (비동기 CLI 실행) 31
한 줄 요약 31
개념 1. subprocess — 외부 CLI 도구 실행 32
개념 2. asyncio.create_subprocess_exec — 비동기로 실행하기 33
3rd-Project-sub2 프로젝트에서 35
연관 개념 35
복습 퀴즈 (3문항) 35
학습 노트: asyncio 병렬 실행 패턴 (run_in_executor + gather) 36
한 줄 요약 36
들어가기 전: 이 노트와 `subprocess_async.md`의 차이 (중요) 36
개념 1. asyncio 이벤트 루프와 coroutine (기초) 37
개념 2. asyncio.gather() — 여러 coroutine을 동시 실행 37
개념 3. loop.run_in_executor() — 동기 함수를 비동기로 감싸기 ⭐ 핵심 38
개념 4. asyncio.wait_for() — 타임아웃 적용 (CLAUDE.md §6 필수) 39
개념 5. return_exceptions=True — 하나 실패해도 나머지 수집 40
개념 6. ThreadPoolExecutor vs ProcessPoolExecutor — 무엇을 쓰나 40
💻 실제 코드 패턴 분석 (Task 4-a 구현 형태) 41
3rd-Project-sub2 프로젝트에서 42
연관 개념 43
복습 퀴즈 (3문항) 43
PostgreSQL 인덱스 종류 학습 노트 44
한 줄 요약 44
1. 비유 44
2. 인덱스 종류별 설명 44
3. 요약 표 46
복습 퀴즈 46
UPSERT 학습 노트 46

한 줄 요약 46
1. 비유 46
2. 개념 46
3. 왜 쓰나 47
4. 코드 (Dev 2) 47
복습 퀴즈 47
Vector DB / 임베딩 학습 노트 47
한 줄 요약 47
1. 비유 48
2. 개념 48
3. 왜 쓰나 48
4. 코드 (Dev 2) 48
복습 퀴즈 49
한 줄 요약 49
1. 비유 50
2. Dev 2에서 사용하는 도구 5가지 50
3. 도구별 역할 요약 51
복습 퀴즈 51
학습 노트: 정적 분석 도구 (Pylint / Bandit / Semgrep / radon / ESLint) 52
한 줄 요약 52
핵심 개념: 정적 분석이란? 52
도구 1. Pylint — Python 코드 품질 52
도구 2. Bandit — Python 보안 취약점 53
도구 3. Semgrep — 다국어 패턴 기반 보안 스캔 53
도구 4. radon — Python 순환복잡도(Cyclomatic Complexity) 54
도구 5. ESLint — JavaScript/TypeScript 코드 품질 54
도구별 역할 한눈에 보기 55
3rd-Project-sub2 프로젝트에서 55
연관 개념 55
복습 퀴즈 (3문항) 56
학습 노트: 다국어 라우터 패턴 + PR diff 파싱 56
한 줄 요약 56
개념 1. 다국어 라우터 패턴 56
개념 2. PR diff 파싱 (unified diff → 변경 파일 목록) 58
두 개념이 합쳐지는 전체 흐름 (단계별) 59
3rd-Project-sub2 프로젝트에서 60
연관 개념 60

복습 퀴즈 (3문항) 60
LangChain 체인 학습 노트 61
한 줄 요약 61
1. 비유 61
2. 개념 61
3. 왜 쓰나 62
4. 코드 (Dev 2) 62
Minor 스킵 전략 (비용 최소화) 63
복습 퀴즈 63
LangGraph StateGraph 학습 노트 63
한 줄 요약 63
1. 비유 63
2. 개념 64
3. 왜 쓰나 64
4. 코드 (Dev 2) 64
복습 퀴즈 65
MAS (Multi-Agent System) 학습 노트 65
한 줄 요약 65
1. 비유 66
2. 개념 66
3. 왜 쓰나 66
4. 코드 (Dev 2) 66
복습 퀴즈 67
학습 노트: vector_search 노드 — pgVector RAG 검색 패턴 67
0. 이 노드가 무슨 일을 하나요? (전체 그림 먼저) 67
개념 1. pgVector 코사인 유사도 검색 68
개념 2. asyncpg — PostgreSQL 비동기 클라이언트 70
개념 3. 임베딩 생성 패턴 (Mock 전환 포함) 71
개념 4. LangGraph 노드에서의 DB 연결 패턴 73
개념 5. RAG Retrieve 패턴 (전체 조립) 74
3rd-Project-sub2 프로젝트에서 76
연관 개념 76
복습 퀴즈 (3문항) 76
학습 노트: 프롬프트 엔지니어링 5기법 (Few-shot / CoT / RAG 주입 / Structured Output / Domain Rules) 77
한 줄 요약 77
큰 그림: 왜 LLM이 다시 판단해야 하나? 77

🍕 1. 비유 — 신입 검사관 교육시키기 77
📚 2. 개념 — 5가지 기법의 기술적 정의 78
🤔 3. 왜 이 프로젝트에서 각 기법이 필요한가 80
💻 4. 실제 코드 분석 (dev2/prompts/severity_judge.py) 80
3rd-Project-sub2 프로젝트에서 82
⚠️ 흔한 실수 / 주의점 83
복습 퀴즈 (3문항) 83
연관 개념 83
학습 노트: llm_judge 노드 — LLM severity 재판단 패턴 83
0. 이 노드가 무슨 일을 하나요? (전체 그림 먼저) 84
개념 1. LangChain + Claude 통합 84
개념 2. USE_MOCK_LLM 패턴 (CLAUDE.md §10-3) 85
개념 3. 모델 라우팅 전략 (비용 최적화) 87
개념 4. json.loads() + 재시도 패턴 (CLAUDE.md §12) 88
개념 5. asyncio + LangChain 비동기 패턴 90
3rd-Project-sub2 프로젝트에서 92
복습 퀴즈 (5문항) 92
학습 노트: store 노드 — PostgreSQL + BigQuery 이중 적재 93
한 줄 요약 93
1. 왜 PostgreSQL + BigQuery 이중 적재인가? 93
2. store 노드의 역할 (파이프라인 마지막 노드) 94
3. Output Contract A 조립 방법 94
4. BigQuery 적재 패턴 (실패해도 파이프라인 계속) 94
5. CLAUDE.md §12 오류 처리 연결 95
복습 퀴즈 (3문항) 95
학습 노트: LangSmith Evaluation Harness (LLM 평가 자동화) 96
한 줄 요약 96
왜 쓰나요? 96
언제 쓰나요? 96
핵심 개념 3가지 96
어떻게 동작하나요? (코드 흐름) 97
3rd-Project-sub2 프로젝트에서 98
실무에서 왜 중요한가 98
복습 퀴즈 (3문항) 99
학습 노트: LangGraph StateGraph 5-노드 파이프라인 (Dev2) 99
한 줄 요약 99
1. StateGraph 기본 구조 — add_node / add_edge / compile() 99

2. TypedDict State — dict 반환 시 merge 동작 100
3. START / END 상수 — 시작/종료 연결 100
4. 5-노드 선형 파이프라인 패턴 101
5. graph.ainvoke() vs graph.invoke() 101
3rd-Project-sub2 프로젝트에서 102
복습 퀴즈 (3문항) 102
학습 노트: FastAPI 엔드포인트 설계 (라우터 + 202 + BackgroundTasks + Mock 테스트) 102
한 줄 요약 103
이 노트에서 다루는 10가지 개념 103
개념 1. FastAPI APIRouter — 창구를 파일별로 나누기 103
개념 2. HTTP 상태코드 202 vs 200 — “접수했어요” vs “다 됐어요” 104
개념 3. BackgroundTasks — 응답 먼저 주고 뒤에서 일하기 105
개념 4. HTTP 동사 (GET / POST / PATCH) — 요청에 “성격” 부여하기 106
개념 5. SQLAlchemy AsyncSession — DB와 대화하는 안전한 통로 107
개념 6. Lazy Import — 필요할 때만 짐 꺼내기 108
개념 7. db.py Lazy 초기화 — DB 엔진을 최초 사용 순간에만 켜기 109
개념 8. Mock으로 async with 흉내내기 — DB 없이 테스트하기 110
개념 9. “patch where it’s used” — Mock을 어디에 걸어야 하나 111
개념 10. ASGITransport — 진짜 서버 없이 API 테스트하기 112
🔁 전체 흐름 한눈에 보기 112
3rd-Project-sub2 프로젝트에서 113
연관 개념 113
복습 퀴즈 (5문항) 114
Apache Airflow DAG 학습 노트 114
한 줄 요약 114
1. 비유 114
2. 개념 114
3. 왜 쓰나 115
4. 코드 (Dev 2) 115
복습 퀴즈 116
TDD (테스트 주도 개발) 학습 노트 116
한 줄 요약 116
1. 비유 116
2. 개념 117
3. 왜 쓰나 117
4. 코드 (Dev 2) 117
5. pytest 핵심 문법 118

복습 퀴즈 119
Dev2 MVP 실행 학습 노트 119
한 줄 요약 119
전체 실행 흐름 119
실제 명령어 (복붙용) 119
주요 트러블슈팅 121
Swagger UI 121
핵심 개념 3가지 121
복습 퀴즈 121
Dev2 전체 실행 플로우 122
1단계 — 환경 준비 (서버 시작 전) 122
2단계 — DB 시작 122
3단계 — FastAPI 서버 시작 122
4단계 — 요청 수신 (두 가지 경로) 123
5단계 — LangGraph 5노드 파이프라인 (경로 A만) 123
6단계 — 결과 조회 124
전체 요약 (한눈에) 125

학습 노트 인덱스

프로젝트: Dev 2 정적 분석 + 보안 스캔 에이전트 기준: 초보자 눈높이 (비유 → 개념 → 이유 → 코드 순서)

Task 1~2 — 분석 노드 구현 **(run_tools)**

핵심: "외부 프로그램을 파이썬에서 어떻게 실행하고, 결과를 어떻게 병렬로 처리하나?"

영역	개념	파일	난이도
Python	subprocess — 외부 프로그램 실행	python/subprocess.md	⭐
보안	정적 분석 도구 (Pylint/Bandit/Semgrep/radon/pip-audit)	security/static_analysis.md	⭐⭐
비동기	asyncio.gather — 병렬 비동기 실행	python/asyncio_gather.md	⭐⭐
AI Native	다국어 라우터 (Python/JS/Java 파일별 도구 선택)	ai/multilang_router.md	⭐⭐
AI Native	subprocess + async 조합	ai/subprocess_async.md	⭐⭐
AI Native	병렬 에이전트 실행 패턴	ai/asyncio_parallel.md	⭐⭐

자주 헷갈리는 것: - Pylint vs Bandit 차이? → Pylint = 코드 스타일, Bandit = 보안 취약점 - subprocess와 asyncio를 같이 쓰는 이유? → 외부 프로그램 실행(subprocess) + 동시 실행(asyncio)

Task 3 — vector_search + llm_judge 노드

핵심: "KMS에서 관련 문서를 찾고, AI가 이슈를 판단한다"

영역	개념	파일	난이도
벡터	Vector DB / 임베딩 (텍스트 → 숫자 변환)	vector/vectordb.md	⭐⭐⭐
AI Native	pgVector RAG (유사도 검색)	ai/pgvector_rag.md	⭐⭐⭐
AI Native	프롬프트 엔지니어링 (Few-shot + CoT)	ai/prompt_engineering.md	⭐⭐
AI Native	LLM Judge 패턴 (Critical/Major/Minor 판정)	ai/llm_judge_patterns.md	⭐⭐⭐
에이전트	LangChain 체인	agent/langchain.md	⭐⭐

자주 헷갈리는 것: - 임베딩이 뭐야? → 텍스트를 숫자 배열로 바꿔서 "비슷한 내용" 수학으로 계산 - CoT(Chain of Thought)가 뭐야? → "단계별로 생각해"라고 AI에게 지시하는 기법

Task 4 — store 노드 (DB 저장)

핵심: "분석 결과를 PostgreSQL에 저장하고, 중복이면 덮어쓴다"

영역	개념	파일	난이도
DB	PostgreSQL 인덱스 종류 (B-tree, GIN, HNSW)	database/pg_indexes.md	⭐⭐⭐
DB	UPSERT (INSERT + 중복이면 UPDATE)	database/upsert.md	⭐
AI Native	store 노드 패턴	ai/store_node_patterns.md	⭐⭐
AI Native	LangSmith 평가 & 추적	ai/langsmith_evaluation.md	⭐⭐

자주 헷갈리는 것: - `ON CONFLICT DO UPDATE`가 뭐야? → 같은 key가 이미 있으면 UPDATE 실행 (= UPSERT) - `await s.commit()` 왜 필요해? → commit 없으면 DB에 실제로 저장 안 됨

Task 5 — LangGraph StateGraph 조립 (graph.py)

핵심: "5개 노드를 연결해서 파이프라인을 만든다"

영역	개념	파일	난이도
에이전트	MAS (멀티 에이전트 시스템)	agent/mas.md	⭐⭐
에이전트	LangGraph StateGraph	agent/langgraph.md	⭐⭐⭐
AI Native	LangGraph 5단계 파이프라인	ai/langgraph_stategraph_pipeline.md	⭐⭐⭐

파이프라인 흐름:

```
run_tools → vector_search → llm_judge → [escalate?] → store
```

자주 헷갈리는 것: - `state`가 뭐야? → 모든 노드가 공유하는 딕셔너리 (데이터 저장소) - `add_edge` vs `add_conditional_edges`? → add_edge=항상 이동, conditional=조건부 분기

Task 6 — FastAPI 엔드포인트 (POST /scan + Insights API + PATCH dismiss)

핵심: "LangGraph 파이프라인을 HTTP API로 외부에 노출한다"

영역	개념	파일	난이도
API	FastAPI APIRouter / include_router (라우터 분리)	ai/fastapi_endpoints.md	⭐
API	HTTP 상태코드 202 Accepted vs 200 OK	ai/fastapi_endpoints.md	⭐
API	FastAPI BackgroundTasks (응답 후 비동기 실행)	ai/fastapi_endpoints.md	⭐⭐
API	HTTP 동사 GET/POST/PATCH 와 멱등성	ai/fastapi_endpoints.md	⭐⭐
DB	SQLAlchemy AsyncSession + 파라미터 바인딩	ai/fastapi_endpoints.md	⭐⭐
Python	Lazy import (함수 내부 import)	ai/fastapi_endpoints.md	⭐⭐
Python	db.py Lazy 초기화 (싱글톤 엔진)	ai/fastapi_endpoints.md	⭐⭐
테스트	Mock으로 async with 흉내내기 (MagicMock/AsyncMock)	ai/fastapi_endpoints.md	⭐⭐⭐
테스트	patch where it's used vs defined	ai/fastapi_endpoints.md	⭐⭐⭐
테스트	httpx ASGITransport (in-process API 테스트)	ai/fastapi_endpoints.md	⭐⭐

Task 6 — FastAPI 엔드포인트 (POST /scan + Insights API + PATCH dismiss)

한 노트(`ai/fastapi_endpoints.md`)에 10개 개념을 비유→개념→이유→코드 순으로 정리.

영역	개념	파일	난이도
API	FastAPI APIRouter / include_router (라우터 분리)	ai/fastapi_endpoints.md	⭐
API	HTTP 상태코드 202 Accepted vs 200 OK	ai/fastapi_endpoints.md	⭐
API	FastAPI BackgroundTasks (응답 후 비동기 실행)	ai/fastapi_endpoints.md	⭐⭐
API	HTTP 동사 GET/POST/PATCH 와 멱등성	ai/fastapi_endpoints.md	⭐⭐
DB	SQLAlchemy AsyncSession + 파라미터 바인딩	ai/fastapi_endpoints.md	⭐⭐
Python	Lazy import (함수 내부 import)	ai/fastapi_endpoints.md	⭐⭐
Python	db.py Lazy 초기화 (싱글톤 엔진)	ai/fastapi_endpoints.md	⭐⭐
테스트	Mock으로 async with 흉내내기 (MagicMock/AsyncMock)	ai/fastapi_endpoints.md	⭐⭐⭐
테스트	patch where it's used vs defined	ai/fastapi_endpoints.md	⭐⭐⭐
테스트	httpx ASGITransport (in-process API 테스트)	ai/fastapi_endpoints.md	⭐⭐

학습 목표 (면접·포트폴리오용) - FastAPI 라우터 구조를 직접 설명할 수 있다 - 202 vs 200 차이를 면접에서 설명할 수 있다 - BackgroundTasks를 비유로 설명할 수 있다 - Mock으로 DB 없이 테스트하는 이유와 방법을 설명할 수 있다 - Lazy import를 왜 쓰는지 설명할 수 있다

복습 순서 추천

```
1일차: MAS → LangGraph → LangChain
2일차: asyncio.gather → subprocess
3일차: Vector DB → PostgreSQL 인덱스
4일차: Airflow → n8n → UPSERT
5일차: 정적 분석 도구 → TDD
6일차: FastAPI 엔드포인트 (APIRouter → 202/BackgroundTasks → HTTP 동사 →
AsyncSession → Lazy import → Mock 테스트)
```

Dev 2 구현 가이드 — 비전공자도 이해하는 버전

작성일: 2026-05-22 대상: 코딩을 처음 배우거나, 개념이 낯선 분

> 이 문서는 기술 용어를 최대한 피하고 일상 비유로 전체 흐름을 설명합니다.

우리가 만드는 게 뭐야?

> 한 줄 요약: 개발자가 코드를 올리면, AI가 자동으로 "이 코드 위험해요, 이 부분 고쳐요"를 알려주는 시스템

비유로 이해하기

병원 검진 시스템이라고 생각하세요.

```
환자(코드)가 병원에 옴
    → 혈액검사(Pylint), 엑스레이(Bandit), MRI(Semgrep) 등 여러 검사
    → 의사(AI/LLM)가 검사 결과 보고 최종 진단
    → 진단서(결과) 작성해서 전달
    → 기록 보관(DB)
```

전체 흐름 한눈에 보기

```
① 개발자가 GitHub에 코드 업로드
        ↓
② n8n이 감지 → "새 코드 왔어요!" 신호 전달
        ↓
③ 파일 종류(언어) 자동 감지 → 맞는 검사 도구 선택 → 동시에 검사
        ↓
```

④ 과거에 비슷한 문제 있었는지 검색 (Milvus)
        ↓
⑤ AI가 검사 결과 보고 심각도 판단 + 개선 방법 작성
        ↓
⑥ 위험(Critical)이면 즉시 알림
        ↓
⑦ 결과를 데이터베이스에 저장
        ↓
⑧ "이런 문제 발견됐어요" 결과 전달

Task 1: 주방 세팅 (Docker + 환경 세팅)

비유

식당을 열기 전에 주방 기기를 설치하는 것입니다.

냉장고 설치 → PostgreSQL (데이터 저장소)
재료 창고 설치 → Milvus (과거 이슈 검색 DB)
자동 주문 시스템 → n8n (GitHub 신호 받는 도구)
일정 관리 시스템 → Airflow (매일 자동 실행 도구)
주방 자체 → dev2 앱 서버

Docker가 뭐야?

택배 박스에 비유합니다.

일반 방식:
"내 컴퓨터에선 되는데 서버에선 안 돼요" 😭
→ 각자 컴퓨터 환경이 달라서 생기는 문제

Docker 방식:
앱 + 필요한 모든 것을 박스에 담아서 어디서든 똑같이 실행
→ "박스째로 실행하니까 어디서든 동일하게 동작"

docker-compose가 뭐야?

명령어 하나로 5개 서비스를 한꺼번에 실행하는 리모컨입니다.

```
docker-compose up   # 이 한 줄로 5개 서비스 동시 실행
```

이 **Task**가 끝나면?

컴퓨터에서 `docker-compose up` 명령어 하나로 모든 서비스가 켜집니다.

Task 2: 서류 양식 만들기 (Pydantic 스키마 + DB 테이블)

비유

Pydantic = 표준 업무 양식

```
"스캔 요청은 이 양식대로만 받겠습니다"

[스캔 요청서]
- 작업번호(task_id): 필수
- PR번호(pr_id): 필수
- 코드내용(pr_diff): 필수
- 개발자정보(developer): 필수
  └── 개발자ID, 실력수준, 기술스택
```

양식에 맞지 않는 요청이 오면? → 자동으로 거절합니다.

DB 테이블 = 창고 선반

```
선반 1: scan_executions (스캔 실행 기록)
    "2026-05-22 오후 3시에 PR #42 스캔함"

선반 2: scan_issues (발견된 문제 상세)
    "auth.py 42번줄에서 SQL 인젝션 위험 발견"

선반 3: scan_summaries (요약 숫자)
    "위험 2개, 경고 5개, 주의 3개"
```

TDD가 뭐야? **(**테스트 주도 개발**)**

레시피 검증에 비유합니다.

```
일반 방식:
요리 만들기 → 먹어보기 → 맛없으면 수정

TDD 방식:
"이 맛이 나야 합니다" 기준 먼저 정하기
→ 기준 충족하는 레시피 만들기
→ 자동으로 검증
```

코드로 보면:

```python
# ① 먼저 "이렇게 동작해야 한다" 테스트 작성
def test_task_payload():
    p = TaskPayload(task_id="001", pr_id="42", ...)
    assert p.developer.skill_level == "junior"  # 이래야 한다!

# ② 위 테스트를 통과하는 코드 구현
# ③ 테스트가 통과하면 완성
```

Task 3: 언어별 맞춤 검사관 팀 + 자동 배정 시스템

비유

어떤 언어로 된 코드든 해당 언어 전문 검사관에게 자동으로 배정됩니다.

```
Python 파일 (.py)   → Python 검사관 팀
JavaScript 파일(.js) → JavaScript 검사관 팀
C# 파일 (.cs)       → C# 검사관 팀
HTML 파일 (.html)   → HTML 검사관 팀
+ 어떤 언어든 → 공통 보안 검사관(Semgrep)이 항상 함께 검사
```

언어별 검사관 팀

Python 검사관 팀

검사관	전문 분야	하는 일
Pylint	코드 품질	문법 오류, 안 쓰는 변수, 네이밍 규칙 위반
radon	코드 복잡도	“이 함수 너무 복잡해서 버그 나기 쉬워요”
Bandit	Python 보안	SQL 인젝션, 하드코딩된 비밀번호 탐지
pip-audit	패키지 취약점	“쓰고 있는 라이브러리에 해킹 가능한 구멍 있어요”

JavaScript / TypeScript 검사관 팀

검사관	전문 분야	하는 일
ESLint	JS/TS 코드 품질	미사용 변수, 타입 오류, 규칙 위반
npm audit	패키지 취약점	npm 라이브러리 보안 구멍 탐지

C# 검사관 팀

검사관	전문 분야	하는 일
Roslyn Analyzer	.NET 코드 분석	C# 코드 품질 + 보안 패턴 검사

HTML / CSS 검사관 팀

검사관	전문 분야	하는 일
HTMLHint	HTML 유효성	태그 오류, XSS 취약점 탐지
Stylelint	CSS 품질	스타일 규칙 위반, 중복 선언

공통 보안 검사관 **(**모든 언어에 적용**)**

검사관	전문 분야	하는 일
Semgrep	보안 패턴	해킹 패턴 목록(OWASP Top 10)과 비교 — 언어 상관없이 전부 검사

자동 배정 시스템 **(router.py)**

파일 확장자를 보고 어떤 검사관 팀을 부를지 자동 결정합니다.

```
코드 파일 도착
    ↓
router.py가 확장자 확인
    ├── .py   → Python 팀 (pylint + radon + bandit + pip-audit)
    ├── .js   → JS 팀 (eslint + npm audit)
    ├── .ts   → TS 팀 (eslint + npm audit)
    ├── .cs   → C# 팀 (roslyn)
    ├── .html → HTML 팀 (htmlhint)
    └── .css  → CSS 팀 (stylelint)
         ↓
    + 항상: Semgrep (공통 보안)
```

subprocess가 뭐야**?**

Python이 터미널 명령어를 대신 실행해주는 것입니다.

```
터미널에서 직접:
$ pylint src/auth.py --output-format=json

Python 코드로:
result = subprocess.run(["pylint", "src/auth.py", "--output-format=json"])
# 결과를 변수에 저장해서 우리가 처리할 수 있음
```

asyncio.gather가 뭐야**?**

동시 주문에 비유합니다.

```
순서대로 검사 (느림):
Pylint 검사(30초) → 완료 → Bandit 검사(30초) → 완료
총 60초 걸림
```

```
asyncio.gather (빠름):
Pylint + Bandit 동시에 검사 시작
총 30초 걸림 (가장 오래 걸리는 것 기준)
```

Task 4-a: 검사관 지휘자 만들기 (run_tools 노드)

비유

Task 3에서 각 검사관을 고용했다면, Task 4-a는 지휘자를 만드는 것입니다.

```
코드가 도착하면 지휘자(run_tools 노드)가:
    1. 파일 확장자를 본다 (.py? .js? .html?)
    2. router.py에게 "어느 팀 불러야 해?" 물어본다
    3. 해당 언어 팀을 동시에 소집한다 (asyncio.gather)
    4. 결과를 바구니에 담는다
```

왜 별도 노드가 필요해?

```
Task 3 = 검사관 채용 (도구 함수 만들기)
Task 4-a = 지휘자 역할 (LangGraph 노드로 묶기)

예시:
run_tools 노드가 없으면:
    → 도구들이 있어도 누가 언제 어떻게 부를지 모름

run_tools 노드 있으면:
    → LangGraph 흐름에 연결됨
    → PR diff에서 파일 목록 추출
    → router.py로 언어 감지
    → 병렬 실행
    → 결과를 State 바구니에 저장
```

실제 흐름

```
PR diff 텍스트:
  "+++ b/src/auth.py"        → Python 파일 감지
  "+++ b/frontend/app.js"    → JavaScript 파일 감지
  "+++ b/views/index.html"   → HTML 파일 감지
          ↓
router.py: {"python": ["auth.py"], "javascript": ["app.js"], "html":
["index.html"]}
          ↓
asyncio.gather로 3개 언어팀 동시 소집
          ↓
static_raw: [Pylint 결과, ESLint 결과, HTMLHint 결과]
security_raw: [Bandit 결과, Semgrep(js) 결과, Semgrep(html) 결과]
```

TDD 포인트

```
# Mock(가짜 도구)을 써서 실제 CLI 없이도 테스트 가능
with patch("dev2.nodes.run_tools._run_for_language",
           return_value={"static": [이슈목록], "security": [이슈목록]}):
    result = await run_tools_node(state)
    assert len(result["static_raw"]) > 0
```

Task 4-b: AI에게 지시서 잘 쓰기 (프롬프트 엔지니어링)

비유

AI에게 막연하게 시키는 것과 구체적으로 시키는 것의 차이입니다.

```
❌ 나쁜 지시:
"이 코드 분석해줘"
→ AI가 매번 다른 기준으로 판단

✅ 좋은 지시:
"예시 3개 줄게. 이것처럼 판단해.
 그리고 1번부터 4번 순서대로 생각해서 답해.
 과거에 비슷한 사례도 참고해."
→ 일관되고 정확한 판단
```

3가지 기법

① Few-shot (예시 포함)

```
예시1: SQL 파라미터 미처리 → Critical (즉시 수정 필요)
예시2: 함수 복잡도 15 → Major (리팩토링 권고)
예시3: 변수명 규칙 위반 → Minor (다음에 수정)

→ 이 예시들처럼 판단해줘
```

② Chain-of-Thought (순서대로 생각하게)

```
"판단하기 전에 이 순서로 생각해:
 1단계: 이슈 패턴이 뭐야?
 2단계: 과거 사례와 비교하면?
 3단계: 최종 심각도는?
 4단계: 개발자 수준에 맞는 설명은?"
```

③ RAG (과거 사례 주입)

```
AI한테: "이 이슈, 과거 PR #98에서
         94% 유사한 케이스가 있었어.
         그때 이렇게 처리했거든. 참고해서 판단해줘."
→ AI가 맥락 있는 판단 가능
```

Task 4-c: AI 판사 만들기 (llm_judge 노드)

비유

검사관(CLI 도구)들이 "이상한 것 같아요"라고 보고하면, 시니어 엔지니어(AI)가 최종 판결을 내리는 과정입니다.

비용 아끼기: 모델 라우팅

AI API는 호출할 때마다 돈이 듭니다.

```
Critical (보안 위험) → 비싼 모델(gpt-4o) 사용
    → 정확도 최우선, 보안이니까 실수하면 안 됨

Major (코드 품질) → 저렴한 모델(gpt-4o-mini) 사용
    → 균형 잡힌 선택

Minor (스타일 위반) → AI 호출 안 함!
    → 굳이 AI 안 써도 됨, 돈 아끼기
```

코드 흐름

```
이슈 목록에서 하나씩 꺼냄
    │
    ├── Minor야? → 그냥 통과 (AI 안 부름)
    │
    └── Critical/Major야?
            │
            ├── 과거 유사 사례 + 이슈 내용을 AI에게 전달
            │
            ├── AI가 최종 심각도 + 개선 방법 작성
            │
            └── AI가 실패하면? → CLI 결과 그대로 사용 (안전장치)
```

Task 4-d: 과거 사례 검색 (vector_search 노드 — RAG)

비유

"이 증상, 예전 환자에서 본 적 있는데?"를 자동으로 찾아주는 것입니다.

임베딩이 뭐야?

텍스트를 숫자로 변환하는 것입니다.

```
"SQL 인젝션 위험" → [0.12, -0.34, 0.89, 0.02, ...]  (1536개 숫자)
"SQL injection"   → [0.11, -0.33, 0.91, 0.01, ...]  (비슷한 숫자!)
"오늘 날씨"       → [0.92,  0.15, -0.44, 0.77, ...]  (전혀 다른 숫자)
```

의미가 비슷하면 숫자도 비슷합니다 → 이걸 이용해서 "비슷한 이슈" 검색

왜 AI 판단(4-c) 전에 실행해야 해?

```
❌ 순서 바꾸면:
AI가 판단할 때 → "과거 사례? 몰라요"
→ 맥락 없이 판단

✅ 올바른 순서:
1. 먼저 유사 사례 검색 (4-d)
2. AI에게 유사 사례 알려주고 판단 (4-c)
→ "PR #98에서 비슷한 케이스가 있었어" 알고 판단
```

Task 4-e: AI 채점 능력 측정 (LLM Evaluation Harness)

비유

AI 선생님이 시험을 얼마나 잘 채점하는지 검증하는 것입니다.

```
정답지(골든 데이터셋) 만들기:
    "SQL 파라미터 미처리" → 정답: Critical
    "변수명 위반"         → 정답: Minor
    "CVE 점수 9.1"       → 정답: Critical
    (10개 준비)

AI한테 채점 시키기
    → AI 답: Critical ✅, Minor ✅, Major ❌ ...

정확도 계산:
    7/10 = 70% → 프롬프트 개선 → 9/10 = 90%
```

왜 필요해?

```
AI가 항상 맞는다는 보장이 없음
→ 측정해야 개선할 수 있음
→ 포트폴리오: "프롬프트 개선으로 정확도 70% → 90% 달성" 수치 제시 가능
```

Task 4-f: 결과 저장하기 (store 노드)

비유

검진 결과를 두 군데에 동시에 제출하는 것입니다.

```
PostgreSQL (병원 진료 기록부)
    → 지금 바로 조회 가능
    → "이 환자 오늘 검진 결과 보여줘"

BigQuery (의료 연구 데이터베이스)
    → 나중에 분석용
    → "지난 1년 동안 SQL 인젝션이 몇 번 발견됐어?"
```

asyncio.gather로 동시 저장

```
# 순서대로 저장: 1초 + 1초 = 2초
await save_postgresql(result)
await save_bigquery(result)

# 동시에 저장: max(1초, 1초) = 1초
await asyncio.gather(
    save_postgresql(result),
    save_bigquery(result),
)
```

Task 5: 부품 조립하기 (LangGraph StateGraph)

비유

앞에서 만든 5개 부품을 설계도대로 조립하는 것입니다.

State = 작업 바구니

```
모든 노드가 공유하는 바구니:
{
    payload:      "PR #42, 개발자 dev-001"            ← Task가 시작할 때 채워짐
    static_raw:   [언어별 코드품질 도구 결과들]          ← run_tools가 채움
                  (Pylint, ESLint, Roslyn, HTMLHint 등)
    security_raw: [언어별 보안 도구 결과들]              ← run_tools가 채움
                  (Bandit, Semgrep, npm audit 등)
    enriched:     [유사 이슈 + AI 판단 결과]            ← vector_search, llm_judge
가 채움
    final_output: [최종 결과]                          ← store가 채움
}
```

조립 흐름

```
run_tools (CLI 검사)
    ↓
vector_search (유사 이슈 검색)
    ↓
llm_judge (AI 최종 판단)
    ↓
Critical 있어? ── Yes ──→ escalate (즉시 알림 발송)
    │                          ↓
    No                      store (저장)
    ↓
store (저장)
    ↓
끝 (END)
```

조건부 엣지란?

```
일반 도로: 항상 직진 (A → B → C)

조건부 도로: 신호등처럼 상황에 따라 다른 방향
    Critical 있으면 → 경보 울리고 → 저장
    Critical 없으면 → 바로 저장
```

Task 6: 창구 만들기 (FastAPI 엔드포인트)

비유

외부에서 우리 시스템에 접근하는 창구(접수처)를 만드는 것입니다.

파일 구조 (창구 3개 파일)

```
api/
├── scan.py        → POST /scan             (스캔 접수)
├── insights.py    → GET  /insights/*       (통계 조회 3종)
└── issues.py      → PATCH /issues/*/dismiss (오탐 표시)
```

> dismiss를 왜 별도 파일로 분리했을까? insights 라우터는 `/insights` 로 시작. dismiss URL은 `/issues/...` 로 시작. 같은 파일에 넣으면 URL이 `/insights/issues/...` 가 되어버림 → 별도 분리

창구 5개

```
POST /scan
    → "이 PR 검사해줘" 접수
    → "접수됐어요"(202) 즉시 답변
    → 뒤에서 조용히 검사 진행

GET /insights/developer/{id}
    → "dev-001 개발자의 문제 패턴 보여줘"
    → "SQL 인젝션 3번, 복잡도 초과 5번..."

GET /insights/team
    → "이번달 팀 전체 Top5 문제"

GET /insights/trends
    → "주차별 위험 이슈 추이"

PATCH /issues/{id}/dismiss
    → "이 이슈는 오탐이야, 무시해"
    → scan_issues.is_false_positive = true 로 업데이트
```

202 상태코드란**?**

```
200 OK       = "완료됐어요"
202 Accepted = "접수됐어요, 처리 중이에요"

왜 202?
검사가 30초 걸리는데 "끝날 때까지 기다려"라고 하면
연결이 끊길 수 있음.
"접수했어요, 나중에 결과 조회해요"가 맞는 방식.
```

Task 7: 자동화 스케줄러 + 연결 자동화 (Airflow + n8n)

Airflow — 자동 알람시계

```
매일 새벽 2시 알람 (daily_etl):
    "PostgreSQL에서 어제 데이터 꺼내서 BigQuery로 옮겨"

매주 월요일 오전 9시 알람 (weekly_report):
    "1. 주간 이슈 요약 만들어"
    "2. 오탐률 높은 규칙 있으면 Slack에 알려줘"
    "3. Bandit/Semgrep 최신 버전 나왔으면 알려줘"
```

Airflow DAG이 뭐야**?**

```
DAG = 할 일 목록 + 순서

dev2_weekly_report DAG:
    할 일 1: 주간 인사이트 집계  ┐
    할 일 2: 오탐률 집계          ↵ (1 끝나면 2 시작)
    할 일 3: 룰셋 버전 체크       ← (2 끝나면 3 시작)
```

n8n — 자동 배달부

```
GitHub에서 PR이 열림
    ↓
n8n이 감지 ("새 PR 왔어요!")
    ↓
자동으로 POST /scan 호출
    (task_id, pr_id, pr_diff, developer 정보 담아서)
```

n8n 없으면 이 연결을 직접 코딩해야 함 (수백 줄). n8n 있으면 UI에서 블록 3개 연결하면 끝.

Task 8: 전체 확인 (테스트 + E2E)

3단계 확인

1단계**:** 부품별 확인 **(**단위 테스트**)**

"llm_judge가 Minor는 AI 안 부르나?" ✅
"vector_search가 유사 이슈 잘 찾나?" ✅
각 노드를 개별로 확인

2단계**:** 조립 확인 **(**통합 테스트**)**

"5개 노드가 연결돼서 잘 동작하나?"
전체 흐름을 한 번에 확인

3단계**:** 실제 확인 **(E2E** 테스트**)**

실제로 PR 스캔 요청 → 결과 확인
curl -X POST http://localhost:8002/scan ...
curl http://localhost:8002/insights/developer/dev-001

전체 **Task** 요약표

Task	비유	핵심 기술
1. Docker 세팅	주방 기기 설치	Docker Compose
2. 스키마	서류 양식 + 창고 선반	Pydantic, PostgreSQL
3. CLI 도구	언어별 맞춤 검사관 팀 + 자동 배정	subprocess, asyncio, router.py
4a. run_tools 노드	검사관 지휘자	**LangGraph** 노드**, asyncio.gather**
4b. 프롬프트	AI 지시서 잘 쓰기	Few-shot, CoT, RAG
4c. llm_judge	AI 판사	LangChain, 모델 라우팅
4d. vector_search	과거 사례 검색	Milvus, 임베딩
4e. Evaluation	AI 채점 능력 측정	LangSmith
4f. store	결과 두 곳에 저장	PostgreSQL, BigQuery
5. StateGraph	5개 부품 조립 (조건부 분기 포함)	LangGraph
6. FastAPI	창구 3파일 (scan/insights/issues)	FastAPI
7. Airflow + n8n	알람 + 자동 배달부	Airflow, n8n
8. 테스트	전체 확인	pytest

이해도 확인 퀴즈

1. **Q**: Docker를 쓰는 이유는? **A**: (내 컴퓨터에서만 되는 문제 방지. 박스째로 실행해서 어디서든 동일하게 동작)
2. **Q**: asyncio.gather가 없으면 어떻게 되나? **A**: (Pylint 30초 + Bandit 30초 = 60초 걸림. gather 쓰면 30초로 단축)
3. **Q**: vector_search가 llm_judge 앞에 있어야 하는 이유는? **A**: (AI가 판단할 때 "과거 유사 사례"를 알고 판단해야 더 정확하기 때문. RAG 패턴)
4. **Q**: POST /scan이 202를 반환하는 이유는? **A**: (검사가 30초 걸리는데 기다리면 연결 끊길 수 있음. "접수됐어요"로 즉시 응답)
5. **Q**: Minor 이슈는 왜 AI(LLM)를 호출하지 않나? **A**: (AI API 호출은 돈이 듦. Minor는 굳이 AI 안 써도 됨 → 비용 절감)
6. **Q**: Python 코드와 JavaScript 코드가 같이 들어오면 어떻게 되나? **A**: (router.py가 파일 확장자(.py/.js)를 보고 각각 맞는 검사관 팀에 자동 배정. 둘 다 Semgrep 공통 보안 검사는 공유)
7. **Q**: Semgrep이 모든 언어에 공통으로 적용되는 이유는? **A**: (보안 취약점 패턴(OWASP Top 10)은 언어와 상관없이 존재. Python이든 JS든 SQL 인젝션은 위험하므로 항상 검사)
8. **Q**: run_tools 노드(Task 4-a)는 Task 3 도구들과 뭐가 달라? **A**: (Task 3 = 검사관 고용(함수 만들기). Task 4-a = 지휘자(LangGraph 노드로 묶어서 흐름에 연결, 언어 감지+병렬 실행+결과 수집))
9. **Q**: PATCH /issues/{id}/dismiss가 왜 /insights/issues/...가 아니라 /issues/...인가? **A**: (insights 라우터는 prefix가 /insights. dismiss는 /issues prefix 별도 라우터(api/issues.py)에 있음. 같은 파일에 넣으면 URL이 /insights/issues/...로 꼬임)

변경 이력

날짜	변경 내용	이유
2026-05-22 (초기)	전체 문서 작성 — Python 전용 5개 도구	기초 설계
2026-05-22 (1차)	Task 3 전면 재작성 — 다국어 지원 (Python/JS/TS/C#/HTML/CSS) + router.py 자동 배정	PRD 다국어 요구사항 반영
2026-05-22 (2차)	Task 4-a 신규 추가 — run_tools 노드 (검사관 지휘자)	구현 계획에 있던 노드 누락 보완
2026-05-22 (2차)	Task 5 State 바구니 — `static_raw/security_raw` 설명을 다국어 도구로 업데이트	다국어 도구 변경에 따른 동기화
2026-05-22 (2차)	Task 6 — dismiss 전용 파일(`api/issues.py`) 분리 이유 설명 추가	URL prefix 충돌 문제 설명
2026-05-22 (2차)	요약표 — Task 4-a 행 추가, Task 5/6 설명 업데이트	구현 계획과 동기화
2026-05-22 (2차)	퀴즈 Q8/Q9 추가 — run_tools 노드, dismiss 라우터 분리	새 개념 이해 확인

subprocess 학습 노트

날짜: 2026-05-22 복습 예정: D+1, D+3, D+7

한 줄 요약

Python 코드 안에서 터미널 명령어를 실행하는 방법 — `pylint src/auth.py`를 코드로 실행하고 결과를 받아오는 것.

---

1. 비유

터미널에서 직접 입력:

```
pylint src/auth.py --output-format=json
```

Python 코드로 같은 작업:

```
import subprocess
result = subprocess.run(["pylint", "src/auth.py", "--output-format=json"],
capture_output=True, text=True)
print(result.stdout)  # 터미널 출력과 동일한 결과
```

subprocess = Python이 터미널 대신 명령어를 실행해주는 도구.

---

2. 개념

`subprocess.run()`: 외부 프로그램 실행 + 결과 반환

- `capture_output=True`: stdout, stderr를 캡처 (화면에 출력 안 됨)
- `text=True`: 결과를 bytes가 아닌 str로 반환
- `returncode`: 0 = 성공, 0 외 = 오류/이슈 발견

---

3. 왜 쓰나

- 없으면: Pylint/Bandit을 Python API로 직접 호출 → 각 도구마다 다른 API 학습 필요, 버전 의존성 높음
- 있으면: CLI 명령어로 실행 → 도구 교체가 쉽고, CLI 옵션 그대로 사용 가능

Dev 2에서: Pylint, radon, Bandit, Semgrep, pip-audit 모두 subprocess로 실행

---

4. 코드 (Dev 2)

```python
# dev2/tools/static/pylint_tool.py
import subprocess, json

def run_pylint(files: list[str]) -> list[dict]:
    if not files:
        return []

    result = subprocess.run(
        ["pylint"] + files + ["--output-format=json"],
        capture_output=True,   # stdout/stderr 캡처
        text=True,             # str 반환 (bytes 아님)
        timeout=30,            # 30초 초과 시 TimeoutExpired 예외
    )

    # Pylint는 이슈가 있어도 returncode != 0 → 이슈 있음 = 정상
    # stdout이 비어있으면 이슈 없음
    if not result.stdout.strip():
        return []

    issues = json.loads(result.stdout)
    return [
        {
            "severity": _map_severity(i["type"]),
```

```
            "line": i["line"],
            "file": i["path"],
            "message": i["message"],
            "rule_id": f"SA-{i['symbol'].upper()}",
            "suggestion": "",
            "similar_issues": [],
        }
        for i in issues
    ]

def _map_severity(pylint_type: str) -> str:
    return {"error": "Critical", "warning": "Major", "convention": "Minor",
"refactor": "Minor"}.get(pylint_type, "Minor")
```

주요 패턴 비교:

```
# 기본: 결과 캡처
result = subprocess.run(["pylint", "src/"], capture_output=True, text=True)
result.stdout   # 표준 출력 (JSON 결과)
result.stderr   # 에러 출력
result.returncode  # 종료 코드

# 타임아웃 처리
try:
    result = subprocess.run(["bandit", "-r", "src/"], capture_output=True,
text=True, timeout=30)
except subprocess.TimeoutExpired:
    return []  # 타임아웃 시 빈 결과 반환

# shell=True는 사용 금지 (보안 위험 — Shell Injection)
subprocess.run("pylint " + user_input, shell=True)   # ❌ 위험
subprocess.run(["pylint", user_input], shell=False)   # ✅ 안전
```

복습 퀴즈

1. Q: `capture_output=True`를 쓰는 이유는? A: (터미널에 출력 대신 result.stdout으로 받아서 Python에서 처리하기 위해)
2. Q: `shell=True`를 쓰면 안 되는 이유는? A: (Shell Injection 보안 취약점. 사용자 입력이 명령어에 포함되면 악의적인 명령 실행 가능)
3. Q: Dev 2에서 subprocess를 쓰는 이유는? A: (Pylint/Bandit 등 CLI 도구를 Python 내에서 실행하고 JSON 결과를 받아 파싱하기 위해)

asyncio.gather 학습 노트

날짜: 2026-05-22 복습 예정: D+1, D+3, D+7

한 줄 요약

여러 작업을 동시에 시작하고, 모두 끝날 때까지 기다리는 함수.

---

1. 비유

음식 배달에 비유:

asyncio.gather 없이 (순차):

```
짜장면 주문 → 기다림(10분) → 탕수육 주문 → 기다림(8분) = 총 18분
```

asyncio.gather 있이 (병렬):

```
짜장면 + 탕수육 동시 주문 → 기다림(10분) = 총 10분
```

Dev 2에서:

```
# 정적 분석(30초) + 보안 스캔(30초)을 동시에 실행 → 총 30초
# 순서대로 하면 60초 걸릴 것을 절반으로 단축
```

---

2. 개념

> `asyncio.gather()`는 여러 비동기 함수**(coroutine)**를 동시에 실행하고, 모든 결과를 리스트로 반환한다.

동기 **vs** 비동기**:**

구분	설명	키워드
동기(sync)	하나 끝나야 다음 시작	일반 함수 `def`
비동기(async)	동시에 여러 개 실행 가능	`async def`, `await`

---

3. 왜 쓰나

- 없으면: Pylint 실행(30초) 끝난 후 Bandit 실행(30초) → 총 60초
- 있으면: 둘 동시 실행 → 총 30초 → PRD "60초 이내" 목표 달성

---

4. 코드 (Dev 2)

```python
import asyncio

# 기본 사용법
results = await asyncio.gather(
    run_static_analysis(payload),   # 동시 시작
```

```
    run_security_scan(payload),      # 동시 시작
)
# 둘 다 끝나면 results = [static_result, security_result]

# return_exceptions=True: 하나가 실패해도 나머지 결과 반환
results = await asyncio.gather(
    run_static_analysis(payload),
    run_security_scan(payload),
    return_exceptions=True   # ← 이게 없으면 하나 실패 시 전체 실패
)

# 실패한 것 걸러내기
valid = [r for r in results if not isinstance(r, Exception)]
```

주의사항:

```
# async def가 아닌 일반 함수는 gather에 넣을 수 없음
async def run_static_analysis(payload):  # async 필수
    ...

# await 없이 호출하면 실행 안 됨
result = run_static_analysis(payload)        # ❌ 실행 안 됨
result = await run_static_analysis(payload)  # ✅ 올바른 사용
```

복습 퀴즈

1. Q: asyncio.gather와 순차 실행의 차이는? A: (gather는 동시 실행 → 시간 절약. 순차는 하나씩 → 느림)
2. Q: `return_exceptions=True`를 쓰는 이유는? A: (하나의 에이전트가 실패해도 나머지 결과는 정상 반환하기 위해)
3. Q: `async def`와 `def`의 차이는? A: (async def는 비동기 함수로 await로 호출해야 함. def는 동기 함수)

학습 노트: subprocess + asyncio (비동기 CLI 실행)

작성일: 2026-05-24 관련 파일: `dev2/tools/python/`, `dev2/tools/common/`, `dev2/nodes/` 복습 예정: D+1, D+3, D+7 관련 **Task**: Task 3 — CLI 도구 + 다국어 라우터 구현

한 줄 요약

Python 코드 안에서 pylint·bandit 같은 외부 검사 도구를 "동시에" 여러 개 돌리고, 모두 끝나면 결과를 한꺼번에 받아오는 방법.

개념 1. subprocess — 외부 CLI 도구 실행

(1) 비유

주방장(내 Python 코드)이 직접 모든 요리를 하지 않고, 전문 요리사(pylint, bandit)에게 "이 재료(파일)로 검사해줘"라고 시킨 뒤 완성된 접시(결과)만 받아오는 것.

요리사가 어떻게 일하는지는 몰라도 됩니다. 주문서(명령어)와 완성품(stdout)만 주고받으면 됩니다.

(2) 명확한 개념

> `subprocess`는 Python 프로그램이 다른 프로그램(외부 실행 파일)을 별도의 프로세스로 띄워서 실행하고, 그 출력(stdout/stderr)과 종료 코드(returncode)를 받아오는 표준 모듈이다.

- **stdout**: 프로그램이 정상적으로 내보낸 결과 (보통 JSON)
- **stderr**: 에러 메시지가 나오는 통로
- **returncode**: 0이면 정상 종료, 0이 아니면 이슈 발견 또는 오류

핵심 원리: 내 프로그램과 검사 도구는 완전히 분리된 프로세스다. 도구가 멈추거나 죽어도 내 프로그램은 살아있다. 그래서 안정적이다.

(3) 왜 쓰나

- 없으면: pylint, bandit, semgrep을 각각 Python 라이브러리로 직접 import해서 호출해야 함 → 도구마다 API가 다르고, 버전 올라가면 import 구조가 바뀌어서 코드가 깨짐
- 있으면: 터미널에서 치던 명령어(`pylint src/auth.py`) 그대로 코드에서 실행 → 도구 교체·추가가 쉽고, CLI 옵션을 그대로 재사용

(4) 실제 코드 (동기 버전 — 기본 이해용)

```
# dev2/tools/python/pylint_tool.py
import subprocess, json

def run_pylint_sync(files: list[str]) -> list[dict]:
    result = subprocess.run(
        ["pylint", *files, "--output-format=json"],  # 명령어를 "리스트"로 전달
        capture_output=True,   # 화면 출력 대신 result.stdout 으로 받기
        text=True,             # bytes 가 아니라 str 로 받기
        timeout=30,            # 30초 넘으면 TimeoutExpired 예외 (CLAUDE.md §6 타임
아웃 규칙)
    )
    if not result.stdout.strip():   # 이슈가 없으면 stdout 이 빔
```

```
        return []
    return json.loads(result.stdout)
```

보안 주의: `shell=True`는 절대 쓰지 않는다.

```
subprocess.run("pylint " + user_input, shell=True)   # 위험 — Shell Injection
subprocess.run(["pylint", user_input])               # 안전 — 인자가 명령어로 해
석되지 않음
```

개념 2. asyncio.create_subprocess_exec — 비동기로 실행하기

(1) 비유

동기 `subprocess.run`은 "요리사 한 명에게 시키고 그 앞에 서서 완성될 때까지 멍하니 기다리는 것". 요리가 5개면 한 명씩 차례로 시켜서 (5분 × 5 = 25분) 걸립니다.

`asyncio.create_subprocess_exec`는 "요리사 5명에게 동시에 주문하고, 나는 그동안 다른 일을 하다가 다 되면 한꺼번에 받는 것". 그래서 가장 오래 걸리는 요리 시간(5분)만 기다리면 됩니다.

(2) 명확한 개념

> `asyncio.create_subprocess_exec`는 외부 프로그램을 비동기(코루틴) 로 실행한다. 도구가 일하는 동안 CPU는 멈춰서 기다리지 않고 다른 도구를 동시에 시작할 수 있다. `await proc.communicate()`로 출력이 다 나올 때까지 기다린 뒤 stdout/stderr를 받는다.

`subprocess.run` (동기) vs `create_subprocess_exec` (비동기) 차이:

구분	동기 subprocess.run	비동기 create_subprocess_exec
호출 방식	그냥 호출	`await` 필요
여러 개 실행	하나 끝나야 다음	`asyncio.gather`로 동시
5개 도구 (각 30초)	최대 150초	약 30초
결과 받기	`result.stdout`	`await proc.communicate()`

(3) 왜 쓰나

- 없으면: pylint(30초) → radon(20초) → bandit(30초) → semgrep(40초)를 순서대로 = 120초. CLAUDE.md §6의 "에이전트 30초 타임아웃" 안에 못 끝남
- 있으면: 4개를 동시에 시작 → 가장 느린 semgrep(40초)만 기다림 → 1개 에이전트 타임아웃(30초) 안에서 여러 도구를 굴릴 수 있음

왜 외부 CLI는 비동기 효과가 클까? 정적 분석 도구는 대부분 "내가 일하는 게 아니라 외부 프로그램이 디스크 읽고 분석하는 동안 기다리는" I/O 대기 시간이다. 이 대기 시간 동안 다른 도구를 동시에 굴리면 전체 시간이 크게 줄어든다.

(4) 실제 코드 **(**비동기 버전**)**

```
# dev2/tools/common/runner.py — 모든 언어 도구가 공통으로 쓰는 실행기
import asyncio, json

async def run_cli(cmd: list[str], timeout: int = 30) -> tuple[str, str, int]:
    """
    외부 CLI 도구를 비동기로 실행하고 (stdout, stderr, returncode)를 반환.
    타임아웃 발생 시 프로세스를 강제 종료하고 예외를 전파한다.
    """
    proc = await asyncio.create_subprocess_exec(
        *cmd,                                # ["pylint", "auth.py", "--output-
format=json"]
        stdout=asyncio.subprocess.PIPE,      # 출력을 파이프로 받음
        stderr=asyncio.subprocess.PIPE,
    )
    try:
        stdout, stderr = await asyncio.wait_for(proc.communicate(),
timeout=timeout)
    except asyncio.TimeoutError:
        proc.kill()                          # 안 죽이면 좀비 프로세스로 남음
        await proc.wait()
        raise                                # CLAUDE.md §6: with 블록 바깥에서 잡도
록 전파
    return stdout.decode(), stderr.decode(), proc.returncode


# 여러 도구를 동시에 실행 (개념 통합)
async def run_python_tools(files: list[str]) -> list[dict]:
    results = await asyncio.gather(
        run_cli(["pylint", *files, "--output-format=json"]),
        run_cli(["bandit", "-r", *files, "-f", "json"]),
        run_cli(["radon", "cc", *files, "-j"]),
        return_exceptions=True,   # 하나 실패해도 나머지 결과는 살림
    )
    issues: list[dict] = []
    for r in results:
        if isinstance(r, Exception):
            continue              # 타임아웃/실패한 도구는 건너뜀
        stdout, _, _ = r
        if stdout.strip():
            issues.extend(json.loads(stdout))
    return issues
```

3rd-Project-sub2 프로젝트에서

파일: `dev2/tools/common/runner.py` (공통 실행기), `dev2/tools/python/· javascript/ ·csharp/` (언어별 도구 래퍼)

역할: 1. 다국어 라우터가 "이 PR에 .py 파일이 3개, .js 파일이 1개 있다"고 알려주면 2. 각 언어 폴더의 도구 래퍼가 `run_cli()`를 호출해 pylint/eslint 등을 비동기 동시 실행 3. 모든 결과를 모아 Output Contract A (`AgentOutput`, `schema.py`)로 변환 → Dev4에게 전달

> CLAUDE.md §6 타임아웃 규칙 연결: `try/except asyncio.TimeoutError`는 `async with AgentHarness()` 바깥에 둬야 `agent_traces.status`가 잘못 'success'로 기록되지 않는다.

연관 개념

- **asyncio.gather** → 여러 코루틴을 동시 실행하고 결과를 리스트로 모음 (`docs/learning/python/asyncio_gather.md`)
- **asyncio.wait_for** → 코루틴에 타임아웃을 씌우는 함수. 시간 초과 시 `TimeoutError`
- 좀비 프로세스 → 부모가 거두지 않은 끝난 자식 프로세스. `proc.kill()` 후 `await proc.wait()`로 정리
- **PIPE** → 프로세스 간 데이터가 흐르는 관. stdout을 PIPE로 받아야 코드에서 읽을 수 있음

복습 퀴즈 (3문항)

1. **(이해)** `subprocess.run`(동기)과 `asyncio.create_subprocess_exec`(비동기)의 가장 큰 차이는 무엇이며, 정적 분석 도구 5개를 돌릴 때 왜 비동기가 유리한가?
   - 답: 동기는 한 도구가 끝나야 다음을 시작(순차)하고, 비동기는 `asyncio.gather`로 동시 실행 가능. 정적 분석은 외부 프로그램이 일하는 I/O 대기 시간이 길어서, 동시에 굴리면 가장 느린 도구 시간만큼만 걸려 30초 타임아웃 안에 들어온다.
2. **(적용)** `await asyncio.wait_for(proc.communicate(), timeout=30)`에서 30초가 지나면 무슨 일이 일어나며, 그다음 코드에서 반드시 해야 할 두 가지는?
   - 답: `asyncio.TimeoutError`가 발생한다. (1) `proc.kill()`로 멈춘 도구 프로세스를 강제 종료하고 (2) `await proc.wait()`로 좀비 프로세스를 정리한 뒤 예외를 전파한다.
3. **(비교/보안)** `subprocess.run("pylint " + user_input, shell=True)`와 `run_cli(["pylint", user_input])` 중 안전한 것은? 그 이유는?
   - 답: 리스트로 전달하는 후자가 안전. `shell=True`는 사용자 입력이 셸 명령으로 해석되어 `; rm -rf /` 같은 악성 명령이 끼어드는 Shell Injection이 가능하지만, 리스트 인자는 pylint의 단순 인자로만 전달되어 명령으로 실행되지 않는다.

학습 노트: asyncio 병렬 실행 패턴 (run_in_executor + gather)

작성일: 2026-05-24 관련 파일: `dev2/tools/python/pylint_tool.py`, `dev2/tools/python/bandit_tool.py`, `dev2/nodes/` (구현 예정: `run_tools` 노드) 복습 예정: D+1, D+3, D+7 관련 **Task**: Task 4-a — subprocess CLI 도구 병렬 실행 노드 구현

한 줄 요약

이미 만들어 둔 동기(sync) 검사 함수(`run_pylint`, `run_bandit`)를 고치지 않고, 별도의 작업자 스레드에 던져서 여러 개를 "동시에" 굴리고, 30초 안에 끝난 것만 모아오는 방법.

들어가기 전: 이 노트와 **`subprocess_async.md`**의 차이 (중요)

같은 "병렬 실행"이지만 두 가지 길이 있습니다. 둘을 헷갈리면 안 됩니다.

구분	`subprocess_async.md` (Task 3)	이 노트 **`asyncio_parallel.md` (Task 4-a)**
접근법	도구 실행을 처음부터 `async`로 다시 작성	기존 동기 함수를 그대로 두고 비동기로 "감싼다"
핵심 함수	`asyncio.create_subprocess_exec`	**`loop.run_in_executor`**
함수 모양	`async def run_cli(...)`	`def run_pylint(...)` (기존 그대로)
언제	새로 만들 때, 가장 효율적	이미 `subprocess.run`으로 짜둔 코드를 병렬화할 때

> 우리 상황: `dev2/tools/python/pylint_tool.py`의 `run_pylint`는 이미 동기 **`subprocess.run`**으로 완성돼 있습니다. 이걸 다시 안 짜고 그대로 살리려면 `run_in_executor`가 정답입니다. 그래서 Task 4-a는 이 노트의 패턴을 씁니다.

개념 1. asyncio 이벤트 루프와 coroutine (기초)

🍕 비유

식당에 종업원이 한 명(이벤트 루프) 있습니다. 손님이 "물 주세요" 하면 종업원이 주방에 주문만 넣고, 물이 나오는 동안 멍하니 서 있지 않고 다른 테이블 주문을 받으러 갑니다. 물이 다 되면 그때 가서 가져다줍니다. 종업원은 한 명인데도 여러 테이블이 동시에 굴러갑니다.

여기서 "물 좀 가져다줘"처럼 잠깐 멈췄다**(await)** 나중에 이어서 할 수 있는 작업이 coroutine입니다.

📚 개념

- 이벤트 루프**(event loop)**: 비동기 작업들을 번갈아 실행하는 "단 하나의 관리자". asyncio 프로그램의 심장.
- **coroutine**(코루틴): `async def`로 정의한 함수. 중간에 `await`를 만나면 멈췄다가, 기다리던 일이 끝나면 그 자리부터 다시 이어 실행된다.
- **await**: "이건 시간이 걸리니, 끝날 때까지 나는 잠깐 양보할게"라는 표시. 이때 이벤트 루프는 다른 coroutine을 실행한다.

핵심 원리: 비동기는 **CPU**가 여러 개라서 빠른 게 아니라, "기다리는 시간(I/O 대기)"에 다른 일을 끼워 넣어서 전체 시간을 줄이는 것입니다. (스레드/멀티코어와 다른 개념 — 개념 6에서 다시)

🤔 왜 쓰나

- 없으면: pylint가 디스크에서 파일을 읽고 분석하는 동안 내 프로그램은 그냥 멈춰서 기다림 → 그 시간이 통째로 낭비
- 있으면: 그 대기 시간에 bandit, radon을 같이 시작 → 전체 시간이 "가장 느린 한 개"만큼으로 줄어듦

개념 2. asyncio.gather() — 여러 coroutine을 동시 실행

🍕 비유

배달 앱으로 짜장면(10분)과 탕수육(8분)을 동시에 주문하면 18분이 아니라 10분 만에 다 옵니다. `gather`는 "여러 주문을 한꺼번에 넣고, 전부 도착할 때까지 기다린 뒤, 받은 음식을 순서대로 정리해서 주는" 역할입니다.

📚 개념

`asyncio.gather(*coros)`는 넘겨준 여러 coroutine을 동시에 시작하고, 전부 끝날 때까지 기다린 뒤 결과를 넘긴 순서 그대로 리스트로 반환한다.

```
results = await asyncio.gather(coro_a, coro_b, coro_c)
# results[0]은 coro_a 결과, results[1]은 coro_b 결과 ... (완료 순서가 아니라 넘긴 순서)
```

🧐 왜 쓰나

- 없으면: pylint(30초) 끝나고 → bandit(30초) → radon(20초)... 순차로 = 80초. CLAUDE.md §6의 "에이전트 30초 타임아웃"을 넘겨서 실패
- 있으면: 셋을 동시에 → 가장 느린 30초만 기다림 → 타임아웃 안에 들어옴

개념 3. loop.run_in_executor() — 동기 함수를 비동기로 감싸기 ⭐ 핵심

🍕 비유

종업원(이벤트 루프)은 한 명뿐인데, `run_pylint`는 "끝날 때까지 자리를 안 비켜주는 고집 센 손님"입니다. 동기 `subprocess.run`은 결과가 나올 때까지 종업원을 30초간 붙잡아 둡니다. 그러면 종업원이 다른 테이블(다른 도구)을 못 받으니 "동시 실행"이 깨집니다.

해결책: 뒤편에 아르바이트생(작업자 스레드)들을 두고, 고집 센 손님은 알바에게 떠넘깁니다. 종업원은 "알바한테 맡겼으니 끝나면 알려줘" 하고 곧장 다른 테이블로 갑니다. 알바가 일하는 동안 종업원은 자유롭습니다.

`run_in_executor`가 바로 "동기 함수를 알바에게 떠넘기는" 도구입니다.

📚 개념

`loop.run_in_executor(executor, func, *args)`는 블로킹(자리를 안 비키는) 동기 함수를 별도의 스레드/프로세스(executor)에서 실행하고, 그 실행을 **await** 가능한 **coroutine**처럼 만들어준다.

- 첫 번째 인자 `executor`에 `None`을 주면 → 파이썬의 기본 **ThreadPoolExecutor** 사용
- `func`는 `()` 없이 함수 이름만 (호출이 아니라 "넘겨주기")
- 인자는 그 뒤에 나열: `run_in_executor(None, run_pylint, py_files)`

왜 이게 필요한가 (핵심 이유): > `subprocess.run`은 **blocking**(블로킹) 함수다. 즉 결과가 나올 때까지 이벤트 루프를 통째로 멈춘다. `gather`에 동기 함수를 그냥 넣으면 "동시"가 되지

않고 한 줄로 서버린다. `run_in_executor`로 다른 스레드에 보내야 이벤트 루프가 자유로워지고 진짜 병렬이 된다.

🧐 왜 쓰나

- 없으면: 기존 `run_pylint`, `run_bandit`을 전부 `async def` + `create_subprocess_exec`로 다시 짜야 함 → 이미 잘 동작하는 동기 함수 10개를 재작성하는 헛수고
- 있으면: `pylint_tool.py`, `bandit_tool.py`의 동기 함수를 한 줄도 안 고치고 그대로 병렬 실행 가능

```
# 이게 안 되는 이유 (gather에 동기 함수 결과를 그냥 넣음)
results = await asyncio.gather(run_pylint(files))  # run_pylint(files)가 먼저 끝까지 실행됨 → 동시성 깨짐, 타입도 안 맞음

# 올바른 방법 (동기 함수를 executor에 떠넘김)
loop = asyncio.get_event_loop()
results = await asyncio.gather(
    loop.run_in_executor(None, run_pylint, files),   # 알바 1에게
    loop.run_in_executor(None, run_bandit, files),   # 알바 2에게
)  # 두 알바가 동시에 일함 → 진짜 병렬
```

개념 4. asyncio.wait_for() — 타임아웃 적용 (CLAUDE.md §6 필수)

🍕 비유

요리사에게 "30분 안에 안 나오면 주문 취소할게"라고 시간 제한을 거는 것. 한 도구가 무한 루프에 빠지거나 거대한 파일로 영원히 안 끝나도, 30초가 지나면 강제로 끊고 넘어갑니다.

📚 개념

> `asyncio.wait_for(coro, timeout=30.0)`은 coroutine에 제한 시간을 씌운다. 시간 안에 끝나면 정상 결과를, 시간을 넘기면 `asyncio.TimeoutError`를 발생시킨다.

🧐 왜 쓰나

- 없으면: 도구 하나가 멈추면 전체 PR 리뷰가 영원히 끝나지 않음. CLAUDE.md §6은 "모든 분석 에이전트 30초 타임아웃"을 강제
- 있으면: 30초 넘은 도구만 잘라내고 나머지 결과로 진행

> ⚠️ run_in_executor의 함정: `wait_for`가 30초 후 `TimeoutError`를 내도, 이미 작업자 스레드에서 돌고 있는 **`subprocess.run`**은 자동으로 안 멈춥니다. 파이썬은 실행 중인 스레드를 강제로 죽일 수 없기 때문입니다. 그래서 동기 함수 안의 **`subprocess.run`**에도

`timeout=30`을 주는 2중 안전장치가 필요합니다. (실제로 `pylint_tool.py` 40~43줄을 보면 `subprocess.run(..., timeout=30)`이 이미 들어 있습니다.)

개념 5. return_exceptions=True — 하나 실패해도 나머지 수집

🍕 비유

5개 음식을 동시 주문했는데 탕수육 가게가 문을 닫았습니다. `return_exceptions=False`(기본값)면 "주문 전체 취소"가 됩니다. `True`면 "탕수육은 '품절' 메모로 받고, 나머지 4개는 정상 수령"합니다.

🧩 개념

`asyncio.gather(..., return_exceptions=True)`를 주면, 어떤 coroutine이 예외를 던져도 `gather` 전체가 중단되지 않는다. 그 자리에는 예외 객체 자체가 결과로 담겨서 돌아온다.

```
results = await asyncio.gather(*tasks, return_exceptions=True)
# 예: [ [pylint이슈들], TimeoutError(...), [bandit이슈들] ]
#         성공              실패(예외 객체)      성공
```

🤔 왜 쓰나

- 없으면: bandit이 미설치라 에러 1번 나면 pylint·radon 결과까지 전부 날아감 → 부분 결과조차 못 줌
- 있으면: 실패한 도구는 건너뛰고(`isinstance(r, Exception)`로 걸러냄) 나머지 이슈는 정상 보고 → CLAUDE.md §12 "도구 실패 시 빈 결과로 진행" 정책과 일치

개념 6. ThreadPoolExecutor vs ProcessPoolExecutor — 무엇을 쓰나

🍕 비유

- **ThreadPoolExecutor(스레드)**: 한 사무실 안의 여러 직원. 같은 책상(메모리)을 공유. 전화 받기/택배 기다리기처럼 기다리는 일**(I/O)** 에 강함.
- **ProcessPoolExecutor(프로세스)**: 아예 다른 사무실(독립 메모리). 무거운 계산을 진짜 동시에 여러 CPU로 돌릴 때 강함.

🧤 개념

구분	ThreadPoolExecutor	ProcessPoolExecutor
단위	스레드 (메모리 공유)	프로세스 (메모리 분리)
강한 작업	**I/O** 대기 (파일/네트워크/외부 명령 대기)	**CPU** 계산 (순수 파이썬 무거운 연산)
GIL 영향	받음 (파이썬 계산은 동시 안 됨)	안 받음 (진짜 병렬 계산)
비용	가벼움	무거움 (프로세스 생성·데이터 복사)

핵심: 파이썬에는 **GIL(Global Interpreter Lock)** 이 있어서 스레드로는 순수 파이썬 계산을 동시에 못 한다. 하지만 `subprocess.run`은 다르다.

🧐 우리는 어느 것? → **ThreadPoolExecutor** (즉 **`run_in_executor(None, ...)`**)
이유: > `subprocess.run`은 "외부 프로그램(**pylint.exe** 등)이 일하는 동안 그냥 기다리는" I/O 대기 작업이다. 실제 분석 계산은 별도 프로세스인 pylint가 하지, 내 파이썬이 하지 않는다. 따라서 GIL과 무관하고, 가벼운 ThreadPoolExecutor로 충분히 병렬이 된다. ProcessPoolExecutor는 불필요하게 무겁고, 함수·인자를 프로세스 간 복사(pickle)해야 하는 제약까지 생긴다.

`run_in_executor`의 첫 인자에 `None`을 주는 것 = "기본 ThreadPoolExecutor 써라"라는 뜻입니다. 그래서 우리 코드 패턴이 `loop.run_in_executor(None, run_pylint, py_files)`인 것입니다.

💻 실제 코드 패턴 분석 (Task 4-a 구현 형태)

```
async def run_tools_parallel(files: list[str]) -> list[dict]:
    loop = asyncio.get_event_loop()                    # ① 이벤트 루프 가져오기
    py_files = [f for f in files if f.endswith(".py")]  # 라우터로 .py만 추림

    tasks = [
        asyncio.wait_for(                              # ④ 30초 타임아웃 씌우기
            loop.run_in_executor(None, run_pylint, py_files),  # ③ 동기함수→스레드, None=ThreadPool
            timeout=30.0,
        ),
        asyncio.wait_for(
            loop.run_in_executor(None, run_bandit, py_files),
            timeout=30.0,
        ),
    ]
    results = await asyncio.gather(*tasks, return_exceptions=True)  # ②⑤ 동시실행+실패허용
```

```
    issues = []
    for r in results:
        if isinstance(r, Exception):                    # ⑤ 타임아웃/실패한 도구는 건너
뜀
            continue
        issues.extend(r)                                # 성공한 도구 이슈만 누적
    return issues
```

단계별 흐름

1. **`asyncio.get_event_loop()`**: 현재 돌고 있는 이벤트 루프(종업원)를 가져온다. `run_in_executor`를 호출하려면 루프가 필요.
2. **`loop.run_in_executor(None, run_pylint, py_files)`** (개념 3): 동기 함수 `run_pylint`를 기본 ThreadPoolExecutor에 떠넘긴다. `run_pylint(py_files)`처럼 호출하지 않고 함수와 인자를 따로 넘기는 점에 주의. 반환값은 await 가능한 객체.
3. **`asyncio.wait_for(..., timeout=30.0)`** (개념 4): 위 작업에 30초 제한을 씌운다. CLAUDE.md §6 준수.
4. **`asyncio.gather(*tasks, return_exceptions=True)`** (개념 2·5): 두 작업을 동시에 시작하고 둘 다 끝날 때까지 await. 하나가 실패해도 예외 객체로 받아 전체는 살린다.
5. 결과 정리 루프: `isinstance(r, Exception)`로 실패/타임아웃 도구를 걸러내고, 성공한 도구의 이슈만 `extend`로 모은다.

코드 학습 리뷰 (개선 기회)

- 잘 쓴 패턴: `return_exceptions=True` + `isinstance` 필터 → 도구 하나가 죽어도 부분 결과 보장. CLAUDE.md §12와 정확히 일치.
- 개선 포인트 **1**: `asyncio.get_event_loop()`는 파이썬 3.10+에서 비권장 경고가 날 수 있다. coroutine 안에서는 **`asyncio.get_running_loop()`** 가 더 안전하다. → 이를 "현재 실행 중인 루프만 명시적으로 가져오기"라고 한다.
- 개선 포인트 **2**: 실패를 `continue`로 조용히 넘기면 "왜 bandit 결과가 없지?"를 나중에 알 수 없다. 건너뛰기 전에 `logger.warning(f"tool failed: {r!r}")` 한 줄을 남기면 CLAUDE.md §10-④("에러는 삼키지 않는다") 정신에 더 부합한다.
- 개선 포인트 **3** (가독성): 더 간결한 최신 문법은 `asyncio.to_thread(run_pylint, py_files)` 다. 이건 `loop.run_in_executor(None, run_pylint, py_files)`와 같은 일을 한 줄로 한다 (파이썬 3.9+). 다만 executor를 직접 고를 수 없으므로, 기본 ThreadPool로 충분한 지금 상황엔 잘 맞는다.

3rd-Project-sub2 프로젝트에서

파일: - `dev2/tools/python/pylint_tool.py` → `run_pylint(files)` (동기, 그대로 재사용) - `dev2/tools/python/bandit_tool.py` → `run_bandit(files)` (동기, 그대로 재사용) - `dev2/`

`tools/router.py` → `detect_languages()`로 파일을 언어별로 그룹핑 (어떤 도구를 돌릴지 결정) - `dev2/nodes/` → 이번 **Task 4-a**에서 만들 **`run_tools`** 노드가 위 패턴으로 도구들을 병렬 실행

역할 흐름: 1. 라우터(`router.py`)가 PR diff의 파일을 언어별로 나눈다 (.py는 pylint+radon+bandit+semgrep). 2. `run_tools` 노드가 각 동기 도구 함수를 `run_in_executor`로 감싸 `gather`로 동시 실행한다. 3. 30초 타임아웃을 넘기거나 실패한 도구는 건너뛰고, 성공한 이슈만 모은다. 4. 모은 이슈를 Output Contract A(`schema.py`의 `AgentResult`, CLAUDE.md §5-A)로 변환해 Dev4에게 전달한다.

> CLAUDE.md §6 연결: 타임아웃 처리(`try/except asyncio.TimeoutError`)는 `async with AgentHarness()` 바깥에 둬야 `agent_traces.status`가 잘못 'success'로 찍히지 않는다. 이번 패턴은 `wait_for` + `return_exceptions=True`로 예외를 gather 결과 리스트에서 받으므로, harness 블록 안에서 예외를 삼키지 않도록 노드 구조를 짤 때 주의.

연관 개념

- **subprocess + create_subprocess_exec** → 도구를 처음부터 async로 짜는 다른 길 (`docs/learning/ai/subprocess_async.md`). 우리는 기존 동기 함수 재사용이라 run_in_executor 채택
- **asyncio.gather** → 동시 실행 + 결과 모으기 (`docs/learning/python/asyncio_gather.md`)
- **GIL (Global Interpreter Lock)** → 파이썬이 한 번에 한 스레드만 파이썬 코드를 실행하게 하는 잠금. subprocess는 외부 프로세스라 GIL과 무관해서 ThreadPool로 충분
- **asyncio.to_thread** → `run_in_executor(None, ...)`의 최신 단축 문법 (파이썬 3.9+)
- 좀비/미정리 작업 → run_in_executor의 스레드는 wait_for가 끊겨도 안 멈추므로 subprocess.run 자체의 timeout이 2중 안전장치

복습 퀴즈 (3문항)

1. **(이해)** `asyncio.gather`에 동기 함수 `run_pylint(files)`를 그냥 넣으면 왜 진짜 병렬이 되지 않는가? 이를 해결하려면 어떤 함수로 감싸야 하는가?
   - 답: `subprocess.run`이 들어있는 동기 함수는 blocking이라 이벤트 루프(종업원)를 끝까지 붙잡아 동시 실행을 막는다. `loop.run_in_executor(None, run_pylint, files)`로 작업자 스레드에 떠넘겨야 루프가 자유로워져 진짜 병렬이 된다.
2. **(적용)** `loop.run_in_executor(None, run_bandit, py_files)`에서 (a) 첫 인자 `None`은 무슨 뜻이며, (b) `run_bandit` 뒤에 `()`를 붙이지 않고 인자를 따로 넘기는 이유는?

- 답: (a) `None`은 "파이썬 기본 ThreadPoolExecutor를 써라"는 뜻이다. (b) executor는 함수를 "나중에 자기가 호출"해야 하므로, 즉시 실행되는 `run_bandit(py_files)` 호출 결과가 아니라 함수 객체 `run_bandit`와 인자 `py_files`를 분리해서 넘긴다.

3. (비교) subprocess로 외부 CLI 도구를 병렬 실행할 때 ThreadPoolExecutor와 ProcessPoolExecutor 중 무엇이 적합하며 그 이유는? GIL은 왜 문제가 안 되는가?
   - 답: ThreadPoolExecutor가 적합하다. subprocess.run은 외부 프로그램이 일하는 동안 기다리는 I/O 대기 작업이고, 실제 계산은 내 파이썬이 아니라 별도 프로세스(pylint 등)가 한다. 따라서 파이썬 코드를 동시에 못 돌리게 막는 GIL의 영향을 받지 않아, 가볍고 빠른 ThreadPool로 충분하다. ProcessPool은 불필요하게 무겁고 pickle 제약까지 생긴다.

PostgreSQL 인덱스 종류 학습 노트

날짜: 2026-05-22 복습 예정: D+1, D+3, D+7

한 줄 요약

책의 목차처럼, 데이터를 빠르게 찾기 위해 미리 만들어두는 지름길.

---

1. 비유

도서관에서 책 찾기: - 인덱스 없음: 책 1만 권을 처음부터 끝까지 다 뒤짐 → 느림 - 인덱스 있음: 목차 보고 바로 해당 페이지로 이동 → 빠름

인덱스 종류 = 목차 방식이 다름. 상황에 맞는 방식 선택해야 최고 효율.

---

2. 인덱스 종류별 설명

B-tree (기본값)

```
CREATE INDEX idx ON scan_executions (developer_id);
```

- 언제: 등호(=), 범위(>, <) 검색
- 비유: 알파벳순 정렬된 목차
- **Dev 2**: `WHERE developer_id = 'dev-001'`

복합 인덱스 (Composite)

```
-- equality 컬럼 먼저, range 컬럼 나중 — 순서 중요!
CREATE INDEX idx ON scan_executions (developer_id, scanned_at DESC);
```

- 언제: 두 컬럼을 같이 검색할 때
- 비유: "저자명 + 출판연도" 두 가지 기준으로 정렬된 목차
- **Dev 2**: `WHERE developer_id = ? AND scanned_at > ?`

BRIN (Block Range Index)

```
CREATE INDEX idx ON scan_executions USING brin (scanned_at);
```

- 언제: 시간순으로 쌓이는 데이터 (시계열)
- 비유: "1월 데이터는 앞쪽, 12월 데이터는 뒤쪽" 구역별 표시
- 장점: B-tree보다 크기 **90%** 작음
- **Dev 2**: `scanned_at >= date_trunc('month', now())`

GIN (Generalized Inverted Index)

```
CREATE INDEX idx ON scan_issues USING gin (similar_issues);
```

- 언제: JSONB, 배열, 전문 검색
- 비유: 책의 색인(Index) — "SQL이라는 단어가 나오는 페이지 목록"
- **Dev 2**: `WHERE similar_issues @> '[{"pr_id": "98"}]'`

부분 인덱스 **(Partial Index)**

```
-- WHERE 조건에 해당하는 행만 인덱싱
CREATE INDEX idx ON scan_issues (scan_id, rule_id)
    WHERE severity = 'Critical';
```

- 언제: 전체 데이터 중 일부만 자주 조회할 때
- 비유: "베스트셀러만 따로 모아놓은 특별 목차"
- 장점: 인덱스 크기 대폭 감소 (Critical은 전체의 5~10%)
- **Dev 2**: Critical 에스컬레이션 조회 전용

커버링 인덱스 **(Covering Index)**

```
CREATE INDEX idx ON scan_executions (developer_id)
    INCLUDE (id, scanned_at);
```

- 언제: SELECT에 필요한 컬럼을 인덱스에 포함시킬 때
- 비유: "목차에 페이지 번호뿐 아니라 챕터 요약도 포함"
- 장점: 테이블 본문 안 봐도 됨 → 더 빠름
- **Dev 2**: `/insights/developer` 쿼리에서 테이블 스캔 제거

---

3. 요약 표

인덱스	언제 쓰나	Dev 2 적용
B-tree	등호/범위 검색	developer_id 검색
복합	컬럼 2개 이상 조건	developer + 날짜
BRIN	시계열 데이터	scanned_at
GIN	JSONB/배열 검색	similar_issues
부분	조건부 소수 데이터	Critical 이슈만
커버링	SELECT 컬럼 포함	insights 쿼리

복습 퀴즈

1. Q: BRIN 인덱스가 B-tree보다 좋은 상황은? A: (시간순으로 순차 증가하는 시계열 데이터. 크기가 90% 작음)
2. Q: 복합 인덱스에서 equality 컬럼을 먼저 써야 하는 이유는? A: (equality가 범위를 좁혀주므로 range 검색 효율이 높아짐)
3. Q: 부분 인덱스(Partial Index)를 쓰는 이유는? A: (자주 조회하는 일부 데이터만 인덱싱 → 인덱스 크기 감소, 성능 향상)

UPSERT 학습 노트

날짜: 2026-05-22 복습 예정: D+1, D+3, D+7

한 줄 요약

있으면 업데이트, 없으면 삽입 — 두 작업을 하나로 합친 SQL.

---

1. 비유

출석부에 비유: - **INSERT**만: 이미 있는 학생을 또 추가 → 중복 에러 - **UPDATE**만: 없는 학생을 수정 → 아무 일도 안 일어남 - **UPSERT**: 있으면 수정, 없으면 추가 → 항상 올바른 결과

---

2. 개념

`INSERT ... ON CONFLICT ... DO UPDATE` 중복 키 충돌 시 에러 대신 UPDATE 실행.

---

3. 왜 쓰나

Dev 2에서 PR을 재스캔할 때: - 같은 `scan_id + agent` 조합의 집계가 이미 있을 수 있음 - 그냥 INSERT하면 UNIQUE 제약 위반 에러 - UPSERT로 안전하게 덮어쓰기

---

4. 코드 (Dev 2)

```
-- scan_summaries UPSERT
-- (scan_id, agent) 조합이 이미 있으면 수치 업데이트
INSERT INTO scan_summaries (scan_id, agent, critical, major, minor)
VALUES ($1, $2, $3, $4, $5)
ON CONFLICT (scan_id, agent)          -- 이 조합이 겹치면
DO UPDATE SET                          -- INSERT 대신 UPDATE 실행
    critical = EXCLUDED.critical,      -- EXCLUDED = 방금 넣으려던 값
    major    = EXCLUDED.major,
    minor    = EXCLUDED.minor;
```

```
# db.py — Python에서 UPSERT 사용
await s.execute(text("""
    INSERT INTO scan_summaries (scan_id, agent, critical, major, minor)
    VALUES (:scan_id, :agent, :critical, :major, :minor)
    ON CONFLICT (scan_id, agent)
    DO UPDATE SET
        critical = EXCLUDED.critical,
        major    = EXCLUDED.major,
        minor    = EXCLUDED.minor
"""), {"scan_id": scan_id, "agent": agent, **summary})
```

복습 퀴즈

1. Q: UPSERT가 필요한 상황은? A: (같은 데이터를 여러 번 삽입할 수 있을 때. 있으면 업데이트, 없으면 삽입)
2. Q: `EXCLUDED`는 무엇을 가리키나? A: (충돌이 발생했을 때 INSERT하려던 새 값)
3. Q: Dev 2에서 UPSERT를 쓰는 이유는? A: (PR 재스캔 시 scan_summaries 중복 삽입 방지)

Vector DB / 임베딩 학습 노트

날짜: 2026-05-22 복습 예정: D+1, D+3, D+7

한 줄 요약

텍스트를 숫자 배열로 변환해서 “비슷한 내용”을 수학적으로 찾을 수 있게 하는 DB.

---

1. 비유

일반 **DB** 검색 (LIKE 검색):

```
"SQL 인젝션" 검색 → 정확히 "SQL 인젝션"이라는 단어가 있는 것만 찾음
"SQL injection" → 못 찾음 (다른 단어니까)
```

Vector DB 검색:

```
"SQL 인젝션" 검색 → 의미가 비슷한 것 찾음
"SQL injection", "쿼리 삽입 취약점", "데이터베이스 공격" 도 찾아줌
```

왜냐하면 텍스트를 숫자 배열(벡터)로 변환했을 때 의미가 비슷하면 숫자도 비슷하기 때문.

---

2. 개념

임베딩(**Embedding**): 텍스트 → 숫자 배열로 변환하는 과정

```
"SQL 인젝션" → [0.12, -0.34, 0.89, 0.02, ...]  (1536개 숫자)
"SQL injection" → [0.11, -0.33, 0.91, 0.01, ...]  (비슷한 숫자!)
"오늘 날씨" → [0.92, 0.15, -0.44, 0.77, ...]     (전혀 다른 숫자)
```

유사도 계산: 두 벡터가 얼마나 비슷한지 = cosine similarity (0~1, 1에 가까울수록 유사)

Vector DB (Milvus): 이 벡터들을 저장하고 빠르게 검색해주는 DB

---

3. 왜 쓰나

Dev 2에서 유사 이슈 검색에 사용: - 없으면: “이 이슈 전에도 나왔나?” 를 알 수 없음 - 있으면: “SQL 인젝션 가능성” 이슈 → 과거에 비슷한 이슈 있었던 PR 번호 + 유사도 반환

---

4. 코드 (Dev 2)

```python
# vector_store/similarity.py
from openai import OpenAI
from pymilvus import MilvusClient

openai_client = OpenAI()

# Step 1: 텍스트 → 벡터 변환 (임베딩)
def _get_embedding(text: str) -> list[float]:
    response = openai_client.embeddings.create(
        model="text-embedding-3-small",  # OpenAI 임베딩 모델
        input=text
    )
    return response.data[0].embedding   # [0.12, -0.34, 0.89, ...] 숫자 배열 반환

# Step 2: Milvus에서 유사 벡터 검색
```

```python
def _search(message: str) -> list[dict]:
    client = MilvusClient(uri="http://localhost:19530")
    embedding = _get_embedding(message)  # 검색어도 벡터로 변환

    results = client.search(
        collection_name="scan_issues",
        data=[embedding],   # 이 벡터와 비슷한 것 찾아줘
        limit=3,            # 상위 3개만
    )

    # distance가 낮을수록 유사 → similarity = 1 - distance
    return [
        {"pr_id": hit.entity["pr_id"], "similarity": round(1 - hit.distance, 3)}
        for hit in results[0]
        if 1 - hit.distance > 0.85  # 85% 이상 유사한 것만
    ]
```

결과 예시:

```
[
  { "pr_id": "98", "date": "2026-04-10", "similarity": 0.94 },
  { "pr_id": "75", "date": "2026-03-22", "similarity": 0.89 }
]
```

→ “PR #98에서 94% 유사한 이슈가 있었음” 이라는 인사이트 제공

복습 퀴즈

1. Q: 임베딩(Embedding)이란? A: (텍스트를 숫자 배열(벡터)로 변환하는 것. 의미가 비슷하면 숫자도 비슷해짐)
2. Q: 일반 LIKE 검색과 Vector DB 검색의 차이는? A: (LIKE는 정확한 단어 일치. Vector DB는 의미 유사도로 검색)
3. Q: Dev 2에서 Milvus를 쓰는 목적은? A: (과거 유사 이슈 검색 – “이 패턴 전에도 나왔나?”)

나# 정적 분석 + 보안 스캔 도구 학습 노트

날짜: 2026-05-22 복습 예정: D+1, D+3, D+7

한 줄 요약

코드를 실행하지 않고 읽기만 해서 버그/보안 취약점을 찾는 도구들.

---

1. 비유

동적 분석 (실행해서 테스트): 음식을 실제로 먹어봐서 맛을 확인 정적 분석 (코드를 읽어서 검사): 레시피만 보고 "이 재료 조합은 위험하다"고 판단

정적 분석은 코드를 실행하지 않으므로 빠르고, PR 단계에서 바로 검사 가능.

---

2. Dev 2에서 사용하는 도구 5가지

Pylint — 코드 품질 검사

```
pylint src/auth.py --output-format=json
```

- 무엇을: 문법 오류, 미사용 변수, 네이밍 규칙 위반
- 결과: 각 이슈에 `message`, `line`, `symbol(rule_id)` 반환
- **Dev 2 severity**: 대부분 Minor (코드 스멜)

```
[{"type": "warning", "line": 10, "message": "Unused variable 'x'", "symbol":
"unused-variable"}]
```

---

radon — 순환복잡도 측정

```
radon cc src/auth.py -j
```

- 무엇을: 함수의 복잡도(분기 수) 측정. 숫자가 클수록 복잡
- 기준: 복잡도 > 10 → Major, > 20 → Critical
- **Dev 2** 의미: "이 함수는 테스트하기 어렵고 버그가 숨기 쉽다"

```
복잡도 5  → A (간단)
복잡도 15 → C (복잡, 리팩토링 권고)
복잡도 25 → F (매우 복잡, 즉시 분리)
```

Bandit — **Python** 보안 취약점 스캔

```
bandit -r src/ -f json
```

- 무엇을: SQL Injection, 하드코딩된 비밀번호, 안전하지 않은 함수 사용
- 결과: `issue_severity`(HIGH/MEDIUM/LOW) + `issue_confidence`
- **Dev 2** 매핑: HIGH → Critical, MEDIUM → Major, LOW → Minor

```
{
  "issue_severity": "HIGH",
  "issue_text": "Possible SQL injection via string-based query construction",
  "line_number": 42,
  "filename": "src/auth.py"
}
```

Semgrep — OWASP 룰셋 기반 패턴 검사

```
semgrep --config=p/owasp-top-ten src/ --json
```

- 무엇을: OWASP Top 10 취약점 패턴 (XSS, CSRF, 인증 미흡 등)
- **Bandit**과 차이: Bandit은 Python 전용, Semgrep은 언어 무관 + 커스텀 룰 가능
- **Dev 2** 의미: Bandit이 못 잡는 패턴까지 이중 검사

---

pip-audit — CVE 취약점 검사

```
pip-audit --format=json
```

- 무엇을: 설치된 패키지의 알려진 보안 취약점(CVE) 검사
- **CVSS** 점수 기준: 7.0+ → Critical, 4.0~6.9 → Major
- **Dev 2** 의미: "이 라이브러리 버전에 해킹 가능한 취약점이 있다"

```
{
  "name": "requests", "version": "2.25.0",
  "vulns": [{"id": "CVE-2023-32681", "fix_versions": ["2.31.0"]}]
}
```

---

3. 도구별 역할 요약

도구	검사 대상	Dev 2 severity 기준
Pylint	코드 스타일/품질	대부분 Minor
radon	함수 복잡도	>10 → Major, >20 → Critical
Bandit	Python 보안 패턴	HIGH→Critical, MEDIUM→Major
Semgrep	OWASP 패턴	룰셋 기준
pip-audit	패키지 CVE	CVSS 7.0+→Critical

복습 퀴즈

1. Q: 정적 분석이 동적 테스트보다 빠른 이유는? A: (코드를 실행하지 않고 읽기만 해서 검사하기 때문)
2. Q: Bandit과 Semgrep의 차이는? A: (Bandit은 Python 전용. Semgrep은 언어 무관 + OWASP 룰셋 기반 패턴 검사)
3. Q: pip-audit이 필요한 이유는? A: (코드 자체가 아닌 사용 중인 라이브러리의 CVE 취약점을 검사하기 위해)

학습 노트: 정적 분석 도구 (Pylint / Bandit / Semgrep / radon / ESLint)

작성일: 2026-05-24 관련 파일: `dev2/tools/python/`, `dev2/tools/javascript/`, `dev2/schema.py` 복습 예정: D+1, D+3, D+7 관련 **Task**: Task 3 — CLI 도구 + 다국어 라우터 구현

한 줄 요약

코드를 실행하지 않고 "읽기만" 해서 버그·보안 취약점·복잡도 문제를 찾아내는 검사 도구들. Dev2가 PR 코드를 자동 리뷰할 때 쓴다.

핵심 개념: 정적 분석이란?

(1) 비유

- 동적 분석(실행해서 테스트): 음식을 실제로 먹어보고 맛을 확인 → 정확하지만 시간이 걸리고, 안 먹어본 메뉴는 모름
- 정적 분석(코드를 읽어서 검사): 레시피만 보고 "설탕 1kg은 너무 많다", "이 재료 조합은 식중독 위험"이라고 판단 → 빠르고, 실행 전에 미리 잡음

(2) 명확한 개념

> 정적 분석(Static Analysis)은 프로그램을 실행하지 않고 소스 코드 자체를 분석해 문법 오류, 코드 스멜, 보안 취약점, 복잡도 등을 찾아내는 기법이다. "정적(static)"은 코드가 멈춰있는(실행 안 된) 상태에서 검사한다는 뜻.

(3) 왜 쓰나

- 없으면: 사람이 모든 PR을 한 줄씩 눈으로 읽어야 함 → 느리고, SQL 인젝션 같은 패턴을 놓침
- 있으면: PR이 올라오자마자 자동으로 수백 개 규칙을 검사 → 사람은 진짜 중요한 판단에만 집중

도구 1. Pylint — Python 코드 품질

개념

- 무엇을: 미사용 변수, 네이밍 규칙 위반, 잠재적 버그, 문법 오류
- 언어: Python 전용
- **Dev2 severity**: 대부분 Minor (코드 스멜), error 타입은 Critical

코드

```
pylint src/auth.py --output-format=json
```

```
[{"type": "warning", "line": 10, "message": "Unused variable 'x'", "symbol":
"unused-variable"}]
```

severity 매핑 (`dev2/tools/python/pylint_tool.py`):

```
{"error": "Critical", "warning": "Major", "convention": "Minor", "refactor":
"Minor"}
```

도구 2. Bandit — Python 보안 취약점

개념

- 무엇을: SQL 인젝션, 하드코딩된 비밀번호, `eval()`·`pickle` 같은 위험 함수 사용
- 언어: Python 전용
- **Dev2 severity**: HIGH → Critical, MEDIUM → Major, LOW → Minor (CLAUDE.md §4 Critical=즉시 Slack 에스컬레이션)

코드

```
bandit -r src/ -f json
```

```
{"issue_severity": "HIGH", "issue_text": "Possible SQL injection ...",
"line_number": 42, "filename": "src/auth.py"}
```

도구 3. Semgrep — 다국어 패턴 기반 보안 스캔

개념

- 무엇을: OWASP Top 10 패턴(XSS, CSRF, 인증 미흡 등)을 룰셋으로 검사
- 언어: 언어 무관 (Python, JS, Java, Go, C# 등) + 커스텀 룰 작성 가능
- **Bandit**과 차이: Bandit은 Python만, Semgrep은 여러 언어 + Bandit이 못 잡는 패턴까지 이중 검사

코드

```
semgrep --config=p/owasp-top-ten src/ --json
```

> Dev2 다국어 라우터에서의 의미: .py·.js·.cs 어떤 파일이든 Semgrep 하나로 보안 검사가 가능해, 언어별 도구가 없는 빈틈을 메우는 "공통 보안 그물" 역할.

도구 4. radon — Python 순환복잡도(Cyclomatic Complexity)

개념

- 무엇을: 함수의 분기(if/for/while) 개수를 세서 복잡도를 측정. 숫자가 클수록 이해·테스트가 어려움
- 언어: Python 전용
- **Dev2** 기준: 복잡도 > 10 ⟶ Major, > 20 ⟶ Critical

코드

```
radon cc src/auth.py -j
```

```
복잡도 5  → A 등급 (간단)
복잡도 15 → C 등급 (복잡, 리팩토링 권고)
복잡도 25 → F 등급 (매우 복잡, 즉시 분리)
```

의미: "이 함수는 너무 복잡해서 버그가 숨기 쉽고 테스트하기 어렵다"는 경고.

도구 5. ESLint — JavaScript/TypeScript 코드 품질

개념

- 무엇을: JS/TS의 미사용 변수, == vs === 오용, 위험한 패턴, 스타일 위반
- 언어: JavaScript / TypeScript 전용 (Python의 Pylint에 해당)
- **Dev2** 위치: `dev2/tools/javascript/eslint_tool.py`에서 호출

코드

```
eslint src/app.js --format json
```

```
[{"filePath": "src/app.js", "messages": [
  {"ruleId": "no-unused-vars", "severity": 2, "line": 5, "message": "'x' is
assigned but never used"}
]}]
```

severity 매핑: ESLint의 `severity: 2`(error) ⟶ Major, `severity: 1`(warning) ⟶ Minor

---

도구별 역할 한눈에 보기

도구	검사 대상	언어	Dev2 severity 기준	tools 경로
Pylint	코드 품질/스타일	Python	error→Critical, warning→Major	`tools/python/`
Bandit	보안 취약점	Python	HIGH→Critical, MEDIUM→Major	`tools/python/`
radon	순환복잡도	Python	>10→Major, >20→Critical	`tools/python/`
ESLint	코드 품질/스타일	JS/TS	error→Major, warning→Minor	`tools/javascript/`
Semgrep	OWASP 패턴 보안	다국어	룰셋 기준	`tools/common/`

패턴 정리: 언어별 "품질 도구"(Pylint/ESLint)는 따로 두고, "보안 도구"는 언어별(Bandit)과 공통(Semgrep)을 함께 써서 빈틈을 메운다.

3rd-Project-sub2 프로젝트에서

파일: `dev2/tools/python/`, `dev2/tools/javascript/`, `dev2/tools/common/`, 결과 변환은 `dev2/schema.py`의 `Issue`/`Summary`/`AgentOutput`

역할: 1. 각 도구를 subprocess로 실행해 JSON 원본 결과를 받음 (`Dev2State.static_raw`, `Dev2State.security_raw`) 2. 도구마다 다른 severity 표현(HIGH/error/숫자2)을 Dev2 공통 기준(Critical/Major/Minor)으로 통일 3. 최종적으로 Output Contract A(`AgentOutput`)로 묶어 Dev4 평가 에이전트에게 전달 4. Critical 보안 이슈가 1건이라도 나오면 CLAUDE.md §6에 따라 즉시 Slack 에스컬레이션

연관 개념

- **subprocess / asyncio** → 이 도구들을 코드에서 비동기로 실행 (`docs/learning/ai/subprocess_async.md`)
- 다국어 라우터 → 파일 확장자를 보고 어떤 도구를 돌릴지 결정 (`docs/learning/ai/multilang_router.md`)
- **Output Contract** → 도구별 제각각 결과를 Dev4가 이해하는 공통 형식으로 변환 (`schema.py` `AgentOutput`)
- **OWASP Top 10** → 가장 흔한 웹 보안 취약점 10가지 목록. Semgrep 룰셋의 기준

복습 퀴즈 (3문항)

1. (이해) 정적 분석이 동적 테스트보다 PR 단계에서 유리한 이유 두 가지는?
   - 답: (1) 코드를 실행하지 않고 읽기만 해서 빠르다. (2) 실행 경로를 타지 않은 코드(예: 예외 상황 분기)의 취약점도 패턴으로 찾아낼 수 있다.
2. (비교) Bandit과 Semgrep은 둘 다 보안 도구인데 왜 둘 다 쓰는가?
   - 답: Bandit은 Python 전용으로 깊게 검사하고, Semgrep은 언어 무관(JS·C# 등)이라 다국어 PR에서도 동작하며 OWASP 룰셋으로 Bandit이 못 잡는 패턴까지 이중 검사한다. 서로의 빈틈을 메운다.
3. (적용) Pylint는 `type: "error"`, Bandit은 `issue_severity: "HIGH"`, ESLint는 `severity: 2`로 결과를 준다. Dev2가 이들을 Dev4에 넘기기 전에 반드시 하는 일은?
   - 답: 도구마다 다른 severity 표현을 Dev2 공통 기준(Critical/Major/Minor)으로 통일 매핑하고, Output Contract A(`AgentOutput`) 형식으로 변환한다. 그래야 Dev4가 도구 종류와 무관하게 일관되게 3-Way 판정을 내릴 수 있다.

학습 노트: 다국어 라우터 패턴 + PR diff 파싱

작성일: 2026-05-24 관련 파일: `dev2/tools/` (python/javascript/csharp/html/css/common), `dev2/nodes/` 복습 예정: D+1, D+3, D+7 관련 **Task**: Task 3 — CLI 도구 + 다국어 라우터 구현

한 줄 요약

PR에 들어온 파일의 확장자(.py, .js, .cs 등)를 보고 "이 파일은 어떤 검사 도구로 보낼지" 자동으로 분류·전달하는 설계. 우체국이 우편물의 주소를 보고 지역별 트럭에 나눠 싣는 것과 같다.

개념 1. 다국어 라우터 패턴

(1) 비유

우체국에 들어온 택배 더미(PR 변경 파일들)를 분류원이 주소(확장자)를 보고 - 서울행 → 서울 트럭 (.py → Python 도구) - 부산행 → 부산 트럭 (.js → JavaScript 도구) - 제주행 → 제주배 (.cs → C# 도구) 로 나눠 싣습니다. 분류원은 택배 안에 뭐가 들었는지 몰라도, 주소만 보고 정확한 트럭으로 보냅니다.

(2) 명확한 개념

> 다국어 라우터(Router)는 입력(파일)을 일정한 규칙(확장자 → 언어)에 따라 적절한 처리기(언어별 도구 모음)로 분배하는 디스패치**(dispatch)** 패턴이다. 핵심은 "분배 로직"과

"실제 처리 로직"을 분리하는 것 – 라우터는 어디로 보낼지만 알고, 검사 방법은 각 언어 모듈이 책임진다.

핵심 원리: 개방-폐쇄 원칙**(OCP)**. 새 언어(예: Go)를 추가할 때 라우터 본체는 거의 건드리지 않고, 매핑 테이블에 한 줄과 새 도구 모듈만 추가하면 된다.

(3) 왜 쓰나

- 없으면: 한 함수 안에 `if .py: pylint... elif .js: eslint... elif .cs: ...` 이런 if-else가 끝없이 늘어남 → 언어 추가할 때마다 거대한 함수 수정 → 버그 위험 + 한 사람만 건드릴 수 있는 병목
- 있으면: 확장자→도구 매핑 테이블 한 곳만 관리 → 언어별 폴더가 독립적이라 팀원이 각자 다른 언어 도구를 동시에 개발 (CLAUDE.md §15 병렬 개발과 맞물림)

(4) 실제 코드

```
# dev2/tools/common/router.py
from dev2.tools.python import run_python_tools
from dev2.tools.javascript import run_javascript_tools
from dev2.tools.csharp import run_csharp_tools
from dev2.tools.html import run_html_tools
from dev2.tools.css import run_css_tools

# 확장자 → 언어 매핑 테이블 (새 언어는 여기에 한 줄 추가)
EXT_TO_LANG = {
    ".py": "python",
    ".js": "javascript", ".jsx": "javascript",
    ".ts": "javascript", ".tsx": "javascript",
    ".cs": "csharp",
    ".html": "html",
    ".css": "css",
}

# 언어 → 도구 실행기 매핑 (라우터의 핵심: dispatch 테이블)
LANG_TO_RUNNER = {
    "python": run_python_tools,
    "javascript": run_javascript_tools,
    "csharp": run_csharp_tools,
    "html": run_html_tools,
    "css": run_css_tools,
}

def route_files(files: list[str]) -> dict[str, list[str]]:
    """파일 목록을 언어별로 그룹핑. 예: {'python': ['a.py'], 'javascript':
['b.js']}"""
    grouped: dict[str, list[str]] = {}
    for f in files:
        ext = f[f.rfind("."):].lower() if "." in f else ""
```

```
        lang = EXT_TO_LANG.get(ext)
        if lang is None:
            continue   # 지원 안 하는 확장자(.md, .txt 등)는 건너뜀
        grouped.setdefault(lang, []).append(f)
    return grouped
```

```
# dev2/nodes/run_tools.py — 라우터 + 비동기 동시 실행 통합
import asyncio

async def run_all_tools(files: list[str]) -> list[dict]:
    grouped = route_files(files)   # 언어별로 묶고
    # 언어별 도구 실행기를 동시에 실행 (asyncio.gather)
    tasks = [LANG_TO_RUNNER[lang](langfiles) for lang, langfiles in
grouped.items()]
    results = await asyncio.gather(*tasks, return_exceptions=True)

    issues: list[dict] = []
    for r in results:
        if isinstance(r, Exception):
            continue   # 특정 언어 도구가 실패해도 나머지는 살림
        issues.extend(r)
    return issues
```

개념 2. PR diff 파싱 (unified diff → 변경 파일 목록)

(1) 비유

편집자가 빨간 펜으로 고친 원고를 받았을 때, "어떤 페이지를(파일) 몇 번째 줄을(line) 고쳤는지" 표시(+++, @@)만 보고 빠르게 찾아내는 것. 원고 전체를 다시 읽지 않고 표시만 따라갑니다.

(2) 명확한 개념

> GitHub Webhook이 보내는 `pr_diff`는 **unified diff** 형식의 텍스트다. 여기서 `+++ b/경로` 줄은 "변경된 파일 경로", `@@ -a,b +c,d @@` 줄(hunk 헤더)은 "몇 번째 줄이 바뀌었는지"를 알려준다. 라우터는 먼저 이 diff에서 변경된 파일 목록을 뽑아내야 어떤 언어 도구로 보낼지 결정할 수 있다.

unified diff 구조 읽는 법:

```
diff --git a/src/auth.py b/src/auth.py   ← 파일 시작
--- a/src/auth.py                          ← 변경 전 파일
+++ b/src/auth.py                          ← 변경 후 파일 (★ 여기서 파일 경로 추출)
@@ -10,3 +10,5 @@ def login():            ← hunk 헤더: +10,5 = 10번째 줄부터 5줄
-    query = "SELECT * FROM users"          ← 삭제된 줄 (앞에 -)
+    query = f"SELECT * FROM users WHERE id={uid}"  ← 추가된 줄 (앞에 +)
```

(3) 왜 쓰나

- 없으면: PR 전체 코드를 다 검사 → 안 바뀐 수백 개 파일까지 검사해 느리고, "이번 PR이 만든 문제"와 "원래 있던 문제"를 구분 못 함
- 있으면: 바뀐 파일·줄만 추려 검사 → 빠르고, 리뷰 코멘트를 정확히 바뀐 줄(`line`)에 달 수 있음 (Output Contract의 `line` 필드)

(4) 실제 코드

```
# dev2/tools/common/diff_parser.py
import re

def parse_changed_files(pr_diff: str) -> list[str]:
    """unified diff 에서 변경된 파일 경로 목록 추출."""
    files: list[str] = []
    for line in pr_diff.splitlines():
        # "+++ b/src/auth.py" 형태에서 경로만 뽑음
        if line.startswith("+++ b/"):
            path = line[len("+++ b/"):].strip()
            if path != "/dev/null":   # 삭제된 파일은 제외
                files.append(path)
    return files


def parse_changed_lines(pr_diff: str) -> dict[str, set[int]]:
    """파일별로 '추가/변경된 줄 번호' 집합 추출 (리뷰 코멘트 위치용)."""
    result: dict[str, set[int]] = {}
    current_file = None
    new_line_no = 0
    for line in pr_diff.splitlines():
        if line.startswith("+++ b/"):
            current_file = line[len("+++ b/"):].strip()
            result.setdefault(current_file, set())
        elif line.startswith("@@"):
            # "@@ -10,3 +10,5 @@" 에서 +10 (변경 후 시작 줄) 추출
            m = re.search(r"\+(\d+)", line)
            new_line_no = int(m.group(1)) if m else 0
        elif line.startswith("+") and not line.startswith("+++"):
            if current_file:
                result[current_file].add(new_line_no)   # 추가된 줄 기록
            new_line_no += 1
        elif not line.startswith("-"):
            new_line_no += 1   # 변경 없는 줄도 줄 번호는 증가
    return result
```

두 개념이 합쳐지는 전체 흐름 (단계별)

1. **Dev1**이 **PR diff** 전달: `PRReviewState.pr_diff`에 unified diff 텍스트가 담겨 옴

2. **diff** 파싱: `parse_changed_files()`로 변경 파일 목록 추출 → `["src/auth.py", "web/app.js"]`
3. 라우팅: `route_files()`로 언어별 그룹핑 → `{"python": ["src/auth.py"], "javascript": ["web/app.js"]}`
4. 비동기 실행: 언어별 도구 실행기를 `asyncio.gather`로 동시 실행 (subprocess로 pylint·eslint 등)
5. 결과 통합 + 위치 매핑: 각 이슈를 `parse_changed_lines()` 결과와 대조해 "이번 PR이 바꾼 줄의 이슈"만 추려 `line`에 정확히 매핑
6. **Output Contract** 변환: `schema.py`의 `AgentOutput`으로 묶어 Dev4에게 전달

3rd-Project-sub2 프로젝트에서

파일: `dev2/tools/common/router.py`(라우터), `dev2/tools/common/diff_parser.py`(diff 파싱), 언어별 `dev2/tools/python·javascript·csharp·html·css/`

역할: - `dev2/tools/` 폴더가 이미 언어별로 나뉘어 있는 이유가 바로 이 라우터 패턴 때문 — 각 폴더가 독립적이라 팀원이 언어별로 병렬 개발 가능 (CLAUDE.md §15) - 라우터는 "분배만" 담당하고 실제 검사는 각 언어 모듈이 책임 → Clean Architecture(CLAUDE.md §2)의 관심사 분리와 일치 - diff 파싱으로 변경 줄만 추려야 Output Contract의 `line` 필드가 정확해지고, Dev4가 GitHub에 정확한 위치로 코멘트를 달 수 있음

연관 개념

- **subprocess / asyncio** → 라우팅 후 각 언어 도구를 비동기 실행 (`docs/learning/ai/subprocess_async.md`)
- 정적 분석 도구 → 라우터가 보내는 목적지(`docs/learning/ai/static_analysis_tools.md`)
- 디스패치 테이블**(dispatch table)** → if-else 대신 딕셔너리로 분기를 관리하는 기법. 라우터의 핵심
- 개방-폐쇄 원칙**(OCP)** → 기존 코드 수정 없이 새 언어를 추가할 수 있는 설계 원칙
- **unified diff** → Git이 변경 내용을 표현하는 표준 텍스트 형식 (`+++`, `@@` 마커)

복습 퀴즈 (3문항)

1. **(이해)** 다국어 라우터에서 "분배 로직"과 "검사 로직"을 분리하면 좋은 점은? Go 언어를 새로 추가하는 상황으로 설명하라.
   - 답: 라우터(분배)는 어디로 보낼지만 알고 검사 방법은 각 언어 모듈이 책임지므로, Go 추가 시 라우터 본체는 거의 안 건드리고 `EXT_TO_LANG`에 `".go"` 한 줄, `LANG_TO_RUNNER`에 매핑 한 줄, 그리고 `tools/go/` 모듈만 추가하면 된다(개방-폐쇄 원칙). 거대한 if-else를 수정할 필요가 없어 버그 위험과 병목이 줄어든다.

2. **(적용)** unified diff에서 변경된 "파일 경로"는 어떤 줄에서, 변경된 "줄 번호"는 어떤 줄에서 얻는가?
   - 답: 파일 경로는 `+++ b/경로` 줄에서 얻고(`/dev/null`은 삭제 파일이라 제외), 변경 줄 번호는 `@@ -a,b +c,d @@` hunk 헤더의 `+c`(변경 후 시작 줄)부터 시작해 `+`로 시작하는 추가 줄을 세어 계산한다.
3. **(비교)** PR 전체 코드를 다 검사하는 것과, diff로 변경 파일/줄만 검사하는 것의 차이 두 가지는?
   - 답: (1) 속도 — 안 바뀐 수백 개 파일까지 검사하면 느리지만, 변경분만 검사하면 빠르다. (2) 정확성 — 변경 줄만 보면 "이번 PR이 만든 문제"와 "원래 있던 문제"를 구분할 수 있고, Output Contract의 `line` 필드로 GitHub의 정확한 위치에 리뷰 코멘트를 달 수 있다.

LangChain 체인 학습 노트

날짜: 2026-05-22 복습 예정: D+1, D+3, D+7

한 줄 요약

LLM에게 질문할 때 "질문 템플릿 → LLM 호출 → 결과 파싱"을 파이프라인처럼 연결하는 도구.

---

1. 비유

공문서 작성에 비유: 1. 양식지**(Prompt Template)**: 빈칸만 채우면 되는 공문서 양식 2. 담당자**(LLM)**: 양식 받아서 검토하고 결재 3. 결재 결과 파싱**(OutputParser)**: 결재문서에서 필요한 정보만 뽑아냄

```
[양식 작성] → [담당자 검토] → [결과 추출]
  Prompt      LLM 호출       OutputParser
```

---

2. 개념

> LangChain은 LLM 호출을 파이프라인**(**체인**)**으로 구성하는 프레임워크. | 연산자로 컴포넌트를 연결한다 (UNIX 파이프와 같은 개념).

주요 컴포넌트**:**

컴포넌트	역할	Dev 2 예시
`ChatPromptTemplate`	질문 템플릿 정의	"이슈: {message}, severity: {current_severity}..."
`ChatOpenAI`	OpenAI API 호출	gpt-4o-mini 호출
`StrOutputParser`	LLM 응답을 문자열로 파싱	JSON 문자열 추출

---

3. 왜 쓰나

- 없으면: `openai.chat.completions.create(...)` 매번 직접 호출 → 반복 코드 많고 관리 어려움
- 있으면: 체인으로 한 번 정의하면 재사용 가능, LangSmith 자동 추적

---

4. 코드 (Dev 2)

```python
# nodes/llm_judge.py
from langchain_openai import ChatOpenAI
from langchain_core.prompts import ChatPromptTemplate
from langchain_core.output_parsers import StrOutputParser

# 1) 템플릿 정의 (빈칸 있는 양식)
prompt = ChatPromptTemplate.from_messages([
    ("system", "당신은 시니어 보안 엔지니어입니다. JSON으로만 답하세요."),
    ("human", "이슈: {message}\n현재 severity: {current_severity}\n개발자 수준:
{developer_level}")
])

# 2) LLM 설정
llm = ChatOpenAI(model="gpt-4o-mini", temperature=0)

# 3) 체인 조립 (파이프 | 연산자로 연결)
chain = prompt | llm | StrOutputParser()

# 4) 실행
response = await chain.ainvoke({
    "message": "SQL 인젝션 가능성",
    "current_severity": "Major",
    "developer_level": "junior",
})
# response → '{"severity": "Critical", "suggestion": "파라미터화된 쿼리 사용하세
요."}'
```

핵심: | 연산자가 파이프처럼 왼쪽 출력 → 오른쪽 입력으로 전달

Minor 스킵 전략 (비용 최소화)

```
# Minor 이슈는 LLM 호출 안 함 → API 비용 절약
if issue["severity"] == "Minor":
    enriched.append(issue)  # 그대로 통과
    continue

# Major/Critical만 LLM 재판단
response = await chain.ainvoke(...)
```

복습 퀴즈

1. Q: LangChain 체인에서 | 연산자의 역할은? A: (왼쪽 컴포넌트의 출력을 오른쪽 컴포넌트의 입력으로 전달)
2. Q: Dev 2에서 LLM을 Minor 이슈에 호출하지 않는 이유는? A: (API 비용 최소화. Minor는 CLI 결과 그대로 사용)
3. Q: ChatPromptTemplate에서 `{message}` 같은 중괄호의 역할은? A: (나중에 실제 값으로 채워질 빈칸 변수)

LangGraph StateGraph 학습 노트

날짜: 2026-05-22 복습 예정: D+1, D+3, D+7

한 줄 요약

에이전트가 어떤 순서로 일할지 "흐름도"를 코드로 그리는 도구.

---

1. 비유

공장 컨베이어 벨트에 비유:

```
[부품 투입] → [조립] → [검수] → [포장] → [출하]
```

각 단계(노드)가 끝나야 다음 단계로 넘어가고, 현재 상태(부품이 어디까지 왔는지)를 **State**가 기억한다.

Dev 2에서:

```
[run_tools] → [llm_judge] → [vector_search] → [store]
  CLI 실행      LLM 판단       유사 이슈 검색     DB 저장
```

---

2. 개념

LangGraph는 에이전트의 상태**(State)**와 흐름**(Graph)**을 정의하는 프레임워크. 노드(Node) = 작업 단위, 엣지(Edge) = 순서, 상태(State) = 노드 간 공유 데이터.

핵심 **3**요소:

요소	역할	Dev 2 예시
State	노드 간 공유되는 데이터	`Dev2State` (static_raw, enriched 등)
Node	실제 작업을 수행하는 함수	`run_tools_node`, `llm_judge_node`
Edge	노드 간 실행 순서	`run_tools → llm_judge → vector_search → store`

---

3. 왜 쓰나

- 없으면: 함수를 직접 순서대로 호출 → 상태 관리가 복잡해짐, 중간에 멈추거나 재시작 불가
- 있으면: 상태를 자동으로 관리, 노드별로 독립 테스트 가능, LangSmith로 각 노드 추적 가능

---

4. 코드 (Dev 2)

```python
# graph.py — LangGraph StateGraph 조립
from langgraph.graph import StateGraph, END
from typing import TypedDict

# State 정의: 노드 간 공유되는 데이터 구조
class Dev2State(TypedDict):
    payload: dict        # 입력 (PR diff, 개발자 정보)
    static_raw: list     # Pylint/radon 원본 결과
    security_raw: list   # Bandit/pip-audit 원본 결과
    enriched: list       # LLM 판단 후 결과
    final_output: list   # Dev4로 전달할 최종 결과

def build_graph():
    g = StateGraph(Dev2State)

    # 노드 등록
    g.add_node("run_tools", run_tools_node)
    g.add_node("llm_judge", llm_judge_node)
    g.add_node("vector_search", vector_search_node)
```

```
    g.add_node("store", store_node)

    # 엣지 (순서) 정의
    g.set_entry_point("run_tools")          # 시작점
    g.add_edge("run_tools", "llm_judge")
    g.add_edge("llm_judge", "vector_search")
    g.add_edge("vector_search", "store")
    g.add_edge("store", END)                # 종료

    return g.compile()

# 실행
graph = build_graph()
result = await graph.ainvoke(initial_state)
```

흐름 시각화:

```
Dev2State 공유 →→→→
```

1. 비유

음식점에 비유하면: - 단일 에이전트: 주방장 1명이 주문받고, 요리하고, 서빙까지 다 함 → 느리고 실수 많음 - **MAS**: 홀 직원(주문) + 요리사(요리) + 서버(서빙)로 역할 분리 → 빠르고 전문적

Dev 2에서는: - **StaticAnalysisAgent**: Pylint/radon만 담당하는 직원 - **SecurityScanAgent**: Bandit/Semgrep만 담당하는 직원 - 둘이 동시에 일함 → 합쳐서 결과 반환

---

2. 개념

> 하나의 복잡한 작업을 여러 에이전트가 분담해서 처리하는 시스템. 각 에이전트는 독립적으로 동작하고, 결과를 합산하거나 순서대로 전달한다.

MAS 패턴 종류 (PRD에 나온 것들):

패턴	설명	Dev 2에서
Parallel Independent	동시 실행, 서로 독립	StaticAnalysis + SecurityScan 동시 실행
Supervisor	상위 에이전트가 하위를 지시	Dev1 오케스트레이터
Aggregator	여러 결과를 하나로 합침	Dev4 평가 에이전트
Cyclical Feedback Loop	결과가 다시 입력으로 돌아옴	Dev5 피드백 루프

---

3. 왜 쓰나

- 없으면: 하나의 에이전트가 정적 분석 + 보안 스캔 + LLM 판단을 순서대로 → 느림
- 있으면: 병렬로 동시 실행 → PRD 목표 "60초 이내 PR 코멘트" 달성 가능

---

4. 코드 (Dev 2)

```python
# agent.py — Dev 2의 MAS 진입점
import asyncio

async def run_dev2(payload: dict) -> list[dict]:
    # 두 에이전트를 동시에 실행 (Parallel Independent 패턴)
    results = await asyncio.gather(
        run_static_analysis(payload),  # StaticAnalysisAgent
```

```
        run_security_scan(payload),     # SecurityScanAgent
        return_exceptions=True          # 하나 실패해도 나머지 반환
    )
    return [r for r in results if not isinstance(r, Exception)]
```

복습 퀴즈

1. Q: MAS에서 Parallel Independent 패턴은 무엇인가? A: (두 에이전트가 서로 독립적으로 동시 실행되는 패턴)
2. Q: Dev 2에서 두 에이전트를 병렬 실행하는 Python 함수는? A: (asyncio.gather)
3. Q: 단일 에이전트 대신 MAS를 쓰는 이유는? A: (속도 향상 + 역할 분리로 유지보수 편리)

학습 노트: vector_search 노드 — pgVector RAG 검색 패턴

작성일: 2026-05-24 관련 파일: `dev2/nodes/vector_search.py` (구현 예정), `dev2/db.py` (참고), `dev2/nodes/llm_judge.py` (다음 노드), `dev2/prompts/severity_judge.py` (similar_cases 소비처), `dev2/schema.py` 복습 예정: D+1, D+3, D+7 관련 **Task**: vector_search 노드 — CLI raw 이슈를 pgVector로 과거 유사 이슈 검색 → enriched 보강

---

0. 이 노드가 무슨 일을 하나요? (전체 그림 먼저)

비유: 신입 의사 옆의 베테랑 간호사

CLI 도구(pylint, bandit, semgrep)는 환자(코드)를 보고 "열이 나네요"라고 증상만 말합니다. 하지만 베테랑 간호사는 그 증상을 듣자마자 "아, 작년에 비슷한 증상으로 온 환자가 **3**명 있었는데 다 같은 병이었어요" 라며 과거 진료 기록(과거 PR 이슈)을 꺼내옵니다.

`vector_search` 노드가 바로 이 베테랑 간호사입니다. CLI 도구가 찾아낸 이슈 메시지를 받아서, 우리 팀의 과거 이슈 데이터베이스(pgVector)에서 의미가 비슷한 과거 사례를 찾아옵니다. 그리고 그 사례를 이슈 옆에 붙여줍니다(`similar_issues`).

이렇게 붙여진 과거 사례는 다음 노드인 `llm_judge`가 "과거에 이런 건 **Major**였으니 이번 것도 **Major**겠네" 라고 더 정확하게 판단하는 데 쓰입니다. 이게 RAG(검색 증강 생성)의 핵심입니다.

파이프라인에서의 위치

```
run_tools_node          →  vector_search_node       →  llm_judge_node
(CLI 도구 원본 결과)        (과거 유사 이슈 검색·주입)     (RAG context로 재판단)
static_raw / security_raw  → enriched 필드 채움      → enriched의 similar_issues 사용
```

- 입력: `state["static_raw"]`, `state["security_raw"]` (run_tools 노드가 채운 CLI 원본)

- 출력: `{"enriched": [...]}` — 각 이슈에 `similar_issues` 필드를 붙인 목록 (`Dev2State.enriched`, schema.py:38)
- 다음 노드: `llm_judge_node`가 `enriched`를 읽어 `issue["similar_issues"]`를 RAG context로 사용 (llm_judge.py:170)

핵심: `vector_search`는 검색만 합니다 (LLM 호출 안 함). LLM 판단은 `llm_judge`의 일입니다. 단일 책임 원칙(SRP) — "검색하는 사람"과 "판단하는 사람"을 나눠야 각각 따로 테스트할 수 있습니다.

개념 1. pgVector 코사인 유사도 검색

비유

도서관에서 책을 찾을 때, 책마다 "주제 좌표"가 적힌 스티커가 붙어 있다고 상상하세요. 내가 찾는 주제의 좌표를 알면, 좌표가 가장 가까운 책들을 순서대로 꺼낼 수 있습니다. pgVector의 `<=>` 연산자가 바로 "두 좌표 사이 거리"를 재는 자입니다.

왜 **pgVector**인가? **(팀 표준, CLAUDE.md §9)**

> ⚠️ **Milvus** 금지. 우리 팀 Vector DB는 pgVector입니다.

pgVector는 별도 Vector DB 서버(Milvus, Pinecone 등)를 띄우지 않고, 이미 쓰고 있는 **PostgreSQL**에 확장**(extension)**만 추가해서 벡터 검색을 하는 방식입니다. - 인프라가 하나로 통일됨 — PostgreSQL 하나만 운영하면 됨 (Docker Compose 단순화) - 메타데이터(트랜잭션, JOIN)와 벡터를 같은 DB에서 다룰 수 있음 - `scan_issues` 같은 일반 테이블과 같은 곳에 있으니 데이터 정합성 관리가 쉬움

`<=>` 연산자: 코사인 거리

```
코사인 거리 = 1 - 코사인 유사도

거리 0.0  → 유사도 1.0 (완전히 같은 의미)
거리 0.3  → 유사도 0.7 (꽤 비슷함)
거리 1.0  → 유사도 0.0 (전혀 관계 없음)
```

핵심 혼동 주의: `<=>`가 돌려주는 값은 거리입니다. 작을수록 비슷합니다. 우리가 원하는 "유사도"는 `1 - 거리`로 계산해야 합니다.

```
SELECT
    chunk_id,
    content,
    metadata,
    1 - (embedding <=> $1::vector) AS similarity   -- 거리를 유사도로 변환
```

```
FROM chunk_embeddings
ORDER BY embedding <=> $1::vector                    -- 거리가 작은 순(=비슷한 순)
LIMIT $2;                                            -- top-k개만
```

- `$1::vector` : 검색 기준 벡터(내 이슈 메시지의 임베딩)를 vector 타입으로 캐스팅
- `ORDER BY embedding <=> $1::vector` : 거리 오름차순 = 가장 비슷한 것부터
- `LIMIT $2` : 상위 k개만 (top-k)

> 다른 연산자도 있음: <->(L2/유클리드 거리), <#>(내적). 우리는 의미 유사도용으로 코사인(<=>)을 씁니다. 텍스트 임베딩은 방향(의미)이 중요하고 크기는 덜 중요하기 때문입니다.

HNSW 인덱스: 왜 필요한가?

브루트포스(인덱스 없음): 검색할 때마다 테이블의 모든 행과 일일이 거리 계산합니다. 10만 개 이슈면 10만 번 계산. 정확하지만 느립니다 (O(n)).

HNSW(Hierarchical Navigable Small World): 미리 벡터들을 "고속도로 + 골목길" 같은 다층 그래프로 연결해 둡니다. 검색할 때 고속도로로 대략 가까운 동네까지 빠르게 이동한 뒤 골목길에서 정밀 탐색합니다. 전수조사 없이 근사적으로 가장 가까운 것을 찾습니다 (대략 O(log n)).

```
비유:
  브루트포스 = 서울 모든 집을 한 채씩 방문해서 목적지 찾기
  HNSW       = 고속도로 타고 강남까지 → 역삼동 → 골목 → 목적지
```

- 트레이드오프: HNSW는 100% 정확하지는 않음(근사). 하지만 보통 95%+ 재현율로 수십~수백 배 빠름. 우리 용도(과거 유사 사례 참고)에는 충분합니다.
- 인덱스 생성 예시 (코사인용 `vector_cosine_ops` 지정 필수):

```
CREATE INDEX idx_chunk_embeddings_hnsw
    ON chunk_embeddings
    USING hnsw (embedding vector_cosine_ops);
```

> ⚠️ 인덱스를 만들 때의 연산자 클래스(vector_cosine_ops)와 쿼리 연산자(<=>)가 짝이 맞아야 인덱스가 실제로 사용됩니다. L2용 인덱스를 만들고 <=>로 검색하면 인덱스를 안 타고 브루트포스로 돌아갑니다.

검색 결과 **0**건 처리 **(CLAUDE.md §12)**

처음 발견되는 패턴이면 과거 사례가 없을 수 있습니다. 이때 예외를 던지면 안 됩니다. 빈 리스트를 반환하고 그냥 진행합니다.

```
if not rows:
    logger.warning("vector_search: 유사 이슈 0건 — '%s'", message[:50])
    return []   # 예외 던지지 않음. similar_issues=[]로 다음 노드 진행
```

이 빈 리스트는 `severity_judge.py:62`에서 "유사 사례 없음 (처음 발견된 패턴)"으로 자연스럽게 표시됩니다.

개념 2. asyncpg — PostgreSQL 비동기 클라이언트

비유

psycopg2(동기)는 창구가 하나인 은행입니다. 한 손님(쿼리)이 일을 끝낼 때까지 뒤의 모든 손님이 줄 서서 기다립니다. asyncpg(비동기)는 번호표 시스템입니다. 한 손님이 서류 작성(DB 응답 대기) 하는 동안 직원은 다른 손님을 처리합니다. 기다리는 시간을 낭비하지 않습니다.

왜 **psycopg2** 대신 **asyncpg**인가?

우리 노드들은 전부 `async def` + `asyncio.gather`로 돌아갑니다 (run_tools.py, llm_judge.py 참고). 이 비동기 흐름 안에서 동기 라이브러리**(psycopg2)**를 쓰면 이벤트 루프 전체가 멈춥니다. DB 응답을 기다리는 동안 다른 이슈의 검색도 다 멈춰버립니다.

asyncpg는 `await`로 동작하므로, 한 이슈가 DB 응답을 기다리는 동안 다른 이슈 검색을 동시에 진행할 수 있습니다. asyncio와 궁합이 맞습니다.

> 참고: 현재 `dev2/db.py`는 SQLAlchemy + `asyncpg` 드라이버(`postgresql+asyncpg://`)를 씁니다. vector_search도 같은 흐름과 일관되게 비동기로 작성합니다. 아래는 asyncpg를 직접 쓰는 패턴이며, 팀이 SQLAlchemy로 통일하면 `text()` + `AsyncSessionLocal` 방식으로도 동일하게 구현 가능합니다.

`connect()` vs `create_pool()` 차이

	`asyncpg.connect()`	`asyncpg.create_pool()`
비유	쿼리마다 은행 문 열고 닫기	미리 직원 N명 대기시켜 둠
동작	매번 새 TCP 연결 생성·종료	연결 N개를 미리 만들어 재사용
비용	연결 생성 비용이 매번 발생 (느림)	연결 재사용으로 빠름
적합	일회성 스크립트, 마이그레이션	서버/노드처럼 반복 호출되는 곳

우리 vector_search 노드는 **PR**마다 여러 이슈를 반복 검색하므로 풀**(pool)**이 맞습니다. 다만 풀은 노드 안에서 매번 만들지 말고 앱 시작 시 한 번 만들어 외부에서 주입해야 합니다 (개념 4 참고).

`fetch` 패턴 + 벡터 파라미터 전달

asyncpg는 `$1, $2` 형태의 위치 파라미터를 씁니다 (`%s` 아님). 벡터는 리스트를 그대로 못 넣고, 문자열 **`'[...]'`**로 바꿔서 **`::vector`**로 캐스팅해야 합니다.

```
import asyncpg

async def search_similar(pool: asyncpg.Pool, query_vec: list[float], k: int) ->
list[dict]:
    # ⚠ 핵심: list[float] → "[0.1,0.2,...]" 문자열로 변환 후 ::vector 캐스팅
    vec_str = "[" + ",".join(str(x) for x in query_vec) + "]"

    sql = """
        SELECT chunk_id, content, metadata,
               1 - (embedding <=> $1::vector) AS similarity
        FROM chunk_embeddings
        ORDER BY embedding <=> $1::vector
        LIMIT $2
    """
    # 파라미터 바인딩만 사용 — f-string으로 SQL 조합 금지 (CLAUDE.md §6 보안 규칙)
    rows = await pool.fetch(sql, vec_str, k)   # asyncpg.Record 리스트 반환
    return [dict(r) for r in rows]             # dict로 변환해 다루기 쉽게
```

- `await pool.fetch(sql, *args)` : SELECT 결과 여러 행을 `Record` 리스트로 반환
- `pool.fetchrow(...)` : 한 행만, `pool.fetchval(...)` : 단일 값만
- `Record`는 dict처럼 `r["chunk_id"]`로 접근 가능. `dict(r)`로 일반 dict 변환 가능

> ⚠️ **f-string**으로 **SQL**을 조합하지 마세요 (CLAUDE.md §6). 반드시 `$1, $2` 파라미터 바인딩. 벡터 문자열도 SQL 본문이 아니라 파라미터 값으로 전달합니다(`$1::vector`). 이렇게 해야 SQL 인젝션이 차단됩니다.

개념 3. 임베딩 생성 패턴 (Mock 전환 포함)

비유

임베딩은 글을 좌표로 바꾸는 번역기입니다. "SQL 인젝션 취약점"이라는 문장을 [0.12, −0.34, ...] 같은 숫자 좌표로 번역합니다. 의미가 비슷한 문장은 좌표도 가까이 찍힙니다.

흐름: 이슈 메시지 → 벡터

```
issue["message"] = "사용자 입력이 SQL 쿼리에 직접 삽입됨"
        ↓ embed_text()
```

```
[0.12, -0.34, 0.89, ...]   ← 길이 = PGVECTOR_DIMENSION
        ↓ ::vector 캐스팅 후 pgVector 검색
과거 유사 이슈 목록
```

`USE_MOCK_LLM=true` 시 Mock 임베딩 (CLAUDE.md §6, §10-3)

개발 중에는 실제 임베딩 API를 부르면 안 됩니다(비용·속도·테스트 독립성). `llm_judge.py`가 Mock LLM을 쓰듯, vector_search도 **Mock** 임베딩을 반환합니다. 호출 측은 Mock인지 실제인지 구분하지 않아도 됩니다 — 환경변수만 바꾸면 됩니다.

```python
import os, hashlib

PGVECTOR_DIMENSION = int(os.getenv("PGVECTOR_DIMENSION", "4096"))  # CLAUDE.md §8: 4096 또는 768

def _mock_embedding(text: str) -> list[float]:
    """
    결정적(deterministic) Mock 임베딩 — 같은 텍스트는 항상 같은 벡터.
    해시 기반이라 테스트에서 재현 가능. 실제 의미는 없지만 차원/타입은 동일.
    """
    seed = int(hashlib.sha256(text.encode()).hexdigest(), 16)
    # 0~1 사이 값을 차원 수만큼 생성 (실제 임베딩과 길이/타입 동일)
    return [((seed >> (i % 64)) & 0xFFFF) / 65535.0 for i in range(PGVECTOR_DIMENSION)]

async def embed_text(text: str) -> list[float]:
    use_mock = os.getenv("USE_MOCK_LLM", "true").lower() == "true"
    if use_mock:
        return _mock_embedding(text)       # 실제 API 호출 없음
    # 실제: 팀 결정 임베딩 모델 (Upstage 4096 / Ollama 768) 호출
    # vec = await call_real_embedding_api(text)
    # assert len(vec) == PGVECTOR_DIMENSION  # 차원 검증
    # return vec
    raise NotImplementedError("실제 임베딩 연동은 Week 5 통합 시")
```

차원 통일 **(CLAUDE.md §8)** — 가장 흔한 첫 버그

임베딩 길이(차원)는 **3곳이 반드시 일치**해야 합니다: 1. `.env`의 `PGVECTOR_DIMENSION` (4096=Upstage / 768=Ollama) 2. DDL의 `vector(4096)` 또는 `vector(768)` 3. seed INSERT의 `::vector(4096)` 또는 `::vector(768)`

하나라도 다르면 첫 **INSERT**/검색에서 **"차원 불일치(dimension mismatch)"** 오류가 납니다. Mock 임베딩도 반드시 `PGVECTOR_DIMENSION` 길이로 생성해야 실제 전환 시 코드 수정이 없습니다.

개념 4. LangGraph 노드에서의 DB 연결 패턴

비유

노드 함수 안에서 매번 DB에 연결하는 것은 요리할 때마다 새 부엌을 짓는 것과 같습니다. 비효율적이고, 테스트할 때 진짜 부엌(DB)이 없으면 요리(테스트)를 못 합니다. 대신 이미 지어진 부엌(연결 풀)을 외부에서 빌려 쓰면 빠르고, 테스트 때는 장난감 부엌(Mock 풀)을 빌려주면 됩니다.

노드 내부 연결 vs 외부 주입(권장)

```
# ❌ 안티패턴: 노드가 직접 연결 생성 (매번 비용 발생 + 테스트 시 실제 DB 필요)
async def vector_search_node(state):
    conn = await asyncpg.connect(os.getenv("POSTGRES_URL"))  # 매 호출마다 연결
    ...

# ✅ 권장: 풀을 외부에서 주입 (functools.partial로 LangGraph에 바인딩)
async def vector_search_node(state: Dev2State, pool: asyncpg.Pool) -> dict:
    ...
    rows = await pool.fetch(sql, vec_str, k)
    ...

# 그래프 빌드 시 풀을 미리 만들어 주입
from functools import partial
pool = await asyncpg.create_pool(POSTGRES_URL)   # 앱 시작 시 1회
graph.add_node("vector_search", partial(vector_search_node, pool=pool))
```

왜 `POSTGRES_URL`을 환경변수로 외부화하나?

- 환경별 전환: 로컬은 `localhost:5432`, Docker는 `postgres:5432`, CI는 테스트 DB — 코드 변경 없이 **.env**만 바꾸면 됩니다 (CLAUDE.md §10-3).
- 비밀 노출 방지: 비밀번호를 코드에 하드코딩하면 git에 올라갑니다. 환경변수는 코드 밖에 둡니다.
- 현재 표준: `dev2/db.py:7`이 이미 `os.getenv("DATABASE_URL", ...)` 패턴을 씁니다. 같은 규칙을 따르세요.

테스트 시 DB를 Mock으로 교체

실제 DB 없이 단위 테스트가 통과해야 합니다 (CLAUDE.md §10-5). 풀을 외부 주입하면 가짜 풀을 넣기 쉽습니다.

```
from unittest.mock import AsyncMock, MagicMock

async def test_vector_search_returns_similar_issues():
    # 가짜 풀: fetch가 미리 정한 행을 반환
    fake_pool = MagicMock()
    fake_pool.fetch = AsyncMock(return_value=[
        {"chunk_id": "c-1", "content": "N+1 쿼리...", "metadata": {"pr_id":
```

```
"PR-001"}, "similarity": 0.82},
    ])
    state = {"static_raw": [{"message": "for 루프 안 쿼리", "severity": "Major"}],
"security_raw": []}

    result = await vector_search_node(state, pool=fake_pool)

    assert result["enriched"][0]["similar_issues"][0]["similarity"] == 0.82  # 실
제 DB 없이 통과

async def test_vector_search_empty_result():
    fake_pool = MagicMock()
    fake_pool.fetch = AsyncMock(return_value=[])      # 0건
    state = {"static_raw": [{"message": "처음 보는 패턴", "severity": "Minor"}],
"security_raw": []}
    result = await vector_search_node(state, pool=fake_pool)
    assert result["enriched"][0]["similar_issues"] == []   # 예외 없이 빈 리스트
(CLAUDE.md §12)
```

개념 5. RAG Retrieve 패턴 (전체 조립)

비유

RAG는 오픈북 시험입니다. LLM이 머릿속 지식만으로 답하지 않고(closed-book), 우리 팀의 과거 이슈 노트(pgVector)를 펼쳐보고(Retrieve) 답합니다(Generate). 그래서 "우리 팀 맥락"에 맞는 정확한 판단이 나옵니다.

Retrieve 흐름

```
1. issue["message"]            (CLI 도구가 찾은 이슈 메시지)
2.   → embed_text(message)     (임베딩: 텍스트 → 벡터, 개념 3)
3.   → pgVector <=> 검색        (코사인 유사도 top-k, 개념 1)
4.   → threshold 필터 (유사도 ≥ 0.7)
5.   → issue["similar_issues"] 주입
6.   → enriched에 담아 다음 노드(llm_judge)로
```

top-k를 **similar_issues**에 주입 — **severity_judge**가 기대하는 형식

`severity_judge.py:50`이 기대하는 각 항목 구조: `{"pr_id": str, "similarity": float, "message": str}`. vector_search는 검색 결과를 정확히 이 형식으로 변환해 넣어야 합니다.

```
SIMILARITY_THRESHOLD = 0.7   # 이 미만은 버림
TOP_K = 3

async def vector_search_node(state: Dev2State, pool: asyncpg.Pool) -> dict:
    raw_issues = state.get("static_raw", []) + state.get("security_raw", [])
    if not raw_issues:
        logger.warning("vector_search_node: raw 이슈 없음 — enriched 빈 리스트")
```

```
        return {"enriched": []}

    enriched: list[dict] = []
    for issue in raw_issues:
        message = issue.get("message", "")
        query_vec = await embed_text(message)              # 개념 3
        vec_str = "[" + ",".join(str(x) for x in query_vec) + "]"

        rows = await pool.fetch("""
            SELECT chunk_id, content, metadata,
                   1 - (embedding <=> $1::vector) AS similarity
            FROM chunk_embeddings
            ORDER BY embedding <=> $1::vector
            LIMIT $2
        """, vec_str, TOP_K)                                # 개념 1, 2

        # threshold 필터 + severity_judge가 기대하는 형식으로 변환
        similar = [
            {
                "pr_id": dict(r).get("metadata", {}).get("pr_id", "unknown"),
                "similarity": round(r["similarity"], 4),
                "message": r["content"],
            }
            for r in rows
            if r["similarity"] >= SIMILARITY_THRESHOLD       # 개념 5: 0.7 미만 버
림
        ]

        if not similar:
            logger.info("vector_search_node: '%s' 유사 이슈 0건(threshold)",
message[:40])

        enriched.append({**issue, "similar_issues": similar})  # 원본 보존 + 필드
추가

    logger.info("vector_search_node 완료: %d개 이슈 보강", len(enriched))
    return {"enriched": enriched}
```

왜 유사도 **0.7** 이상만 쓰나?

- 노이즈 차단: top-k는 무조건 k개를 돌려줍니다. 정말 비슷한 게 없어도 "그나마 덜 다른 것" k개를 줍니다. 유사도 0.3짜리(거의 무관한) 사례를 LLM에 넣으면 오히려 판단을 오염시킵니다.
- **RAG** 품질 = 검색 품질: 쓰레기를 넣으면 쓰레기가 나옵니다(garbage in, garbage out). threshold는 "충분히 관련 있는 것만" 거르는 안전장치입니다.
- **0.7**은 출발점: 절대 정답값이 아닙니다. 실제 데이터로 검색 품질을 보며 0.65~0.8 사이에서 튜닝합니다.

3rd-Project-sub2 프로젝트에서

파일: `dev2/nodes/vector_search.py` (구현 예정) 역할: `run_tools_node`가 만든 CLI raw 이슈(`static_raw`/`security_raw`)를 받아, 각 이슈 메시지를 임베딩 → pgVector 코사인 검색으로 과거 유사 이슈를 찾고, `similar_issues` 필드를 붙여 `enriched`로 반환한다. 이 `enriched`는 다음 노드 `llm_judge_node`가 RAG context로 사용한다.

연결 관계: - 앞 노드: `dev2/nodes/run_tools.py` → `static_raw`, `security_raw` 제공 - 뒤 노드: `dev2/nodes/llm_judge.py:170` → `issue["similar_issues"]` 소비 - 형식 계약: `dev2/prompts/severity_judge.py:50` → `{"pr_id", "similarity", "message"}` 기대 - DB 패턴 참고: `dev2/db.py` → 비동기 + `os.getenv` 외부화 + 파라미터 바인딩 - State 정의: `dev2/schema.py:38` → `enriched: list[dict]`

연관 개념

- 임베딩**(Embedding)** → 텍스트를 의미 좌표(숫자 배열)로 바꾸는 것. `docs/learning/vector/vectordb.md` 참고
- **HNSW** 인덱스 → 전수조사 없이 빠르게 근사 최근접 이웃을 찾는 그래프 인덱스
- 코사인 거리 **<=>** → 1 - 코사인 유사도. 작을수록 비슷함
- **asyncpg** 풀 → 미리 만든 연결 N개를 재사용하는 비동기 PostgreSQL 클라이언트
- **RAG** → 검색(Retrieve) + 생성(Generate). 외부 지식을 찾아 LLM에 넣어주는 패턴
- **Mock** 전환 → `USE_MOCK_LLM`으로 실제 API 없이 개발/테스트 (llm_judge.py와 동일 철학)

복습 퀴즈 **(3**문항**)**

1. **(**이해 확인**)** pgVector에서 `embedding <=> $1::vector`가 0.3을 반환했습니다. 이 두 벡터의 코사인 유사도는 얼마이고, “비슷한 편”인가요 “다른 편”인가요?

2. **(**적용**)** vector_search 노드가 어떤 이슈에 대해 과거 사례를 한 건도 못 찾았습니다(0건). CLAUDE.md §12에 따르면 이 노드는 무엇을 반환해야 하며, 다음 노드(llm_judge)와 severity_judge는 이를 어떻게 처리하나요?

3. **(**비교**)** 10만 개 이슈가 쌓인 운영 환경에서 HNSW 인덱스를 만들지 않고 `<=>` 검색을 하면 무슨 일이 생기나요? HNSW를 만들 때 `vector_cosine_ops`를 빠뜨리면 어떤 문제가 발생하나요?

학습 노트: 프롬프트 엔지니어링 5기법 (Few-shot / CoT / RAG 주입 / Structured Output / Domain Rules)

작성일: 2026-05-24 최종 수정: 2026-05-26 관련 파일: `dev2/prompts/severity_judge.py`, `dev2/schema.py`, `dev2/vector_store/` 복습 예정: D+1, D+3, D+7 관련 **Task**: Task 4-b — LLM severity 재판단 프롬프트 빌더 구현 + `/feature-dev` Issue 스키마 확장

한 줄 요약

CLI 도구(rule-based)가 기계적으로 매긴 severity를, LLM이 "예시 + 단계적 사고 + 과거 사례 + 정해진 답안 형식 + 도메인별 규칙"의 5가지 장치로 더 똑똑하게 다시 판단하도록 만드는 프롬프트 작성 기술.

큰 그림: 왜 LLM이 다시 판단해야 하나?

Task 3에서 한 것 **(rule-based)**

`dev2/tools/python/pylint_tool.py` 같은 CLI 도구는 고정된 표로 severity를 매긴다.

```
{"error": "Critical", "warning": "Major", "convention": "Minor"}
```

- 장점: 빠르고, 똑같은 입력엔 똑같은 결과 (결정론적)
- 한계: 맥락**(context)**을 모른다. "warning이면 무조건 Major"라고만 안다.
  - 같은 warning이라도 인증(auth) 코드면 더 위험할 수 있다 → 모름
  - 이 팀이 과거에 똑같은 이슈로 사고가 났는지 → 모름
  - junior 개발자에겐 친절한 설명이 필요한지 → 모름

Task 4-b에서 할 것 **(context-aware)**

LLM은 위 맥락을 읽고 "이 도구는 **Major**라고 했는데, 과거 **PR #12**에서 같은 패턴이 보안 사고로 이어졌으니 **Critical**로 올린다" 같은 판단을 할 수 있다. 그러려면 LLM에게 맥락을 잘 떠먹여 줘야 한다 → 그게 프롬프트 엔지니어링.

> rule-based(CLI 도구)는 속도와 일관성, context-aware(LLM)는 맥락 이해. Dev2는 둘을 합친다: 도구가 1차 필터 → LLM이 2차 재판단.

🍕 1. 비유 — 신입 검사관 교육시키기

당신이 새로 들어온 신입 품질검사관에게 "불량품 등급(심각/보통/경미)을 매기라"고 시킨다고 상상하자. 그냥 "알아서 판단해"라고 하면 사람마다 기준이 들쭉날쭉하다. 그래서 4가지 장치를 준다:

기법	비유	효과
Few-shot	“이전 검사관들이 매긴 정답 예시 3개를 보여줌”	판단 기준을 맞춤(calibration)
Chain-of-Thought	“1번 무슨 결함? 2번 과거랑 비교? 3번 등급 결정 — 순서대로 적으라고 체크리스트를 줌”	건너뛰지 않고 차근차근 사고
RAG 주입	“예전에 같은 결함으로 리콜 났던 사고 기록을 책상 위에 올려줌”	우리 회사 맥락 반영
Structured Output	“결과는 반드시 이 양식(빈칸 채우기)에 적으라고 함”	다음 사람이 바로 읽음
Domain Rules	“Frontend/Backend 담당 매뉴얼을 따로 주고, 맡은 분야 규칙대로 판단하라고 함”	언어/프레임워크 특화 정확도↑

이 5개가 없으면? 신입은 자기 마음대로, 매번 다른 형식으로, 회사 사정 모르고 판단한다. 이 5개가 있으면? 베테랑처럼 일관되고, 우리 회사에 맞고, 기계가 읽을 수 있는 결과를 낸다.

📚 2. 개념 — 5가지 기법의 기술적 정의

(1) Few-shot Prompting

프롬프트 안에 입력→출력 예시 몇 개**(few)**를 넣어, LLM이 그 패턴을 모방해 답하게 하는 기법. 예시 0개 = zero-shot, 1개 = one-shot, 여러 개 = few-shot. - 핵심 원리: LLM은 “주어진 예시의 패턴을 이어서 완성”하려는 성질이 있다. 정답 예시를 보면 그 기준에 맞춰 답을 낸다(=calibration, 기준 보정). - 보통 3~5개가 적당. 너무 많으면 토큰 낭비 + 한쪽으로 편향(bias).

(2) Chain-of-Thought (CoT)

LLM에게 최종 답만 말하지 말고, 추론 과정을 단계별로 풀어내라고 지시하는 기법. - 핵심 원리: LLM은 “한 번에 정답”보다 “중간 사고를 거치며” 답할 때 정확도가 올라간다. (사람이 암산보다 종이에 적으며 풀 때 덜 틀리는 것과 같다) - “단계별로 생각해봐(Let’s think step by step)” 또는 번호 매긴 분석 단계를 제시하는 방식.

(3) RAG Context 주입 (Retrieval-Augmented Generation)

외부 저장소(여기선 pgVector)에서 관련 정보를 검색**(Retrieval)**해 프롬프트에 끼워 넣어(Augmented), LLM이 그 정보를 근거로 답을 생성(Generation)하게 하는 기법. - 핵심 원리: LLM은 학습 시점 이후의 정보나 "우리 회사 내부 데이터"는 모른다. 검색해서 넣어주면 그걸 근거로 답한다. - Dev2에선: 현재 이슈와 의미적으로 유사한 과거 **PR** 이슈를 pgVector 코사인 유사도로 찾아 주입.

(4) Structured Output

LLM 응답을 자유 문장이 아니라 정해진 구조**(**주로 **JSON)**로 강제하는 기법. - 핵심 원리: 프로그램이 받아 쓰려면 형식이 일정해야 한다. "JSON으로만 응답"이라고 명시 + 키 이름/타입을 미리 지정. - 받은 뒤엔 반드시 검증**(validation)** 해야 한다 → `dev2/schema.py`의 Pydantic `Issue` 모델로 검증. - 현재 출력 필드 **(5**개**)**: `severity`, `issue_type`, `suggestion`, `resolution_example`, `rationale` - `issue_type`: `"Bug" | "Vulnerability" | "Code Smell"` — 이슈 성격 분류 - `resolution_example`: `"A 말고 B 써라"` 형태의 직관적 한 줄 교정 코드

(5) Domain Rules (도메인별 규칙 주입**)**

분석 대상 언어/프레임워크에 따라 다른 룰셋을 프롬프트에 주입하는 기법. - 핵심 원리: `eval()` 사용은 Python에서 경고지만 JavaScript에서는 XSS의 직접 원인이다. 같은 이슈도 도메인마다 severity가 달라진다. 파일 확장자와 `tech_stack`으로 FE/BE를 자동 판별 후 해당 규칙 섹션만 주입한다. - **Frontend Rules** (`_FRONTEND_RULES`): `eval()`, `dangerouslySetInnerHTML`, `var` 선언 등 JS/TS/React 특화 - **Backend Rules** (`_BACKEND_RULES`): SQL f-string, 하드코딩 API 키, `== None`, N+1 쿼리, `with` 구문 누락 등 Python/FastAPI 특화 - 도메인 감지 우선순위: 파일 확장자(`.js/.ts/.jsx/.tsx`) → `tech_stack` 리스트 (react/vue/svelte 포함 여부)

🧐 3. 왜 이 프로젝트에서 각 기법이 필요한가

기법	이 프로젝트에서의 구체적 이유
Few-shot	"SQL 미처리=Critical, 복잡도15=Major, 네이밍=Minor"처럼 **Dev2**의 **severity** 기준을 **LLM**에 학습시킨다. 안 넣으면 LLM이 자기 멋대로 등급을 매겨 도구 결과와 충돌.
CoT	severity는 단순 분류가 아니라 "패턴 파악→과거 비교→등급→레벨별 조언→한 줄 교정"의 **5**단계 판단이다. 단계를 강제해야 LLM이 과거 사례 비교를 건너뛰지 않는다.
RAG 주입	LLM은 우리 팀 과거 **PR** 이력을 모른다. pgVector로 찾은 `similar_cases`를 넣어줘야 "예전에 이걸로 사고났음 → 등급 상향" 같은 팀 맞춤 판단이 가능. CLAUDE.md §4 Dev1의 KMS 검색 결과를 활용하는 지점.
Structured Output	LLM 출력은 다음 단계(Dev4 평가, `dev2/schema.py` `Issue` 모델)로 넘어간다. JSON이 깨지면 파이프라인 전체가 멈춘다. CLAUDE.md §12 "LLM 응답이 잘못된 JSON" 처리 규칙과 직결. 현재 5개 필드(`severity`, `issue_type`, `suggestion`, `resolution_example`, `rationale`).
Domain Rules	Python `eval()`과 JS `eval()`은 위험도가 다르다. 파일 확장자+`tech_stack`으로 FE/BE를 자동 판별해 언어별 규칙을 동적으로 주입, 도메인 특화 정확도를 높인다.

한 줄 요약: **CLI** 도구는 **"규칙"**만 안다. **LLM**은 **"맥락"**을 더해야 가치가 생긴다. 그 맥락을 떠먹여 주는 5가지 숟가락이 이 5기법이다.

💻 4. 실제 코드 분석 (`dev2/prompts/severity_judge.py`)

전체 구조

이 함수는 LLM을 호출하지 않는다. "완성된 프롬프트 문자열"만 만들어 반환한다. (LLM 호출은 다른 노드에서, 개발 중엔 `USE_MOCK_LLM=true`로 Mock 처리 — CLAUDE.md §6)

부분별 역할

(A) Few-shot 블록 — 판단 기준 보정

```
FEW_SHOT_EXAMPLES = """
[판단 예시]
예시 1) SQL 파라미터 미처리 → Critical (입력값 직접 쿼리 삽입, 즉시 수정)
예시 2) 순환복잡도 15 → Major (리팩토링 권고, 테스트 커버리지 확인)
```

```
예시 3) 변수명 snake_case 위반 → Minor (코드 스멜, 다음 PR에서 수정)
"""
```

- 모듈 상단 상수로 분리 → 재사용 + 토큰 절약(매번 새로 안 만듦).
- 3개 예시가 각각 Critical/Major/Minor를 하나씩 → 3등급 기준을 골고루 보여줌(편향 방지).

(B) RAG context 조립 — 검색 결과를 문자열로

```
cases_text = "\n".join(
    f"- PR #{c['pr_id']} (유사도 {c['similarity']}): {c.get('message','')}"
    for c in similar_cases
) if similar_cases else "유사 사례 없음"
```

- `similar_cases`는 pgVector(`dev2/vector_store/`) 코사인 유사도 검색 결과.
- `c.get('message','')` → 키가 없어도 안 터지게 방어(빈 문자열 기본값).
- 빈 리스트 처리: 검색 0건이면 "유사 사례 없음"으로 대체 → CLAUDE.md §12 "pgVector 검색 결과 0건" 규칙 준수. (예외를 던지지 않고 진행)

(C) CoT 블록 — 단계별 사고 유도

```
[분석 단계 — Chain-of-Thought]
1단계) 현재 이슈의 패턴이 무엇인가? (SQL 인젝션? 복잡도? 코드 스멜?)
2단계) 과거 유사 사례와 비교하면 어떤 차이가 있나?
3단계) 최종 severity(Critical|Major|Minor)와 issue_type(Bug|Vulnerability|Code
Smell)을 결정하라
4단계) {developer_level} 수준의 개발자에게 맞춤 개선 방법을 제시하라
5단계) "A 말고 B 써라" 형태의 한 줄 resolution_example을 작성하라
```

- 번호 매긴 **5**단계로 LLM이 추론을 건너뛰지 못하게 함 (4단계 → 5단계로 확장).
- 3단계에 `issue_type` 결정 추가 → Bug/Vulnerability/Code Smell 분류 강제.
- 5단계에 `resolution_example` 작성 → "A 말고 B 써라" 형태의 한 줄 교정 코드.
- 4번에 `developer_level` 주입 → junior면 친절히, senior면 간결히 (CLAUDE.md §5-C 개인화 피드백).

(D) Structured Output 지시 — **JSON** 강제

```
JSON으로만 응답:
{{"severity": "Critical|Major|Minor", "issue_type": "Bug|Vulnerability|Code
Smell", "suggestion": "구체적 수정 방법 (코드 예시 포함)", "resolution_example": "A 말
고 B 써라 형태의 한 줄 교정 코드", "rationale": "판단 근거 한 줄"}}
```

- `{{ }}` 더블 중괄호 → f-string에서 리터럴 중괄호 출력 (`{ }` 한 개면 변수 치환으로 오인돼 에러).
- `severity` 값을 `Critical|Major|Minor`로 제한 → 받은 뒤 `dev2/schema.py`의 `Issue.severity` Literal과 매칭.
- `issue_type` 추가: `Bug|Vulnerability|Code Smell` — 이슈 성격을 3가지로 분류.

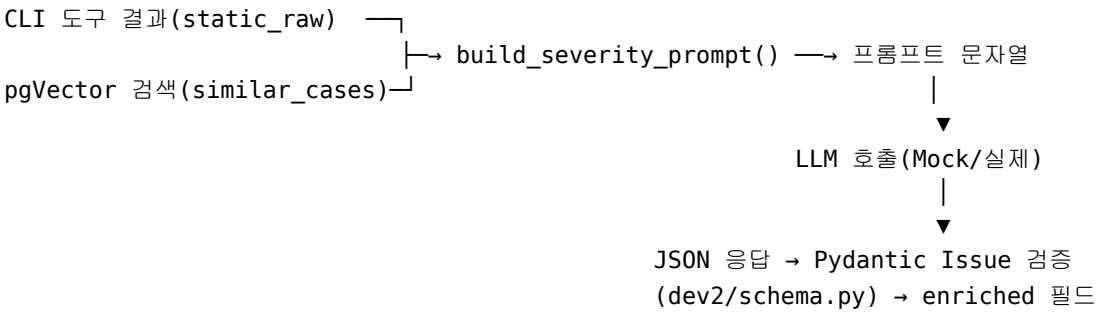
- `resolution_example` 추가: "A 말고 B 써라" 한 줄 교정 코드 → 개발자가 즉시 적용 가능한 구체적 안내.
- `rationale`(판단 근거)을 요구 → CoT의 결론을 명시적으로 받아 LangSmith 추적/디버깅에 활용.

(E) Domain Rules 주입 — 도메인별 규칙 선택

```
# 파일 확장자 + tech_stack으로 FE/BE 자동 판별
ext = file_name.rsplit(".", 1)[-1].lower() if "." in file_name else ""
fe_exts = {"js", "jsx", "ts", "tsx", "vue", "svelte"}
fe_stacks = {"javascript", "typescript", "react", "vue", "svelte", "nextjs"}
is_frontend = ext in fe_exts or bool(fe_stacks & {s.lower() for s in tech_stack})
domain_rules = _FRONTEND_RULES if is_frontend else _BACKEND_RULES
```

- 파일 확장자 우선 → `tech_stack`으로 보완 (확장자 없는 경우 대비).
- `_FRONTEND_RULES`: `eval()`, `dangerouslySetInnerHTML`, `var` 선언 등 JS/TS/React 특화 위험 패턴.
- `_BACKEND_RULES`: SQL f-string 조합, 하드코딩 API 키, `== None`, N+1 쿼리, `with` 구문 누락 등 Python/FastAPI 특화.
- 왜 따로 주입하나: `eval()`은 Python에서 경고 수준이지만 JS에서는 XSS 직접 원인(Critical). 같은 이슈도 도메인마다 severity가 다르다.

데이터 흐름 한눈에



3rd-Project-sub2 프로젝트에서

파일: `dev2/prompts/severity_judge.py` 역할: Task 3의 CLI 도구(rule-based)가 1차로 매긴 severity를, LLM이 과거 사례까지 보고 2차 재판단하도록 만드는 프롬프트 빌더. 출력은 `Dev2State.enriched`("pgVector 유사 이슈 + LLM 판단 결과")에 담겨, 최종적으로 `Issue` 모델로 검증돼 Output Contract A(`dev2/schema.py` `AgentOutput`)로 Dev4에 전달된다.

연결 관계: - 입력 `similar_cases` ← `dev2/vector_store/` (pgVector 코사인 검색) - 출력 JSON → `dev2/schema.py` `Issue` 모델로 검증 - JSON 파싱 실패 시 → CLAUDE.md §12: 재시도 1회 후 `{"issues": [], "error": "parse_failed"}`

⚠️ 흔한 실수 / 주의점

1. **f-string** 중괄호 — JSON 예시의 `{ }`는 반드시 `{{ }}`로. 한 개면 `KeyError` 또는 변수 치환 오류.
2. **JSON**을 믿지 마라 — LLM이 `json ...` 코드블록으로 감싸거나 설명을 덧붙이는 경우가 있다. 받은 뒤 반드시 파싱+Pydantic 검증, 실패 시 §12 규칙대로 재시도.
3. **Few-shot** 편향 — 예시를 Critical만 3개 넣으면 LLM이 전부 Critical로 매긴다. 등급을 골고루.
4. **RAG 0**건 미처리 — `similar_cases`가 빈 리스트일 때 "유사 사례 없음"으로 안 바꾸면 프롬프트에 어색한 빈칸이 생긴다.
5. 프롬프트 안에서 **LLM** 호출 금지 — 이 함수는 문자열만 만든다. 호출은 노드 책임(테스트 용이성↑).

복습 퀴즈 (3문항)

1. **(이해)** CLI 도구의 severity 매핑(rule-based)과 LLM 재판단(context-aware)의 가장 큰 차이는 무엇이며, Dev2는 왜 둘을 같이 쓰는가?
2. **(적용)** `similar_cases`가 빈 리스트로 들어왔다. 코드의 어느 부분이 이를 처리하며, 만약 그 처리가 없다면 프롬프트와 파이프라인에 각각 무슨 문제가 생기는가? (힌트: CLAUDE.md §12)
3. **(비교)** Few-shot과 RAG 주입은 둘 다 "프롬프트에 예시/정보를 넣는다"는 점이 비슷하다. 두 기법이 무엇을 넣는지, 왜 넣는지가 어떻게 다른가?

연관 개념

- **pgVector** / 코사인 유사도 → `docs/learning/vector/vectordb.md` (similar_cases를 만드는 검색)
- 정적 분석 도구 → `docs/learning/ai/static_analysis_tools.md` (재판단 대상인 1차 severity 생성)
- **Pydantic** 검증 → `dev2/schema.py` `Issue` 모델 (Structured Output을 받아 검증)
- **LangSmith** 추적 → CLAUDE.md §3 (rationale 필드가 디버깅에 활용됨)

학습 노트: llm_judge 노드 — LLM severity 재판단 패턴

작성일: 2026-05-24 관련 파일: `dev2/nodes/llm_judge.py` (구현 예정), `dev2/prompts/severity_judge.py` (완성됨), `dev2/nodes/run_tools.py` (참고), `dev2/schema.py` 복습 예정: D+1, D+3, D+7 관련 **Task**: Task 5 — LLM severity 재판단 LangGraph 노드 구현

---

0. 이 노드가 무슨 일을 하나요? (전체 그림 먼저)

비유: 1차 채점 후 선생님의 재검토

CLI 도구(pylint, bandit, semgrep)는 빠른 자동 채점기입니다. 규칙에 맞으면 기계적으로 "이건 Critical, 저건 Minor" 라벨을 붙입니다. 그런데 자동 채점기는 문맥을 모릅니다. 예를 들어 `eval()` 사용을 무조건 Critical로 찍지만, 실제로는 테스트 코드 안의 안전한 사용일 수도 있죠.

`llm_judge` 노드는 경력 많은 선생님이 자동 채점 결과를 한 번 더 검토하는 역할입니다. 과거 우리 팀의 비슷한 사례(pgVector 검색 결과)를 참고해서, "이건 도구가 과하게 잡았네, Minor로 내리자" 또는 "이건 도구가 놓쳤네, Major로 올리자" 라고 등급을 재조정합니다.

파이프라인에서의 위치

```
run_tools_node          →  llm_judge_node          →  (Dev4로 전달)
(CLI 도구 원본 결과)        (LLM이 severity 재판단)
static_raw / security_raw  → enriched 필드 채움
```

- 입력: `state["static_raw"]`, `state["security_raw"]` (run_tools 노드가 채운 CLI 원본)
- 출력: `{"enriched": [...]}` — LLM이 재판단한 결과 (`Dev2State.enriched` 필드, schema.py:38)
- **LangGraph**가 이 **dict**를 **state**에 자동 **merge** 합니다 (run_tools_node와 동일한 반환 규칙)

> 핵심 분리: `severity_judge.py`는 프롬프트 문자열만 만듭니다 (LLM 호출 안 함). `llm_judge.py`는 그 프롬프트를 받아 실제 **LLM**을 호출하고 응답을 파싱합니다. 이렇게 나누면 프롬프트 로직은 LLM/API 없이 순수 Python으로 테스트할 수 있습니다.

개념 1. LangChain + Claude 통합

비유

`ChatAnthropic`는 **Claude**에게 전화를 거는 전화기입니다. 전화기 모델(`model=`)을 고르고, 할 말(`messages`)을 정한 뒤 통화 버튼(`invoke`)을 누르면, 상대방 대답(`.content`)이 돌아옵니다.

개념

`langchain_anthropic.ChatAnthropic`는 LangChain이 제공하는 Claude 전용 채팅 모델 래퍼입니다. LangChain의 표준 인터페이스(`invoke` / `ainvoke`)를 따르기 때문에, LangGraph 노드 안에서 다른 LangChain 컴포넌트와 똑같은 방식으로 쓸 수 있습니다.

왜 **LangChain** 래퍼를 쓰나? **(anthropic SDK** 직접 호출 대신**)**

- **LangSmith** 자동 추적: config로 callbacks를 주입하면 모든 호출이 LangSmith에 자동 기록됩니다 (CLAUDE.md §3 "예외 없음"). anthropic SDK를 직접 쓰면 이 추적을 수동으로 붙여야 합니다.
- 표준 메시지 타입: `HumanMessage`, `SystemMessage` 같은 공통 타입을 써서 다른 LangChain 코드와 호환됩니다.
- 모델 교체 용이: 나중에 `claude-sonnet-4-6` ↔ `claude-haiku-4-5` 같은 모델 라우팅(개념 3)을 깔끔하게 할 수 있습니다.

실제 코드

```
from langchain_anthropic import ChatAnthropic
from langchain_core.messages import HumanMessage

# 모델 인스턴스 생성 — model 이름은 팀 표준만 사용 (CLAUDE.md §9)
llm = ChatAnthropic(
    model="claude-sonnet-4-6",   # ⚠ OpenAI 금지. 팀 표준 Claude만 허용
    temperature=0,               # severity 판단은 일관성이 중요 → 0 (창의성 불필요)
    max_tokens=1024,
)

# 동기 호출
response = llm.invoke([HumanMessage(content=prompt)])
text = response.content          # ← 응답 텍스트는 .content 로 추출
```

⚠️ 팀 규칙 **(CLAUDE.md §9)**: LLM은 `claude-sonnet-4-6` 등 Claude 모델만. OpenAI 임포트(`from langchain_openai import ...`)는 PR 리뷰에서 반려됩니다. 임베딩/Vector DB도 마찬가지로 pgVector만 (Milvus 금지).

실무 팁 — **`temperature=0`**: severity 등급은 "정답이 하나"인 분류 작업입니다. 같은 입력에 매번 다른 등급이 나오면 안 되므로 `temperature=0`으로 무작위성을 제거합니다. 글쓰기/요약이라면 0.7 정도를 쓰지만, 판정 노드에서는 0이 정석입니다.

개념 2. USE_MOCK_LLM 패턴 (CLAUDE.md §10-3)

비유

실제 식당을 열기 전에 종이 인형으로 손님 응대 연습을 합니다. 종이 인형(Mock)은 진짜 손님과 똑같은 자리에 앉고 똑같은 주문서를 내밀지만, 음식값(API 비용)은 안 듭니다. 연습이 끝나면 스위치 하나(`USE_MOCK_LLM=false`)로 진짜 손님을 받습니다.

개념

USE_MOCK_LLM 환경변수로 실제 **LLM** 호출 / 가짜 응답을 코드 변경 없이 전환하는 패턴입니다. Mock 함수는 실제 LLM과 완전히 동일한 **JSON** 구조를 반환해야, 호출하는 쪽 코드가 진짜인지 가짜인지 몰라도 됩니다.

왜 중요한가**? (3**가지 이유**)**

1. 비용: 개발 중 노드를 수십 번 돌리는데 매번 실제 Claude를 호출하면 API 비용이 쌓입니다. Mock은 공짜입니다.
2. 속도: 실제 LLM은 응답에 1~3초 걸립니다. Mock은 즉시 반환 → 단위 테스트가 빠릅니다.
3. 테스트 독립성: CLAUDE.md §10-5 "단위 테스트는 실제 DB/LLM 없이 통과해야 함". CI 환경에 API 키가 없어도, 네트워크가 끊겨도 테스트가 돌아갑니다.

실제 코드

```
import os
import json

# 개발 중 기본값 true (CLAUDE.md §10-3: os.getenv("KEY", "default") 패턴)
USE_MOCK_LLM = os.getenv("USE_MOCK_LLM", "true").lower() == "true"


def _mock_llm_response(current_severity: str) -> str:
    """실제 LLM과 동일한 JSON 구조를 반환하는 가짜 응답.
    severity_judge.py 프롬프트가 요구하는 형식과 정확히 일치해야 함."""
    return json.dumps({
        "severity": current_severity,        # Mock은 원래 등급 그대로 통과
        "issue_type": {"Critical": "Vulnerability", "Major": "Bug", "Minor":
"Code Smell"}.get(current_severity, "Bug"),
        "suggestion": "[MOCK] 개발용 가짜 제안 — 실제 LLM 미호출",
        "resolution_example": "[MOCK] 실제 환경에서 교정 코드 예시가 제공됩니다.",
        "rationale": "[MOCK] USE_MOCK_LLM=true",
    }, ensure_ascii=False)


async def _call_llm(prompt: str, model: str, current_severity: str) -> str:
    if USE_MOCK_LLM:
        return _mock_llm_response(current_severity)   # 같은 자리에 종이 인형
    # 실제 호출 (개념 5에서 async 처리 설명)
    llm = ChatAnthropic(model=model, temperature=0, max_tokens=1024)
    response = await llm.ainvoke([HumanMessage(content=prompt)])
    return response.content
```

실무 팁: Mock 응답을 만들 때 "성공 케이스"만 만들지 말고, 잘못된 **JSON**을 반환하는 **Mock**도 하나 두면 개념 4(재시도)를 테스트할 수 있습니다. 예: `return "이건 JSON이 아닌 일반 텍스트입니다"`.

개념 3. 모델 라우팅 전략 (비용 최적화)

비유

병원 분류(triage)와 같습니다. 응급실에 환자가 오면 중증도에 따라 다른 의사에게 보냅니다. 생명이 위급하면(Critical) 최고 전문의(sonnet), 치료가 필요하지만 급하진 않으면(Major) 일반 당직의(haiku), 가벼운 찰과상(Minor)이면 의사 없이 간호사 처치(LLM 호출 생략)로 끝냅니다.

개념

이슈의 **severity**별로 다른 모델을 쓰거나 아예 **LLM**을 건너뛰는 전략입니다. 모든 이슈를 가장 비싼 모델로 판단하는 건 낭비이고, 모두 싼 모델로 하면 중요한 판단의 정확도가 떨어집니다.

왜 **severity**별로 다른 모델을 쓰나?

severity	처리 방식	이유
Critical	`claude-sonnet-4-6`	보안 취약점/배포 차단급. 오판 비용이 가장 큼 → 가장 정확한 모델 필요
Major	`claude-haiku-4-5`	리팩토링 권고급. 빠르고 저렴한 모델로 충분
Minor	**LLM** 호출 생략	코드 스멜. CLI 도구 판단을 그대로 신뢰 → 비용/시간 0

Critical은 양이 적은데 중요도가 높고, Minor는 양이 많은데 중요도가 낮습니다. 비싼 모델을 "적게 중요한 곳에" 집중시키면 전체 비용을 크게 줄이면서 정확도는 유지됩니다.

실제 코드

```
# severity → 모델 매핑 (None이면 LLM 호출 생략)
SEVERITY_MODEL_ROUTE: dict[str, str | None] = {
    "Critical": "claude-sonnet-4-6",   # 가장 정확
    "Major":    "claude-haiku-4-5",    # 빠르고 저렴
    "Minor":    None,                  # LLM 생략, CLI 판단 그대로
}
```

```python
async def _judge_one(issue: dict, skill_level: str, similar_cases: list[dict],
tech_stack: list[str] | None = None) -> dict:
    current = issue["severity"]
    model = SEVERITY_MODEL_ROUTE.get(current, "claude-haiku-4-5")  # 기본값 haiku

    if model is None:
        # Minor → LLM 건너뜀. CLI 결과를 그대로 enriched에 통과
        return {**issue, "rationale": "CLI 도구 판단 채택 (LLM 생략)"}

    # severity_judge.py의 프롬프트 빌더 재사용 (LLM 호출 안 하는 순수 함수)
    from dev2.prompts.severity_judge import build_severity_prompt
    prompt = build_severity_prompt(
        message=issue["message"],
        current_severity=current,
        developer_level=skill_level,
        similar_cases=similar_cases,
        tech_stack=tech_stack,
        file_name=issue.get("file", ""),
    )
    raw = await _call_llm(prompt, model=model, current_severity=current)
    return _parse_with_retry(raw, fallback=issue)   # 개념 4
```

실무 팁: 라우팅 표를 코드 상단 상수로 빼두면, 나중에 "Major도 sonnet으로 올리자" 같은 정책 변경을 한 줄로 처리할 수 있습니다. if/else 안에 모델명을 흩어 놓으면 변경 지점을 찾기 어렵습니다.

개념 4. json.loads() + 재시도 패턴 (CLAUDE.md §12)

비유

번역가에게 "결과를 표 형식으로 줘"라고 했는데, 가끔 줄글로 답하는 경우가 있습니다. 그럴 때 화내지 않고 "표 형식으로 다시 줘"라고 한 번 더 부탁합니다. 그래도 안 되면, 일단 원래 자료를 그대로 쓰고 넘어갑니다(작업 전체를 멈추진 않음).

개념

LLM은 "JSON으로만 답하라"고 해도 가끔 앞뒤에 설명을 붙이거나 형식을 어깁니다. 그래서 `json.loads()`가 `json.JSONDecodeError`를 던질 수 있습니다. CLAUDE.md §12 규칙: 캐치 → **1**회 재시도 → **2**회 실패 시 **fallback(**원래 **severity** 그대로**)** 반환.

왜 **LLM** 응답을 직접 **DB**에 저장하면 안 되나**?**

- LLM은 `"severity": "Crtical"`처럼 오타를 낼 수 있고, 정의에 없는 값(`"Blocker"`)을 만들 수도 있습니다.

- 검증 없이 저장하면 Dev4의 3-Way 판정(`Critical >= 1 → escalate`)이 깨지거나, DB 제약 조건 위반이 납니다.
- CLAUDE.md §6 보안 규칙: "LLM 응답을 직접 DB에 저장 시 반드시 파싱/검증 후 저장". 즉 파싱**(json.loads)** + 검증**(severity**가 허용값인지**)** 후에만 신뢰합니다.

실제 코드

```
import json
import logging

logger = logging.getLogger(__name__)
VALID_SEVERITIES = {"Critical", "Major", "Minor"}


def _parse_with_retry(raw: str, fallback: dict, retry_fn=None) -> dict:
    """LLM 응답 JSON 파싱 + 검증. 실패 시 원래 issue로 fallback."""
    for attempt in range(2):   # 최초 1회 + 재시도 1회 = 총 2회 (CLAUDE.md §12)
        try:
            parsed = json.loads(raw)
            sev = parsed.get("severity")
            if sev not in VALID_SEVERITIES:       # 검증: 허용값인지 확인
                raise ValueError(f"허용되지 않은 severity: {sev}")
            # 검증 통과 → 신뢰 가능한 결과
            return {
                **fallback,
                "severity": sev,
                "suggestion": parsed.get("suggestion", fallback.get("suggestion",
"")),
            }
        except (json.JSONDecodeError, ValueError) as e:
            # CLAUDE.md §12: 원본 응답을 로그에 남김 (나중에 디버깅용)
            logger.warning("llm_judge JSON 파싱 실패 (시도 %d): %s | 원본: %.200s",
                           attempt + 1, e, raw)
            if attempt == 0 and retry_fn is not None:
                raw = retry_fn()   # 1회 재시도 (LLM 다시 호출)
                continue
            break
    # 2회 실패 → fallback: CLI 도구 원래 severity 그대로 사용 (예외 안 던짐)
    logger.error("llm_judge 2회 파싱 실패 → CLI severity fallback 사용")
    return {**fallback, "rationale": "LLM 파싱 실패 — CLI 판단 유지"}
```

⚠️ 실무 함정: 재시도를 무한 루프로 돌리면 비용 폭탄 + 파이프라인 멈춤이 됩니다. CLAUDE.md §12는 명확히 "재시도 **1**회"로 한정합니다. 그리고 최종 실패 시에도 예외를 던져 전체를 멈추는 게 아니라, 원래 **severity**로 안전하게 진행합니다 (§10-4 "에러는 삼키지 않되, 로그 후 진행").

실무 팁 — **JSON** 추출 보강: 실전에서는 LLM이 ```` ```json ... ``` ```` 코드블록으로 감싸는 경우가 흔합니다. `json.loads` 전에 정규식으로 `{...}` 부분만 뽑아내는 전처리를 두면 1차 실패율이 크게 줄어듭니다.

개념 5. asyncio + LangChain 비동기 패턴

비유

이미 `run_tools.py`에서 "종업원 한 명이 여러 테이블을 동시에" 굴리는 패턴을 봤습니다. LLM 호출도 똑같습니다. Critical 이슈 3개를 LLM에 보낼 때, 하나씩 줄 세워(3초 x 3 = 9초) 기다리지 말고 동시에 보내서**(**약 **3**초**)** 한꺼번에 받아옵니다.

개념

- **`llm.ainvoke([...])`** (async invoke): LangChain이 제공하는 네이티브 비동기 호출. `await`로 기다리는 동안 이벤트 루프가 다른 이슈 호출을 진행합니다. → 이게 **1**순위 정석.
- **`llm.invoke([...])`** (동기): 호출이 끝날 때까지 이벤트 루프를 막습니다. async 노드 안에서 그냥 쓰면 다른 호출이 멈춰 병렬성이 사라집니다.
- **LangGraph** 노드는 **`async def`**여야 run_tools_node처럼 다른 async 노드/도구와 매끄럽게 연결됩니다.

왜 **`asyncio.to_thread()`**가 필요할 때가 있나**?**

대부분의 LangChain 모델은 `ainvoke`를 지원하므로 그걸 쓰면 됩니다. 하지만 만약 **`ainvoke`**를 지원하지 않는 동기 전용 함수(예: 일부 임베딩 라이브러리, 레거시 SDK)를 async 노드에서 호출해야 한다면, `run_tools.py`에서 subprocess를 처리한 것과 똑같은 방법으로 감쌉니다:

```
# run_tools.py에서 이미 쓰는 패턴 (line 159) — 동기 함수를 스레드풀로 오프로드
result = await asyncio.to_thread(lambda: some_sync_llm_call(prompt))
```

`asyncio.to_thread()`는 동기 함수를 별도 스레드에서 돌리고 그 동안 이벤트 루프는 멈추지 않게 해줍니다. 즉 "동기 함수를 비동기인 척 감싸는" 도구입니다.

실제 코드 **(**노드 전체 골격**)**

```
import asyncio
from dev2.schema import Dev2State


async def llm_judge_node(state: Dev2State) -> dict:
    """LangGraph 노드: CLI 도구 결과의 severity를 LLM으로 재판단한다.

    흐름:
        1. static_raw + security_raw 합치기 (run_tools 노드 출력)
```

```
        2. 이슈별 severity 라우팅 (Critical→sonnet, Major→haiku, Minor→생략)
        3. asyncio.gather로 LLM 호출 병렬 실행
        4. JSON 파싱 + 검증 + fallback → enriched 반환
    """
    issues = state.get("static_raw", []) + state.get("security_raw", [])
    if not issues:
        return {"enriched": []}   # CLAUDE.md §12: 빈 입력도 예외 없이 진행

    skill_level = state.get("skill_level", "junior")   # §12: 미존재 시 junior 기본
값


    # similar_cases는 vector_search 노드가 채운다고 가정 (pgVector 검색 결과)
    similar = state.get("similar_cases", [])

    # ── LLM 호출 병렬 실행 (run_tools_node의 gather 패턴과 동일) ──
    tasks = [_judge_one(issue, skill_level, similar) for issue in issues]
    enriched = await asyncio.gather(*tasks, return_exceptions=False)

    return {"enriched": enriched}   # LangGraph가 state에 merge
```

실무 팁 — **ainvoke vs to_thread** 선택: - LangChain 모델 호출 → **`ainvoke`** (네이티브 비동기, 가장 효율적) - `ainvoke`가 없는 동기 라이브러리 → **`asyncio.to_thread`** (run_tools.py 방식) - 둘을 섞지 말 것. ainvoke가 있는데 to_thread로 감싸면 스레드를 불필요하게 낭비합니다.

실무 팁 — **LangSmith config 주입 (CLAUDE.md §10-6)**: callbacks는 노드 함수에 하드코딩하지 말고 graph.ainvoke(config=...)로 주입합니다. 그래야 통합 시 trace 프로젝트명을 환경별로 바꿀 수 있습니다.

3rd-Project-sub2 프로젝트에서

파일	역할
`dev2/nodes/llm_judge.py` (구현 대상)	LLM을 호출해 severity 재판단. 입력 `static_raw`/`security_raw` → 출력 `enriched`
`dev2/prompts/severity_judge.py` (완성)	프롬프트 문자열만 생성 (`build_severity_prompt`). LLM 호출 안 함 → 순수 테스트 가능
`dev2/nodes/run_tools.py` (참고)	async + `asyncio.to_thread` + `gather` 패턴의 살아있는 예시. llm_judge도 같은 골격
`dev2/schema.py`	`Dev2State.enriched` 출력 필드 정의, `Issue` Pydantic 검증 모델
`dev2/.env.example`	`USE_MOCK_LLM=true`, `ANTHROPIC_API_KEY` 설정 위치

통합 시 주의 **(Week 5)**

- `USE_MOCK_LLM=false`로 전환할 때, Mock과 실제 응답의 JSON 키가 정확히 같은지 다시 확인 (`severity`, `issue_type`, `suggestion`, `resolution_example`, `rationale` — 5개 필드).
- LLM 호출이 추가되므로 LangSmith trace에 노드별 비용/지연이 찍힙니다 — 모델 라우팅 효과를 여기서 검증.

복습 퀴즈 (5문항)

1. (이해) `severity_judge.py`와 `llm_judge.py`의 책임을 한 문장씩으로 구분해 설명하세요. 왜 이렇게 나눴나요?
2. (적용) `USE_MOCK_LLM=true`일 때 Mock 함수가 반드시 지켜야 할 조건은 무엇인가요? 안 지키면 어떤 문제가 생기나요?
3. (적용) Minor 이슈에 대해 LLM을 호출하지 않는 이유를 비용/정확도 관점에서 설명하세요.
4. (비교) `llm.invoke()`와 `llm.ainvoke()`의 차이는? async 노드 안에서 `invoke()`를 쓰면 무슨 일이 생기나요?
5. (적용) LLM이 JSON 대신 일반 텍스트를 2번 연속 반환했습니다. CLAUDE.md §12에 따르면 이 노드는 어떻게 동작해야 하나요? (예외를 던져야 하나요?)

정답 힌트

1. severity_judge=프롬프트 생성(순수 함수, LLM 없이 테스트 가능) / llm_judge=실제 LLM 호출+파싱. 분리하면 프롬프트 로직을 API 없이 단위 테스트 가능.

2. 실제 LLM과 동일한 JSON 구조(키 이름까지, 5개 필드: `severity`, `issue_type`, `suggestion`, `resolution_example`, `rationale`) 반환. 안 지키면 false로 전환 시 파싱 코드가 깨짐.
3. Minor는 양이 많고 중요도 낮음 → 호출 비용 대비 효용이 적음. CLI 판단을 그대로 신뢰.
4. invoke=동기(이벤트 루프 블록), ainvoke=비동기(다른 호출과 병렬 가능). async 노드에서 invoke 쓰면 병렬성이 사라져 호출이 직렬화됨.
5. 1회 재시도 후에도 실패 → 예외를 던지지 않고 원래 CLI severity로 fallback, 로그 기록 후 진행.

학습 노트: store 노드 — PostgreSQL + BigQuery 이중 적재

작성일: 2026-05-24 관련 파일: `dev2/nodes/store.py` (구현 예정), `dev2/db.py`, `dev2/schema.py`
복습 예정: D+1 (5/25), D+3 (5/27), D+7 (5/31)

한 줄 요약

store 노드는 분석이 끝난 이슈 목록을 "지금 바로 쓰는 운영 DB(PostgreSQL)"와 "나중에 통계 내는 분석 창고(BigQuery)" 두 곳에 나눠 저장하고, Dev4에게 넘길 표준 양식(Output Contract A)을 조립하는 파이프라인의 마지막 단계다.

---

1. 왜 PostgreSQL + BigQuery 이중 적재인가?

비유

편의점(PostgreSQL)과 대형 물류창고(BigQuery)의 차이. - 편의점: 손님이 오면 즉시 한두 개 꺼내준다. 빠르고 자주 쓴다. (실시간 조회/삽입) - 물류창고: 한 번에 수만 개를 쌓아두고 "이번 달 어느 상품이 제일 많이 나갔나"를 계산한다. (대용량 집계)

개념: **OLTP vs OLAP**

구분	PostgreSQL (OLTP)	BigQuery (OLAP)
목적	운영 트랜잭션	분석/집계
쿼리	"PR-001 이슈 보여줘" (행 단위)	"지난 3개월 Critical 추이" (수백만 행 집계)
속도	단건 INSERT/SELECT 빠름	대량 스캔 빠름 (열 기반 저장)
비용	항상 켜둠	스캔량 과금

왜 분리하나?

한 DB로 둘 다 하면 서로 방해한다. 분석용 무거운 집계 쿼리가 돌면 운영 DB가 느려져서, PR 리뷰 응답이 지연된다. 그래서 운영(OLTP)과 분석(OLAP)을 물리적으로 분리한다.

왜 **USE_MOCK_LLM=true** 시 **BigQuery skip?**

개발 중에는 분석 통계가 필요 없고, BigQuery는 클라우드 인증·과금이 걸린 외부 서비스라 로컬 개발자 환경에 부담이다. `BIGQUERY_PROJECT_ID`가 없으면 조용히 건너뛰어, DB 없이도 파이프라인이 끝까지 돈다 (CLAUDE.md §7 Mock 기반 병렬 개발).

---

2. store 노드의 역할 (파이프라인 마지막 노드)

```
enriched (list[dict])   ← llm_judge_node가 채운 최종 이슈들
    ↓ store_node
    ├── PostgreSQL: scan_executions → scan_issues → scan_summaries (순서 중요)
    └── BigQuery:   분석 DW 적재 (BIGQUERY_PROJECT_ID 없으면 skip)
    ↓
final_output (Output Contract A 형식 list[dict])  → Dev4로 전달
```

PostgreSQL 삽입 순서가 중요한 이유: `scan_issues`와 `scan_summaries`는 `scan_id`(FK)가 필요하므로, `insert_scan_execution`이 먼저 `scan_id`를 만들어야 한다.

---

3. Output Contract A 조립 방법

enriched 이슈는 출처가 섞여 있다. `_source` 필드로 분리해서 에이전트별 그룹을 만든다.

```
static_issues   = [i for i in enriched if i.get("_source") == "static"]
security_issues = [i for i in enriched if i.get("_source") == "security"]
```

각 그룹을 AgentOutput으로 만들고 `model_dump()`로 직렬화 → final_output 리스트에 담는다. 이렇게 하면 Dev4는 "static_analysis 결과", "security_scan 결과"를 따로 받는다 (CLAUDE.md §5-A).

---

4. BigQuery 적재 패턴 (실패해도 파이프라인 계속)

```python
def _load_bigquery(output: dict, payload: dict) -> None:
    project = os.getenv("BIGQUERY_PROJECT_ID", "")
    if not project:
        return  # 로컬 개발 스킵 (인증·과금 없음)
    try:
        from google.cloud import bigquery  # lazy import
        client = bigquery.Client(project=project)
        # rows 삽입 ...
    except Exception as e:
        logger.error("BigQuery 적재 실패: %s", e)
        # 로그만 — 파이프라인은 계속 (에러 재전파 안 함)
```

왜 **BigQuery** 실패가 전체 파이프라인을 막으면 안 되나?

BigQuery는 "나중에 통계 낼 부가 기능"이지, "PR 리뷰를 완성하는 필수 기능"이 아니다. 운영 DB(PostgreSQL) 저장이 성공했고 Output Contract가 만들어졌으면, Dev4는 이미 리뷰를

진행할 수 있다. 여기서 BigQuery 실패로 예외를 던지면, 분석용 부가 작업 때문에 정작 중요한 코드 리뷰가 통째로 죽는다. 이건 CLAUDE.md §12의 "Slack API 실패 → 로그만 남기고 무시" 와 같은 철학이다. 부가 서비스는 핵심 흐름을 막지 않는다.

---

5. CLAUDE.md §12 오류 처리 연결

대상	실패 시 처리	이유
PostgreSQL 삽입	에러 로그 + 재전파(raise)	운영 DB는 핵심. 저장 실패는 데이터 유실이므로 상위가 알아야 함 (CLAUDE.md §10-4 "에러 삼키지 않는다")
BigQuery 적재	로그만, 무시	분석용 부가 기능. 핵심 흐름 막으면 안 됨
`_build_output()` 검증	Pydantic AgentOutput 검증 후 직렬화	Dev4가 받는 계약 위반 방지

`_build_output()`에서 Pydantic 검증이 왜 필요한가?
enriched는 여러 노드를 거치며 자유롭게 dict 키가 추가/변형된 상태다. 그대로 Dev4에 넘기면 계약 위반(필드 누락, 잘못된 severity 값 등)이 런타임 한참 뒤에 터진다. AgentOutput으로 한 번 통과시키면, 잘못된 데이터가 store 노드 경계에서 즉시 걸러진다. "에이전트 출력은 Output Contract를 Pydantic으로 검증 후 전달" (CLAUDE.md §10-2)이 이 지점이다.

복습 퀴즈 (3문항)

1. (이해) PostgreSQL을 OLTP, BigQuery를 OLAP라고 부르는데, 한 DB로 둘 다 처리하면 어떤 문제가 생기는가?
2. (적용) BigQuery 적재가 실패했을 때 `raise`로 예외를 올리면 어떤 나쁜 일이 생기는가? PostgreSQL 삽입 실패와 처리를 다르게 하는 이유는?
3. (비교) `_build_output()`에서 enriched dict를 그대로 `final_output`에 넣지 않고 `AgentOutput(...).model_dump()`를 거치게 하는 이유는? 검증을 생략하면 언제 문제가 드러나는가?

학습 노트: LangSmith Evaluation Harness (LLM 평가 자동화)

작성일: 2026-05-24 관련 파일: `dev2/tests/eval/golden_dataset.py`, `dev2/tests/eval/test_severity_eval.py` 복습 예정: D+1, D+3, D+7

한 줄 요약

"AI가 매긴 점수가 정답과 얼마나 맞는지"를 사람이 만든 정답지(골든 데이터셋)와 자동으로 비교해서 점수로 보여주는 채점 시스템.

왜 쓰나요?

일반 코드는 "맞다/틀리다"가 명확합니다. `add(2, 3)`은 항상 5여야 합니다. 하지만 LLM은 같은 입력에도 출력이 흔들리고, "이 정도면 좋은 답"이라는 기준이 애매합니다.

이걸 측정 안 하면 생기는 문제: - 프롬프트를 고쳤는데 좋아졌는지 나빠졌는지 감으로만 판단하게 됨 - 모델을 sonnet → haiku로 바꿨을 때 품질이 떨어졌는지 모르고 배포함 - "AI 정확도 80%"라는 말을 숫자로 증명할 수 없음

Evaluation Harness는 이 "감"을 "숫자"로 바꿉니다.

언제 쓰나요?

- 적합한 상황:
  - 프롬프트/모델을 바꾸기 전후 품질 비교 (A/B)
  - 정답이 정해진 분류 작업 (severity = Critical/Major/Minor)
  - 회귀 테스트 (모델 교체 후 기존 성능 유지 확인)
- 부적합한 상황:
  - 정답이 하나로 안 정해지는 창작 작업 (소설 생성 등) → 사람 평가나 LLM-as-judge 필요
  - 골든 데이터셋을 만들 수 없는 초기 단계

핵심 개념 3가지

1. 골든 데이터셋 **(Golden Dataset)**

사람이 직접 만든 "정답지". `[{입력, 기대 정답}, ...]` 형태. LLM이 만들면 안 됨 — 채점 기준 자체가 흔들리기 때문. 그래서 수동으로 만든다.

2. evaluate() vs 일반 pytest

항목	pytest 단위테스트	LangSmith evaluate()
판정	assert (통과/실패)	score 0~1 (정확도 %)
결과 추세	안 남음	대시보드에 시계열 누적
목적	"터지나 안 터지나"	"얼마나 잘하나"

3. evaluator 함수

(run, example)을 받아 {"score": 0 또는 1}을 반환하는 채점관 함수. - run.outputs: 우리 함수가 실제로 뱉은 결과 - example.outputs: 골든 데이터셋의 정답 - 둘이 같으면 1점, 다르면 0점 → 전체 평균이 정확도

어떻게 동작하나요? (코드 흐름)

```
from langsmith import evaluate
from langsmith.schemas import Run, Example

# 1) 골든 데이터셋 — 사람이 만든 정답지
GOLDEN_DATASET = [
    {
        "inputs":  {"message": "f-string으로 SQL 조합", "current_severity":
"Major"},
        "outputs": {"expected": "Critical"},   # 사람이 정한 정답
    },
    # ... 5개
]

# 2) 평가 대상 함수 — inputs를 받아 우리 시스템 결과를 반환
def target_function(inputs: dict) -> dict:
    severity = judge_severity(inputs["message"], inputs["current_severity"])
    return {"severity": severity}

# 3) 채점 함수 — run(실제) vs example(정답) 비교
def severity_accuracy(run: Run, example: Example) -> dict:
    predicted = run.outputs.get("severity")        # 우리 함수 출력
    expected  = example.outputs["expected"]         # 골든 정답
    return {"score": int(predicted == expected)}    # 같으면 1, 다르면 0

# 4) 실행 — 5개 입력을 모두 돌려 평균 점수 계산
results = evaluate(
    target_function,
    data=GOLDEN_DATASET,
    evaluators=[severity_accuracy],
```

```
    experiment_prefix="dev2-severity-judge",
)
```

흐름 단계: 1. evaluate()가 데이터셋의 각 row의 `inputs`를 `target_function`에 넣음 2. 함수 결과가 `run.outputs`에 담김 3. 정답 `example.outputs`와 함께 `severity_accuracy`로 전달 4. 각 row마다 0/1 → 5개 평균 = 정확도 (예: 4/5 = 0.8) 5. LangSmith 대시보드에 `dev2-severity-judge-{timestamp}` 실험으로 기록

"score"가 0 또는 1인 이유

`int(True) = 1`, `int(False) = 0`. 한 건당 맞으면 1, 틀리면 0으로 채점하면, 여러 건의 평균이 그대로 정확도 비율이 된다. (맞은 개수 / 전체 개수 = 평균 score). 부분 점수가 필요하면 0.5 같은 실수도 가능.

3rd-Project-sub2 프로젝트에서

파일: `dev2/tests/eval/golden_dataset.py`, `dev2/tests/eval/test_severity_eval.py`

역할: - `dev2/nodes/llm_judge.py`의 `_judge_single_issue()`가 매기는 severity(Critical/Major/Minor)가 사람이 정한 정답과 얼마나 일치하는지 측정. - llm_judge는 `{"severity", "suggestion", "rationale"}`를 반환 → evaluator는 그중 `severity`만 채점. - CI에서는 `LANGCHAIN_API_KEY`가 없으면 `@pytest.mark.skipif`로 자동 스킵 (CLAUDE.md §6 비용 통제와 동일 철학). - 정확도 70% 미만이면 테스트 실패 → 품질 게이팅.

```
import os
import pytest

@pytest.mark.skipif(
    not os.getenv("LANGCHAIN_API_KEY"),
    reason="LANGCHAIN_API_KEY 없음 — CI에서 LangSmith 평가 스킵",
)
def test_severity_accuracy_above_threshold():
    results = evaluate(target_function, data=GOLDEN_DATASET,
                       evaluators=[severity_accuracy],
                       experiment_prefix="dev2-severity-judge")
    avg = sum(r["evaluation_results"]["results"][0].score
              for r in results) / len(GOLDEN_DATASET)
    assert avg >= 0.7, f"정확도 {avg:.0%} < 70% 임계값"
```

실무에서 왜 중요한가

- 모델 교체 회귀 테스트: sonnet → haiku 바꿔도 같은 데이터셋으로 점수 비교 → 안전하게 비용 절감
- 프롬프트 **A/B**: severity_judge 프롬프트 개선 전후를 같은 실험명으로 비교, 대시보드에서 추세 확인

- 품질 게이팅: 70% 미만이면 머지 차단 (CI 통합)
- 비용 통제: API 키 없는 CI 환경에서는 자동 스킵 → 평소엔 안 돌고 필요할 때만 실행

복습 퀴즈 (3문항)

1. (이해) evaluator 함수가 받는 `run.outputs`와 `example.outputs`는 각각 무엇을 담고 있나요?
2. (적용) severity 분류가 5개 중 3개 맞았다면 평균 score는 얼마이고, 70% 임계값을 통과하나요?
3. (비교) 일반 pytest의 `assert`와 LangSmith `evaluate()`의 score 방식은 무엇이 다르고, 왜 LLM에는 score 방식이 더 적합한가요?

학습 노트: LangGraph StateGraph 5-노드 파이프라인 (Dev2)

작성일: 2026-05-24 관련 파일: `dev2/schema.py`, `dev2/nodes/run_tools.py`, `dev2/nodes/vector_search.py`, `dev2/nodes/llm_judge.py`, `dev2/nodes/store.py` 복습 예정: D+1, D+3, D+7

한 줄 요약

LangGraph StateGraph는 "공장 컨베이어 벨트"다. 각 작업대(노드)가 부품(state)을 받아 자기 일을 한 뒤 다음 작업대로 넘기고, 우리는 작업대 순서(엣지)만 그려주면 알아서 흐른다.

---

1. StateGraph 기본 구조 — add_node / add_edge / compile()

비유: 레고 조립 설명서를 만드는 것. 블록(노드)을 등록(add_node)하고, "이 블록 다음에 저 블록"을 화살표(add_edge)로 잇고, 마지막에 "조립 완료(compile)"를 누르면 실행 가능한 그래프 객체가 나온다.

핵심: `StateGraph(StateType)`로 그래프를 만들고 → `add_node("이름", 함수)`로 작업대 등록 → `add_edge("A", "B")`로 순서 연결 → `compile()`로 실행 가능한 객체를 얻는다. compile() 전에는 "설계도", 후에는 "실행기"다.

```
from langgraph.graph import StateGraph
from dev2.schema import Dev2State
from dev2.nodes.run_tools import run_tools_node
from dev2.nodes.vector_search import vector_search_node

builder = StateGraph(Dev2State)          # 1) State 타입 지정
builder.add_node("run_tools", run_tools_node)        # 2) 작업대 등록
builder.add_node("vector_search", vector_search_node)
builder.add_edge("run_tools", "vector_search")        # 3) 순서 연결
graph = builder.compile()                # 4) 실행기 완성
```

---

2. TypedDict State — dict 반환 시 merge 동작

비유: 화이트보드(state) 하나를 여러 사람이 돌려가며 쓴다. 각 노드는 자기가 채운 칸만 적은 메모(dict)를 제출하고, LangGraph가 화이트보드의 해당 칸만 덮어쓴다. 적지 않은 칸은 그대로 보존된다.

핵심: 노드는 전체 state가 아니라 바뀐 필드만 담은 **dict**를 반환한다. LangGraph는 기본 reducer로 이 dict를 기존 state에 얕은 병합**(shallow merge)**한다 (같은 키는 덮어쓰기, 없던 키는 추가). 그래서 `run_tools_node`가 `{"static_raw": [...], "security_raw": [...]}`만 반환해도 `pr_diff` 같은 기존 필드는 유지된다.

```
# dev2/nodes/run_tools.py 의 실제 패턴
async def run_tools_node(state: Dev2State) -> dict:
    diff = state.get("pr_diff", "")          # 기존 state 읽기
    ...
    # 바뀐 필드만 반환 → LangGraph가 state에 merge
    return {"static_raw": static_raw, "security_raw": security_raw}
```

주의: 기본 동작은 덮어쓰기다. 리스트를 누적하려면 `Annotated[list, operator.add]`로 reducer를 따로 지정해야 한다 (Dev2는 단계별 덮어쓰기라 기본 동작으로 충분).

---

3. START / END 상수 — 시작/종료 연결

비유: 컨베이어 벨트의 "입구(START)"와 "출구(END)" 표지판. 입구에서 첫 작업대로, 마지막 작업대에서 출구로 화살표를 그어야 흐름이 완성된다.

핵심: `langgraph.graph`에서 `START`, `END`를 import한다. `add_edge(START, "첫노드")`로 진입점을, `add_edge("마지막노드", END)`로 종료점을 지정한다. (`set_entry_point` / `set_finish_point`도 같은 역할의 구버전 API다.) START/END를 빼먹으면 "진입점이 없다"는 컴파일 에러가 난다.

```
from langgraph.graph import StateGraph, START, END

builder = StateGraph(Dev2State)
builder.add_node("run_tools", run_tools_node)
builder.add_node("store", store_node)
builder.add_edge(START, "run_tools")     # 입구 → 첫 노드
builder.add_edge("store", END)           # 마지막 노드 → 출구
```

---

4. 5-노드 선형 파이프라인 패턴

비유: 5단계 공장 라인. 원자재 검사(run_tools) → 과거 불량 사례 조회(vector_search) → 검사관 판정(llm_judge) → 심각 건 보고(escalate) → 창고 입고(store). 한 줄로 쭉 이어지는 직선 라인이라 분기가 없다.

핵심: 노드들을 순서대로 `add_edge`로 줄줄이 잇기만 하면 선형 파이프라인이 된다. 각 노드는 앞 노드가 채운 state 필드를 입력으로 받고, 다음 노드가 쓸 필드를 채워서 dict로 반환한다. Dev2는 `static_raw/security_raw` → `enriched` → (judged) → final_output 순으로 state가 풍부해진다.

```
from langgraph.graph import StateGraph, START, END
from dev2.schema import Dev2State
from dev2.nodes.run_tools import run_tools_node
from dev2.nodes.vector_search import vector_search_node
from dev2.nodes.llm_judge import llm_judge_node
from dev2.nodes.escalate import escalate_node      # Critical Slack 알림
from dev2.nodes.store import store_node

def build_dev2_graph():
    b = StateGraph(Dev2State)
    b.add_node("run_tools", run_tools_node)
    b.add_node("vector_search", vector_search_node)
    b.add_node("llm_judge", llm_judge_node)
    b.add_node("escalate", escalate_node)
    b.add_node("store", store_node)

    b.add_edge(START, "run_tools")
    b.add_edge("run_tools", "vector_search")
    b.add_edge("vector_search", "llm_judge")
    b.add_edge("llm_judge", "escalate")
    b.add_edge("escalate", "store")
    b.add_edge("store", END)
    return b.compile()
```

---

5. graph.ainvoke() vs graph.invoke()

비유: invoke는 "전화 받을 때까지 수화기 들고 기다리기"(동기), ainvoke는 "메시지 보내고 답 올 때까지 다른 일 하기"(비동기). Dev2 노드들은 DB·LLM·subprocess를 `await`하므로 비동기 라인이라 ainvoke를 써야 한다.

핵심: 노드 함수가 `async def`면 `await graph.ainvoke(state)`로, 일반 `def`면 `graph.invoke(state)`로 실행한다. 둘 다 초기 state(dict)를 받아 모든 노드를 거친 최종 **state(dict)**를 반환한다. LangSmith 추적은 `config={"callbacks":[tracer]}`로 주입.

```
import asyncio
from langchain_core.tracers import LangChainTracer
```

```python
async def main():
    graph = build_dev2_graph()
    initial: Dev2State = {"task_id": "t-1", "pr_id": "42", "pr_diff": diff_text}
    tracer = LangChainTracer(project_name="dev2-myid")   # CLAUDE.md §3
    final_state = await graph.ainvoke(initial, config={"callbacks": [tracer]})
    print(final_state["final_output"])   # Output Contract A (Dev4로 전달)

asyncio.run(main())
```

3rd-Project-sub2 프로젝트에서

파일: `dev2/nodes/*.py` (각 노드), 그래프 조립부 (예: `dev2/graph.py` 신규) 역할: Dev2는 "Parallel Independent Agent"지만 내부적으로는 StateGraph 선형 파이프라인. run_tools가 CLI 도구를 병렬 실행 → vector_search가 pgVector로 과거 유사 이슈 주입 → llm_judge가 심각도 판정 → escalate가 Critical 알림 → store가 PostgreSQL+BigQuery 적재 후 Output Contract A(`final_output`)를 만들어 Dev4 Aggregator로 넘긴다.

복습 퀴즈 (3문항)

1. 노드 함수가 `{"static_raw": [...]}`만 반환했는데 `pr_diff` 필드가 사라지지 않는 이유는?
2. async def 노드로 이뤄진 그래프를 `graph.invoke()`로 실행하면 어떤 문제가 생기나?
3. `add_edge(START, "run_tools")`를 빠뜨리면 compile() 시점에 무슨 일이 일어나는가?

정답 메모

1. LangGraph가 반환 dict를 기존 state에 얕은 병합(merge)하기 때문. 반환하지 않은 키는 보존된다.
2. 코루틴이 await되지 않아 노드가 실제로 실행되지 않거나 RuntimeWarning/에러 발생. async 그래프는 ainvoke 필수.
3. 진입점(entry point)이 없어 그래프 검증 단계에서 컴파일 에러가 난다.

학습 노트: FastAPI 엔드포인트 설계 (라우터 + 202 + BackgroundTasks + Mock 테스트)

작성일: 2026-05-24 관련 파일: `dev2/api/scan.py`, `dev2/api/insights.py`, `dev2/api/issues.py`, `dev2/main.py`, `dev2/db.py`, `dev2/tests/test_insights.py` 복습 예정: D+1, D+3, D+7 관련 **Task**: Task 6 — FastAPI 엔드포인트 (POST /scan + Insights API 3종 + PATCH /issues/{id}/dismiss)

한 줄 요약

"PR을 분석해줘" 같은 요청을 받는 창구(웹 **API**)를 만드는 일입니다. 오래 걸리는 일(LLM 분석)은 접수증만 주고 뒤에서 처리하고, 통계 조회는 바로 답을 주며, 실제 DB·LLM 없이도 테스트가 통과하도록 만들었습니다.

이 노트에서 다루는 **10**가지 개념

1. FastAPI APIRouter — 창구를 파일별로 나누기
2. HTTP 상태코드 202 vs 200 — "접수했어요" vs "다 됐어요"
3. BackgroundTasks — 응답 먼저 주고 뒤에서 일하기
4. HTTP 동사 (GET / POST / PATCH) — 요청에 "성격" 부여하기
5. SQLAlchemy AsyncSession — DB와 대화하는 안전한 통로
6. Lazy Import — 필요할 때만 짐 꺼내기
7. db.py Lazy 초기화 — DB 엔진을 최초 사용 순간에만 켜기
8. Mock으로 async with 흉내내기 — DB 없이 테스트하기
9. "patch where it's used" — Mock을 어디에 걸어야 하나
10. ASGITransport — 진짜 서버 없이 API 테스트하기

개념 **1. FastAPI APIRouter** — 창구를 파일별로 나누기

🏬 비유

큰 시청에 가면 창구가 "여권 발급", "전입신고", "민원 접수"로 나뉘어 있습니다. 한 창구에서 모든 걸 다 처리하면 줄이 엉키고, 직원도 헷갈립니다. APIRouter는 이 "창구를 종류별로 나누는" 도구입니다. 그리고 `include_router`는 나눠 만든 창구들을 한 건물(앱)에 입주시키는 것입니다.

🧩 개념

> `APIRouter`는 관련된 엔드포인트(URL)들을 한 묶음으로 모아 별도 파일로 분리하는 FastAPI 객체다. `prefix`로 공통 경로를, `tags`로 문서 그룹명을 지정한다. 메인 앱에서 `app.include_router(router)`로 등록하면 그 라우터의 모든 경로가 앱에 합쳐진다.

```
# dev2/api/insights.py
router = APIRouter(prefix="/insights", tags=["insights"])

@router.get("/team")          # 실제 경로는 prefix + 경로 = "/insights/team"
async def get_team_insights():
    ...
```

```python
# dev2/main.py — 조립만 담당
app = FastAPI(...)
app.include_router(scan_router)      # POST /scan
app.include_router(insights_router)  # GET /insights/*  (prefix="/insights")
app.include_router(issues_router)    # PATCH /issues/*  (prefix="/issues")
```

🧐 왜 쓰나

- 관심사 분리: `scan`(트리거), `insights`(통계), `issues`(오탐 표시)는 성격이 완전히 다릅니다. 한 파일에 몰아넣으면 수백 줄짜리 거대 파일이 됩니다.
- 병렬 개발 (이 프로젝트의 핵심 이유): 6명이 동시에 개발하는데(CLAUDE.md §15), 모두 `main.py` 하나를 고치면 Git 충돌(merge conflict)이 끊임없이 납니다. 파일을 나누면 각자 자기 파일만 건드립니다.
- 테스트 독립성: 라우터별로 Mock을 따로 걸어 격리 테스트할 수 있습니다.

> 핵심: `main.py`는 "조립"만 합니다. 비즈니스 로직(SQL, 분기)은 각 라우터 파일에 둡니다. 이게 CLAUDE.md의 "FastAPI 라우터는 수신·직렬화만 담당" 원칙과 같은 맥락입니다.

개념 2. HTTP 상태코드 202 vs 200 — "접수했어요" vs "다 됐어요"

🍕 비유

피자를 주문할 때 두 가지 경우가 있습니다. - 편의점에서 음료수 사기: 돈 내면 그 자리에서 바로 음료를 받습니다 → **200 OK** (요청 = 완료) - 피자 배달 주문: "주문 접수됐습니다, 30분 뒤 도착합니다" 하고 영수증(접수증)만 받습니다. 피자는 아직 안 왔습니다 → **202 Accepted** (요청은 받았지만 결과는 나중)

📚 개념

- **200 OK**: 요청을 처리해서 결과를 지금 응답에 담아 보냈다.
- **202 Accepted**: 요청을 접수만 했고, 실제 처리는 나중에 진행된다. 응답에는 결과가 아니라 "접수증"이 담긴다.

```python
# dev2/api/scan.py
@router.post("/scan", status_code=202)   # ← 202로 명시
async def trigger_scan(payload, background_tasks):
    ...
    return {"status": "accepted", "task_id": payload.task_id}  # 결과 아님, 접수증
```

🤔 왜 쓰나 (면접 단골 질문)

LLM 분석은 수 초 ~ 수십 초가 걸립니다. 만약 200으로 "분석 다 끝낼 때까지" 클라이언트를 붙잡아 두면: - 클라이언트(브라우저/GitHub Webhook)가 **HTTP** 타임아웃(보통 30초)에 걸려 연결이 끊깁니다. - 끊기면 분석은 했는데 결과를 못 주는 황당한 상황이 됩니다.

그래서 **"일단 접수증(202)부터 빠르게 주고, 분석은 뒤에서"** 하는 구조로 분리합니다. 결과는 나중에 별도 경로(GitHub 코멘트, Slack 알림, 또는 GET 조회)로 전달됩니다.

> 면접 답변 한 줄: "오래 걸리는 비동기 작업은 200으로 끝까지 기다리게 하면 타임아웃 위험이 있어, 202로 즉시 접수증을 반환하고 실제 처리는 백그라운드로 분리합니다."

개념 3. BackgroundTasks — 응답 먼저 주고 뒤에서 일하기

🔍 비유

식당 종업원(서버)이 주문(요청)을 받습니다. 손님에게 "주문 들어갔습니다"라고 먼저 말하고(응답 전송), 그 다음에 주방에 "이거 만들어주세요" 하고 메모를 붙여 둡니다(백그라운드 작업 등록). 손님은 기다리지 않고 자리에 앉고, 요리는 뒤에서 만들어집니다. 개념 2의 202와 짝을 이루는 "실제 일하는 부분"이 바로 이것입니다.

📚 개념

> `BackgroundTasks`는 FastAPI가 제공하는 내장 기능으로, **HTTP** 응답을 클라이언트에 보낸 직후 등록된 함수를 실행한다. `background_tasks.add_task(함수, 인자)`로 등록만 하면, 함수 호출이 그 자리에서 일어나지 않고 응답 이후로 미뤄진다.

```
# dev2/api/scan.py (실제 코드)
@router.post("/scan", status_code=202)
async def trigger_scan(payload: TaskPayload, background_tasks: BackgroundTasks):
    initial_state = _payload_to_state(payload)        # 1. 입력을 State로 변환
    background_tasks.add_task(run_dev2, initial_state) # 2. 등록만! 아직 실행 안 함
    return {"status": "accepted", "task_id": payload.task_id}  # 3. 즉시 응답
    # 4. (응답 전송 후) run_dev2(initial_state)가 백그라운드에서 실행됨
```

실행 순서가 핵심입니다: 1. `_payload_to_state()` — 입력 검증·변환 (지금 실행) 2. `add_task()` — "이거 나중에 실행해줘" 등록 (실행 아님, 예약) 3. `return` — 202 접수증을 클라이언트로 전송 4. 응답이 나간 뒤 `run_dev2()`가 LangGraph 5노드 파이프라인을 돌림

🤔 왜 쓰나 / 언제 쓰나

- 적합: 이메일 발송, 알림, 로깅, 그리고 이 프로젝트처럼 응답을 막으면 안 되는 무거운 후처리.

- 부적합: 몇 시간씩 걸리거나 실패 시 재시도·추적이 꼭 필요한 작업. 이런 건 Celery, Redis Queue, Airflow 같은 전용 작업 큐를 씁니다 (서버가 죽으면 BackgroundTasks는 그냥 사라지기 때문).
- 우리는 "수십 초짜리 단발성 분석"이라 BackgroundTasks가 딱 맞습니다. (CLAUDE.md §6의 타임아웃 분리 원칙과도 연결됩니다.)

개념 4. HTTP 동사 (GET / POST / PATCH) — 요청에 "성격" 부여하기

비유

우체국에 갈 때 행동의 종류가 다릅니다. - **GET**: 우편함을 "열어보기만" 하기 (꺼내지도, 바꾸지도 않음) → 몇 번 봐도 안전 - **POST**: 새 편지를 "부치기" (보낼 때마다 새 편지가 생김) → 두 번 누르면 편지 두 통 - **PATCH**: 봉투에 적힌 주소 중 "한 줄만 고쳐 쓰기" → 같은 수정을 몇 번 해도 결과는 똑같음

개념

HTTP 동사(메서드)는 요청이 서버에 어떤 영향을 주는지를 약속한 규칙이다. - **GET**: 읽기 전용. 서버 상태를 바꾸지 않는다(side effect 없음). 멱등**(idempotent)**. - **POST**: 새 자원/작업 생성. 호출할 때마다 새 결과 → 비멱등. - **PATCH**: 자원의 일부 필드만 수정. 보통 멱등. - (참고) PUT: 자원 전체 교체. DELETE: 삭제.

멱등**(idempotent)** = "같은 요청을 여러 번 보내도 결과가 똑같다." 네트워크가 끊겨 재시도해도 안전한지를 판단하는 핵심 개념.

이 프로젝트의 선택 근거:

엔드포인트	동사	이유
`/insights/developer/{id}`, `/insights/team`, `/insights/trends`	**GET**	SELECT만 함, 데이터 안 바꿈 → 읽기 전용
`/scan`	**POST**	새 분석 "작업"을 생성, 부를 때마다 새 작업 → 비멱등
`/issues/{id}/dismiss`	**PATCH**	`is_false_positive` 한 필드만 true로 변경, 여러 번 해도 결과 동일 → 멱등

🤔 왜 **PATCH**인가 **(DELETE/PUT**이 아닌 이유**)**

- **DELETE**가 아닌 이유: 오탐 표시는 이슈를 "지우는" 게 아닙니다. 기록은 남겨야 합니다 (감사 추적, Dev5 피드백 학습용). 그래서 "표시만" 하는 PATCH.
- **PUT**이 아닌 이유: PUT은 레코드 전체를 통째로 교체합니다. 우리는 `is_false_positive` 딱 한 칸만 바꾸므로 PATCH가 정확합니다.

개념 5. SQLAlchemy AsyncSession — DB와 대화하는 안전한 통로

☎️ 비유

DB에 질문하려면 "전화 회선(세션)"을 하나 빌려야 합니다. 다 쓰고 나면 회선을 반드시 반납해야 다른 사람이 씁니다. `async with`는 "통화 끝나면 자동으로 전화를 끊어주는" 똑똑한 전화기입니다. 깜빡하고 안 끊을 일이 없습니다.

📚 개념

- **AsyncSession**: 비동기로 DB와 대화하는 세션 객체. `await`로 쿼리를 보내고 기다린다.
- 컨텍스트 매니저 **(`async with`)**: 블록을 벗어날 때 세션을 자동으로 닫아(반환) 준다. 누수(leak) 방지.
- 파라미터 바인딩 **(`:param`)**: 값을 SQL 문자열에 직접 끼워 넣지 않고 따로 전달 → SQL 인젝션 차단.
- **`r.mappings().all()`**: 쿼리 결과를 `{컬럼명: 값}` 딕셔너리 목록으로 받는다. (그냥 `.all()`은 튜플이라 컬럼명을 모름)

```
# dev2/api/insights.py
async with AsyncSessionLocal() as s:        # 회선 빌리기 (블록 끝나면 자동 반납)
    r = await s.execute(
        text("""
            SELECT si.rule_id, si.severity, COUNT(*) as count
            FROM scan_issues si
            JOIN scan_executions se ON si.scan_id = se.id
            WHERE se.developer_id = :did     -- ← :did 자리표시자
            GROUP BY si.rule_id, si.severity
            ORDER BY count DESC
        """),
        {"did": developer_id},               # ← 값은 따로 전달 (바인딩)
    )
    issues = [dict(row) for row in r.mappings().all()]  # 컬럼명 dict 목록
```

🤔 왜 파라미터 바인딩이 필수인가 **(보안)**

만약 `f"... WHERE developer_id = '{developer_id}'"` 처럼 f-string으로 값을 끼우면, 누가 `developer_id`에 `'; DROP TABLE scan_issues; --` 같은 값을 넣었을 때 테이블이 통째로 삭제

될 수 있습니다(SQL 인젝션). 바인딩(`:did`)을 쓰면 DB가 그 값을 "그냥 문자열 데이터"로만 취급해서 명령으로 실행되지 않습니다. CLAUDE.md §6의 "f-string으로 SQL 조합 금지" 규칙이 이것입니다.

개념 6. Lazy Import — 필요할 때만 짐 꺼내기

🎒 비유

캠핑을 갈 때 텐트를 "차에 미리 다 펼쳐서" 가면 무겁고, 캠핑장 안 가면 헛수고입니다. 보통은 차 트렁크에 접어 넣고, 캠핑장에 도착한 순간(필요한 그 시점) 펼칩니다. Lazy import는 모듈을 파일 맨 위에서 미리 불러오지 않고, 함수가 실제로 호출될 때 불러오는 것입니다.

📚 개념

> 보통 import는 파일 최상단에 둬서 파일을 읽는 순간 함께 로드된다. **Lazy import**는 import 문을 함수 내부에 넣어, 그 함수가 호출되는 순간에만 모듈을 로드한다.

```
# dev2/api/insights.py
@router.get("/developer/{developer_id}")
async def get_developer_insights(developer_id: str):
    from dev2.db import AsyncSessionLocal   # ← 함수 내부에서 import (lazy)
    async with AsyncSessionLocal() as s:
        ...
```

🧐 왜 쓰나 (이 프로젝트의 진짜 이유)

`dev2.db` 모듈은 `asyncpg`(PostgreSQL 비동기 드라이버)를 필요로 합니다. 만약 `insights.py` 맨 위에서 `from dev2.db import AsyncSessionLocal`을 하면: - **CI**/테스트 환경에 **asyncpg**가 안 깔려 있으면 `insights.py`를 import하는 순간 에러가 터집니다. - 그러면 DB랑 전혀 상관없는 테스트(예: 라우터 등록 확인)까지 전부 실패합니다.

함수 안으로 옮기면, 그 함수를 실제로 호출할 때만 asyncpg가 필요해집니다. 테스트에서는 Mock으로 갈아끼우므로 asyncpg가 없어도 import·테스트가 통과합니다. (CLAUDE.md §13: "실제 DB/LLM 없이 통과"의 기반)

> 주의: lazy import는 "이 모듈은 무거운/선택적 의존성을 쓴다"는 신호일 때 적절합니다. 평범한 가벼운 모듈은 그냥 상단 import가 정석입니다. (그래서 코드에 `# noqa: PLC0415` 주석으로 "의도된 lazy import"임을 명시했습니다.)

개념 7. db.py Lazy 초기화 — DB 엔진을 최초 사용 순간에만 켜기

🪠 비유

사무실에 비싼 복합기(DB 엔진)가 있습니다. 출근하자마자 켜두면, 아무도 안 쓰는 날엔 전기만 낭비됩니다. 똑똑한 방법은 누군가 처음 인쇄 버튼을 누르는 순간 켜고, 그 다음부터는 켜진 채로 계속 쓰는 것입니다. 한 번만 켜고 계속 재사용 → 이게 싱글톤(singleton)에 가까운 발상입니다.

📚 개념

> 개념 6의 lazy import가 "모듈 가져오기를 미루는 것"이라면, 이건 한 단계 더 나아가 무거운 객체(**DB** 엔진) 생성 자체를 최초 사용 시점까지 미루는 패턴이다. 전역 변수를 `None`으로 두고, 처음 필요할 때 만든 뒤 재사용한다.

```
# dev2/db.py
_engine = None
_session_factory = None

def _get_session_factory():
    global _engine, _session_factory
    if _session_factory is None:                          # 최초 1회만 진입
        _engine = create_async_engine(DATABASE_URL)    # ← 여기서 asyncpg 필요
        _session_factory = sessionmaker(
            _engine, class_=AsyncSession, expire_on_commit=False
        )
    return _session_factory                               # 이후엔 기존 것 재사용

def AsyncSessionLocal():
    return _get_session_factory()()   # 세션 컨텍스트 매니저 반환
```

🧐 왜 쓰나

- **import** 안전성: 모듈 최상단에서 `create_async_engine(...)`을 호출하면 `dev2.db`를 import하는 순간 asyncpg가 없으면 폭발합니다. 함수 안으로 넣으면 import는 안전하고, 실제 DB 쓸 때만 asyncpg가 필요합니다 (개념 6과 같은 목표를 객체 생성 레벨에서 달성).
- 자원 절약: 엔진은 DB 커넥션 풀을 가진 무거운 객체입니다. 매 요청마다 만들면 낭비 → `if _session_factory is None`으로 딱 한 번만 만들고 재사용합니다.

개념 8. Mock으로 async with 흉내내기 — DB 없이 테스트하기

🎭 비유

영화에서 배우가 실제로 절벽에서 떨어지지 않습니다. 스턴트 대역(Mock)이 대신합니다. 관객(테스트)이 보기엔 똑같이 동작하지만, 진짜 위험(진짜 DB 연결)은 없습니다. 우리는 `AsyncSessionLocal()`이라는 "진짜 DB 통로"를 가짜 대역으로 바꿔치기합니다.

🧩 개념

> `async with AsyncSessionLocal() as s:` 한 줄이 동작하려면 가짜 객체가 갖춰야 할 게 여러 겹이다: 1. `AsyncSessionLocal()` 호출 → 컨텍스트 매니저를 반환해야 함 2. 그 매니저는 `__aenter__`/`__aexit__`(async 버전)을 지원해야 함 → `AsyncMock` 3. `__aenter__`이 세션(s)을 내놔야 함 4. `await s.execute(...)`가 결과 객체를 내놔야 함 5. 결과`.mappings().all()`이 행 목록을 내놔야 함

```
# dev2/tests/test_insights.py
def _mock_db_ctx(rows=None):
    rows = rows or []

    # (5) r.mappings().all() → rows  를 흉내
    mock_result = MagicMock()
    mock_result.mappings.return_value.all.return_value = rows

    # (4) await s.execute(...) → mock_result  를 흉내 (await 가능해야 하므로
AsyncMock)
    mock_session = AsyncMock()
    mock_session.execute.return_value = mock_result

    # (1~3) async with ... as s → mock_session  을 흉내
    mock_cm = AsyncMock()
    mock_cm.__aenter__.return_value = mock_session  # as s 자리
    mock_cm.__aexit__.return_value = False          # 예외 전파 허용
    return mock_cm
```

🤔 핵심 포인트: **MagicMock vs AsyncMock**

- **AsyncMock**: `await` 붙여 호출하는 것들 (`s.execute`, `__aenter__`, `__aexit__`). 비동기 흉내.
- **MagicMock**: `await` 없이 그냥 호출하는 것들 (`mappings()`, `all()`). 동기 흉내.
- `mock.xxx.return_value.yyy.return_value` 체인은 "`mock.xxx()` 결과에 또 `.yyy()`를 부른 결과"를 미리 정해두는 문법입니다. `r.mappings().all()`처럼 점이 이어지는 호출을 흉내 낼 때 씁니다.

왜 이렇게까지 하나

실제 PostgreSQL을 띄우지 않고도 "엔드포인트가 DB 결과를 받아 올바른 JSON으로 포장하는가"를 검증할 수 있습니다. 테스트가 빠르고(네트워크 없음), 어디서나 돌고(DB 설치 불필요), DB 상태에 흔들리지 않습니다(결정적).

개념 9. "patch where it's used" — Mock을 어디에 걸어야 하나

🚪 비유

같은 "철수"라도 회사에서 부르는 이름과 집에서 부르는 이름이 다릅니다. Mock을 걸 때는 "코드가 그 객체를 실제로 부르는 이름"에 걸어야 합니다. 엉뚱한 이름에 걸면 가짜로 안 바뀝니다.

📚 개념

> Mock 패치의 원칙: "정의된 곳"이 아니라 "사용되는 곳"의 이름을 패치한다. 단, 그 "사용되는 이름"은 import 방식에 따라 달라진다.

이 프로젝트에 두 패턴이 공존합니다:

(A) 모듈 상단 **import** → 사용처 이름으로 패치

```
# scan.py 상단: from dev2.graph import run_dev2
# → scan.py 안에 run_dev2라는 "복사된 이름"이 생김
with patch("dev2.api.scan.run_dev2", ...):   # ← 사용처(scan) 이름으로 패치
```

(B) 함수 내부 **lazy import** → 원본 모듈 이름으로 패치

```
# insights.py: 함수 안에서 from dev2.db import AsyncSessionLocal
# → 함수가 호출될 때마다 dev2.db에서 매번 새로 가져옴
with patch("dev2.db.AsyncSessionLocal", ...):  # ← 원본(dev2.db) 이름으로 패치
```

🧐 왜 패턴 **(B)**는 원본 위치인가

패턴 A는 파일을 로드할 때 `run_dev2`라는 이름을 `scan.py` 네임스페이스에 복사해 둡니다. 그래서 그 복사본(`dev2.api.scan.run_dev2`)을 바꿔야 합니다.

패턴 B(lazy import)는 이름을 미리 복사하지 않습니다. 함수가 호출되는 매 순간 `dev2.db`로 가서 `AsyncSessionLocal`을 꺼냅니다. 그래서 복사본이 없고, 원본(**`dev2.db.AsyncSessionLocal`**)을 바꾸면 호출 시점에 자동으로 가짜를 가져옵니다. (개념 6 lazy import의 부수 효과 — 패치가 더 단순해집니다.)

> 외우는 법: "상단 import면 그 파일 이름으로, 함수 안 import면 원본 모듈 이름으로 패치."

개념 10. ASGITransport — 진짜 서버 없이 API 테스트하기

🏠 비유

집을 사기 전에 "모델하우스"에서 구조를 둘러봅니다. 실제 집을 짓지 않아도 방 배치를 확인할 수 있죠. ASGITransport는 진짜 서버를 띄우지(포트 열고 네트워크 듣고) 않고, FastAPI 앱을 같은 프로세스 안에서 직접 호출해 테스트하게 해줍니다.

📚 개념

> **ASGI**는 파이썬 비동기 웹 앱의 표준 인터페이스다. `ASGITransport(app=app)`는 HTTP 소켓(네트워크) 없이 httpx 클라이언트의 요청을 곧장 FastAPI 앱 함수로 전달한다(in-process). `base_url`은 형식상 필요한 가짜 주소다.

```
# dev2/tests/test_insights.py
transport = ASGITransport(app=app)                                  # 앱을 직접 연결
async with AsyncClient(transport=transport, base_url="http://test") as client:
    r = await client.post("/scan", json={...})
assert r.status_code == 202
```

🧐 왜 이 방식인가 **(옛 방식이 deprecated 된 이유)**

- 이전 방식: `AsyncClient(app=app, ...)` — httpx 0.20부터 deprecated 되었습니다.
- httpx가 "transport(전송 방식)"라는 개념을 명시적으로 분리했기 때문입니다. `app=app`은 "이 앱으로 보낸다"는 의도가 숨어 있었지만, `transport=ASGITransport(app=app)`는 "ASGI 방식으로 이 앱에 직접 전달한다"가 명확합니다. 미래에 다른 transport(실제 네트워크, WSGI 등)로 바꿀 때 일관성이 생깁니다.
- 장점: 포트 충돌 없음, 빠름(네트워크 왕복 없음), 어디서나 동작.

🔁 전체 흐름 한눈에 보기

```
[POST /scan — 쓰기/트리거]
클라이언트 → POST /scan
   → FastAPI가 TaskPayload(Pydantic)로 자동 검증
   → _payload_to_state() 변환
   → background_tasks.add_task(run_dev2, state)   (예약만)
   → 202 {"status":"accepted", ...} 즉시 반환  ← 여기서 클라이언트는 해방
   ——(응답 전송 후)——
   → run_dev2(state) 백그라운드 실행 (LangGraph 5노드)

[GET /insights/* — 읽기/조회]
클라이언트 → GET /insights/team
   → (함수 내부) from dev2.db import AsyncSessionLocal   (lazy)
   → async with AsyncSessionLocal() as s:  (필요 시 엔진 lazy 초기화)
```

```
   → SELECT ... :param 바인딩 → r.mappings().all()
   → 200 {"top_issues":[...]} 반환

[PATCH /issues/{id}/dismiss — 일부 수정/멱등]
클라이언트 → PATCH /issues/1/dismiss
   → UPDATE scan_issues SET is_false_positive=true WHERE id=:id
   → 200 {"issue_id":1, "is_false_positive":true}
```

3rd-Project-sub2 프로젝트에서

파일	역할
`dev2/main.py`	FastAPI 앱 조립. 3개 라우터 + /health 등록 (비즈니스 로직 없음)
`dev2/api/scan.py`	POST /scan → 202 + BackgroundTasks로 `run_dev2` 비동기 실행
`dev2/api/insights.py`	GET /insights/{developer,team,trends} → 집계 통계 (읽기 전용, DB 실패 시 빈 결과)
`dev2/api/issues.py`	PATCH /issues/{id}/dismiss → 오탐 마킹 (멱등, 기록 유지)
`dev2/db.py`	AsyncSession lazy 초기화. `AsyncSessionLocal()` 컨텍스트 매니저 제공
`dev2/tests/test_insights.py`	5개 엔드포인트를 Mock(DB/run_dev2)으로 격리 테스트, ASGITransport 사용

이 엔드포인트들은 CLAUDE.md의 **CQRS** 분리(쓰기 = POST /scan, 읽기 = GET /insights), **§6** 보안(파라미터 바인딩), **§13 Mock** 테스트 원칙을 구현한 결과물입니다.

연관 개념

- **Pydantic** 검증 → `TaskPayload`로 요청 바디를 자동 검증 (CLAUDE.md §5-B 입력 계약)
- **LangGraph** 파이프라인 → `run_dev2`가 실행하는 5노드 그래프 (agent/langgraph.md)
- **asyncio** 병렬 실행 → 백그라운드 안에서 도구들을 동시 실행 (ai/asyncio_parallel.md)
- **UPSERT** → insights가 읽는 `scan_summaries`를 채우는 쓰기 패턴 (database/upsert.md)

복습 퀴즈 (5문항)

1. (이해) 친구가 "POST /scan에 왜 200이 아니라 202를 썼어?"라고 물으면 피자 배달 비유로 30초 안에 설명해보세요.
2. (이해) `BackgroundTasks`로 등록한 `run_dev2`는 응답을 보내기 전에 실행될까요, 후에 실행될까요? 그 이유는?
3. (적용) 새 엔드포인트 `DELETE /issues/{id}`를 추가한다면 적절할까요? 우리가 PATCH를 고른 이유를 근거로 판단해보세요.
4. (적용) `insights.py`의 SQL을 `f"... WHERE developer_id = '{developer_id}'"`로 바꾸면 어떤 보안 문제가 생기나요? 한 문장으로.
5. (비교) `patch("dev2.api.scan.run_dev2")`와 `patch("dev2.db.AsyncSessionLocal")`는 패치 위치가 다릅니다. 각각 왜 그 위치를 골랐나요? (힌트: import가 파일 상단인지 함수 내부인지)

정답이 막히면 개념 2·3(2번), 개념 4(3번), 개념 5(4번), 개념 6·9(5번)를 다시 보세요.

Apache Airflow DAG 학습 노트

날짜: 2026-05-22 복습 예정: D+1, D+3, D+7

한 줄 요약

"매일 새벽 2시에 이 작업 순서대로 실행해줘"를 코드로 정의하는 도구.

---

1. 비유

알람 + 할 일 목록에 비유: - 알람**(schedule)**: 매일 새벽 2시에 울림 - 할 일 목록**(DAG)**: 알람 울리면 이 순서대로 실행 1. PostgreSQL에서 어제 데이터 꺼내기 2. BigQuery에 올리기 3. 완료 알림

Airflow = 이 알람과 할 일 목록을 관리하는 시스템.

---

2. 개념

DAG (Directed Acyclic Graph): 방향이 있고 순환이 없는 그래프. 쉽게 말해 "순서가 있는 작업 흐름"을 코드로 표현한 것.

핵심 용어:

용어	설명	예시
DAG	작업 흐름 전체	`dev2_daily_etl`
Task	DAG 안의 개별 작업	`extract`, `load`
schedule_interval	실행 주기	`"0 2 * * *"` = 매일 새벽 2시
`>>` 연산자	작업 순서 정의	`extract >> load`

cron 표현식 읽는 법:

```
"0 2 * * *"
 | | | | └── 요일 (모든 요일)
 | | | └──── 월 (모든 달)
 | | └────── 일 (모든 날)
 | └──────── 시 (2시)
 └────────── 분 (0분)
= 매일 새벽 2시 0분에 실행
```

---

3. 왜 쓰나

- 없으면: PostgreSQL → BigQuery ETL을 수동으로 실행하거나, 서버에 cron 직접 설정
- 있으면: 코드로 스케줄 관리, 실패 시 자동 재시도, UI에서 실행 현황 확인

---

4. 코드 (Dev 2)

```python
# dags/daily_etl.py
from airflow import DAG
from airflow.operators.python import PythonOperator
from datetime import datetime, timedelta

# Task 함수 정의
def extract(**context):
    # PostgreSQL에서 어제 데이터 읽기
    rows = fetch_yesterday_scans()
    context["ti"].xcom_push(key="rows", value=rows)  # 다음 Task에 데이터 전달

def load(**context):
    # 이전 Task에서 데이터 받기
    rows = context["ti"].xcom_pull(key="rows")
    # BigQuery에 적재
    load_to_bigquery(rows)

# DAG 정의
with DAG(
    "dev2_daily_etl",
```

```
    schedule_interval="0 2 * * *",      # 매일 새벽 2시
    start_date=datetime(2026, 5, 22),
    catchup=False,                        # 과거 실행 안 함
    default_args={
        "retries": 1,                     # 실패 시 1회 재시도
        "retry_delay": timedelta(minutes=5)
    },
) as dag:

    extract_task = PythonOperator(task_id="extract", python_callable=extract)
    load_task = PythonOperator(task_id="load", python_callable=load)

    extract_task >> load_task  # extract 끝나면 load 실행
```

XCom: Task 간 데이터 전달 방법

```
# Task A: 데이터 저장
context["ti"].xcom_push(key="rows", value=rows)

# Task B: 데이터 받기
rows = context["ti"].xcom_pull(key="rows")
```

복습 퀴즈

1. Q: DAG에서 `>>` 연산자의 역할은? A: (Task 실행 순서 정의. `A >> B` = A 끝나면 B 실행)
2. Q: `"0 2 * * *"` cron 표현식의 의미는? A: (매일 새벽 2시 0분에 실행)
3. Q: Dev 2에서 Airflow를 쓰는 이유는? A: (매일 PostgreSQL → BigQuery ETL 자동화 + 주간 인사이트 리포트 자동 생성)

TDD (테스트 주도 개발) 학습 노트

날짜: 2026-05-22 복습 예정: D+1, D+3, D+7

한 줄 요약

코드 짜기 전에 테스트 먼저 쓰는 개발 방식 — "이렇게 동작해야 한다"를 먼저 정의하고 코드를 맞춰간다.

---

1. 비유

일반 개발:

```
코드 작성 → 실행해서 확인 → 버그 발견 → 수정 → 반복
```

TDD:

테스트 작성 (실패) → 코드 작성 (테스트 통과) → 리팩토링 → 반복

자동차 설계에 비유: - 일반: 차 먼저 만들고 → 충돌 테스트 - TDD: 충돌 기준 먼저 정의 → 그 기준 통과하는 차 설계

---

2. 개념

TDD 사이클 (Red → Green → Refactor):

1. **Red**: 실패하는 테스트 작성
2. **Green**: 테스트를 통과하는 최소한의 코드 작성
3. **Refactor**: 코드 정리 (테스트는 계속 통과 상태 유지)

---

3. 왜 쓰나

- 없으면: 구현 후 테스트 → 테스트가 구현에 맞춰짐 → 잘못된 동작도 테스트가 통과
- 있으면: 테스트가 명세서 역할 → 구현이 테스트에 맞춰짐 → 올바른 동작 보장

Dev 2에서: 각 노드(run_tools, llm_judge 등)를 TDD로 구현 → 노드 교체 시 회귀 버그 방지

---

4. 코드 (Dev 2)

예시: **llm_judge_node Minor 스킵 TDD**

```
# Step 1: RED — 실패하는 테스트 먼저 작성
# dev2/tests/test_nodes.py

import pytest
from unittest.mock import AsyncMock, patch

@pytest.mark.asyncio
async def test_llm_judge_skips_minor():
    from dev2.nodes.llm_judge import llm_judge_node  # 아직 없음 → ImportError
    minor_issue = {
        "severity": "Minor", "line": 1, "file": "a.py",
        "message": "unused var", "rule_id": "SA-001",
        "suggestion": "fix", "similar_issues": []
    }
    state = {"payload": PAYLOAD, "enriched": [minor_issue], ...}

    with patch("dev2.nodes.llm_judge._get_cached_llm") as mock_llm:
        result = await llm_judge_node(state)
```

```
        mock_llm.assert_not_called()   # Minor는 LLM 호출 안 해야 함
        assert result["enriched"][0]["severity"] == "Minor"

# 실행 → FAIL (ImportError: No module named 'dev2.nodes.llm_judge')
pytest dev2/tests/test_nodes.py::test_llm_judge_skips_minor -v

# Step 2: GREEN — 테스트 통과하는 최소 코드 작성
# dev2/nodes/llm_judge.py

async def llm_judge_node(state):
    result = []
    for issue in state["enriched"]:
        if issue["severity"] == "Minor":   # Minor 스킵
            result.append(issue)
            continue
        # ... LLM 호출
    return {"enriched": result}

# 실행 → PASS
pytest dev2/tests/test_nodes.py::test_llm_judge_skips_minor -v

# Step 3: REFACTOR — 코드 정리 (테스트는 계속 통과)
# MODEL_ROUTING dict로 정리, lru_cache 추가 등
```

---

5. pytest 핵심 문법

```
# 기본 테스트
def test_something():
    assert result == expected

# 비동기 테스트
@pytest.mark.asyncio
async def test_async():
    result = await some_async_func()
    assert result == expected

# Mock: 외부 의존성 대체
from unittest.mock import patch, AsyncMock

with patch("module.function", return_value="fake"):
    result = function_that_calls_it()

# AsyncMock: async 함수를 대체
with patch("module.async_func", new_callable=AsyncMock, return_value="fake"):
    result = await function_that_calls_it()
```

복습 퀴즈

1. Q: TDD Red-Green-Refactor 순서의 의미는? A: (Red=실패 테스트 작성, Green=최소 코드로 통과, Refactor=코드 정리)
2. Q: TDD에서 Mock을 쓰는 이유는? A: (LLM API 호출, DB 같은 외부 의존성 없이 단위 테스트 가능하게 하기 위해)
3. Q: Dev 2에서 TDD로 얻는 이점은? A: (각 노드의 동작 보장 + LangGraph 순서 변경 시 회귀 버그 즉시 감지)

Dev2 MVP 실행 학습 노트

날짜: 2026-05-29 복습 예정: D+1, D+3, D+7

한 줄 요약

Docker postgres 켜고 → FastAPI 서버 띄우고 → curl로 엔드포인트 호출하면 MVP 동작.

전체 실행 흐름

```
docker compose up -d postgres
  → 컨테이너 시작 + migrations/ 폴더 SQL 자동 실행 → 테이블 생성

uvicorn dev2.main:app --port 8005
  → FastAPI 서버 기동 → /scan, /insights, /issues, /health 엔드포인트 활성화

POST /scan/mock-bandit
  → mock_vulnerable/ 폴더 Bandit 스캔
  → bandit_result.json 파싱
  → db.py 함수로 PostgreSQL INSERT
  → 결과 JSON 반환

GET /insights/developer/{id}
  → DB에서 scan_issues 집계 → 통계 반환
```

실제 명령어 (복붙용)

1. postgres 시작

```
cd sub2_new/dev2

# Windows에서 필수 (Docker Desktop이 아닌 default context 사용)
docker context use default

docker compose up -d postgres
```

```
# 테이블 확인
docker exec dev2-postgres-1 psql -U dev2 -d dev2_db -c "\dt"
```

2. FastAPI 서버 시작

```
cd sub2_new

DATABASE_URL="postgresql+asyncpg://dev2:dev2@localhost:5432/dev2_db" \
USE_MOCK_LLM=true \
ANTHROPIC_API_KEY=sk-ant-dummy \
LANGCHAIN_TRACING_V2=false \
python -m uvicorn dev2.main:app --port 8005
```

포트 8002/8003은 다른 팀원 서비스가 점유 중 → 8005 사용

3. 헬스 체크

```
curl http://localhost:8005/health
# {"status":"ok","service":"dev2"}
```

4. Demo 시나리오 실행 **(Prototype 2)**

```
curl -X POST http://localhost:8005/scan/mock-bandit
# {"status":"success","scan_id":4,"issue_count":10,"summary":
{"critical":2,"major":3,"minor":5}}
```

5. 결과 조회

```
# API로 조회
curl http://localhost:8005/insights/developer/dev2
curl http://localhost:8005/insights/team

# DB 직접 조회
docker exec dev2-postgres-1 psql -U dev2 -d dev2_db \
  -c "SELECT severity, rule_id, message FROM scan_issues ORDER BY severity,
line;"
```

6. 정식 **/scan** 엔드포인트 **(Prototype 1)**

```
curl -X POST http://localhost:8005/scan \
  -H "Content-Type: application/json" \
  -d '{
    "task_id": "test-001",
    "pr_id": "PR-42",
    "pr_diff": "diff --git a/auth.py b/auth.py\n...",
    "developer": {
      "id": "dev1",
      "skill_level": "junior",
      "tech_stack": ["python"]
    },
    "context": {
      "architecture_docs": [],
```

```
      "coding_conventions": "",
      "previous_pr_issues": []
    }
  }'
# → 202 Accepted (LangGraph 5노드 파이프라인 백그라운드 실행)
```

주요 트러블슈팅

증상	원인	해결
`dockerDesktopLinuxEngine` 에러	Docker context가 desktop-linux	`docker context use default`
`password authentication failed`	.env와 실제 볼륨 비밀번호 불일치	`POSTGRES_PASSWORD=dev2` (볼륨 초기화 시점 값)
서버 올렸는데 `/feeds/search` 라우트	다른 팀원 서비스가 포트 점유	다른 포트(8005) 사용
`PYTHONPATH` 없으면 `dev2` 모듈 못 찾음	Python이 `sub2_new/dev2`를 패키지로 인식 못함	`PYTHONPATH=/c/.../sub2_new` 설정

Swagger UI

`http://localhost:8005/docs`

브라우저에서 모든 엔드포인트 GUI 테스트 가능.

핵심 개념 3가지

1. 마이그레이션 자동 실행: `docker-compose.yml`의 `volumes: ./migrations:/docker-entrypoint-initdb.d` → postgres 최초 시작 시 SQL 파일 자동 실행
2. **202 Accepted** 패턴: POST /scan은 즉시 응답 반환 후 BackgroundTasks로 무거운 LLM 분석 실행 (클라이언트 타임아웃 방지)
3. **USE_MOCK_LLM=true**: LLM API 없이도 전체 파이프라인 테스트 가능 (개발 단계)

복습 퀴즈

1. Q: postgres 컨테이너가 테이블을 자동 생성하는 이유는? A: (docker-compose의 migrations 볼륨 마운트 → initdb.d 자동 실행)
2. Q: POST /scan이 202를 반환하는 이유는? A: (LLM 분석이 수십 초 걸려 클라이언트 타임아웃 방지 위해 BackgroundTasks 패턴 사용)

3. Q: Windows에서 Docker 사용 시 먼저 해야 할 것은? A: (`docker context use default` — desktop-linux context는 Docker Desktop 전용)

Dev2 전체 실행 플로우

날짜: 2026-05-29 복습 예정: D+1, D+3, D+7

1단계 — 환경 준비 (서버 시작 전)

```
Windows 필수
  └── docker context use default
        (desktop-linux context → default로 전환, 안 하면 Docker 연결 오류)
```

2단계 — DB 시작

```
docker compose up -d postgres
        │
        ▼
pgvector/pgvector:pg16 컨테이너 기동
        │
        ▼
volumes: ./migrations:/docker-entrypoint-initdb.d
        │
        ▼  (postgres 최초 시작 시 자동 실행)
SQL 파일 자동 실행 → 테이블 생성
  • scan_executions  (스캔 실행 이력)
  • scan_issues      (개별 이슈)
  • scan_summaries   (심각도별 집계)
  • chunk_embeddings (pgVector RAG용)
```

3단계 — FastAPI 서버 시작

```
python -m uvicorn dev2.main:app --port 8005
        │
        ▼
main.py — 라우터 3개 등록
  • scan_router    → POST /scan, POST /scan/mock-bandit
  • insights_router→ GET  /insights/developer/{id}, /insights/team, /insights/
trends
  • issues_router  → PATCH /issues/{id}/dismiss
  • /health        → GET  {"status":"ok","service":"dev2"}
```

4단계 — 요청 수신 (두 가지 경로)

경로 A — 정식 스캔 (Prototype 1)

```
Dev1 Orchestrator
  └── POST /scan + TaskPayload JSON
            │
            ▼
      FastAPI 파싱 & Pydantic 검증
            │
            ▼
      _payload_to_state() — TaskPayload → Dev2State 변환
            │
            ▼
      BackgroundTasks.add_task(run_dev2, state)
            │
            ▼
      202 Accepted 즉시 반환 ← 클라이언트가 기다리지 않아도 됨
            │
            ▼  (응답 전송 후 백그라운드에서 실행)
      run_dev2(state) → LangGraph 5노드 파이프라인 시작
```

경로 B — Mock Bandit 스캔 (Prototype 2)

```
curl -X POST /scan/mock-bandit
  (TaskPayload 없이 호출 가능 — MVP 테스트용)
        │
        ▼
  asyncio.to_thread(subprocess.run, ["bandit", ...])
        │
        ▼
  bandit_result.json 생성
        │
        ▼
  JSON 파싱 → issues 리스트 + summary 딕셔너리
        │
        ▼
  db.py 함수 직접 호출 (경로 A와 달리 LangGraph 안 거침)
        │
        ▼
  200 OK + {"status":"success","scan_id":N,"issue_count":10,...}
```

5단계 — LangGraph 5노드 파이프라인 (경로 A만)

```
START
  │
  ▼
[run_tools] ─── ESLint / Pylint / Bandit / pip-audit 병렬 실행
  │               PR diff에서 +줄만 추출 → 각 도구로 분석
```

```
  │              결과: state["static_raw"], state["security_raw"]
  │
  ▼
[vector_search] ─── pgVector RAG Retrieve
  │                  이슈와 유사한 과거 사례 검색 (chunk_embeddings)
  │                  결과: state["enriched"] (유사 이슈 첨부)
  │
  ▼
[llm_judge] ─── claude-sonnet-4-6 RAG Generate
  │              유사 사례 + PR diff 컨텍스트로 severity 재판단
  │              Minor → Major, Major → Critical 상향 가능
  │              결과: state["enriched"] (severity 확정)
  │
  ▼
[_route_after_judge()] ─── 조건부 분기 (Agentic Workflow 핵심)
  │
  ├── Critical ≥ 1 ──▶ [escalate] ─── Dev4에 즉시 알림 전송
  │                          │          Slack #team-security 에스컬레이션
  │                          ▼
  └── Critical = 0 ─────▶ [store]
                             │
                             ▼
                      PostgreSQL INSERT
                        • scan_executions (실행 이력)
                        • scan_issues     (이슈 목록)
                        • scan_summaries  (집계)
                      BigQuery INSERT (OLAP 분석용)
                             │
                             ▼
                             END
                      state["final_output"] 반환
```

6단계 — 결과 조회

```
GET /insights/developer/{developer_id}
        │
        ▼
  scan_executions + scan_issues JOIN
        │
        ▼
  {"developer_id":..., "total_issues":..., "critical":..., ...}

GET /insights/team
        │
        ▼
  전체 팀 집계 통계

PATCH /issues/{id}/dismiss
```

```
        |
        ▼
  오탐(false positive) 마킹 → dismissed=true
```

전체 요약 (한눈에)

```
[환경준비] docker context use default
    ↓
[DB]    docker compose up -d postgres  →  테이블 자동 생성
    ↓
[서버]  uvicorn dev2.main:app --port 8005
    ↓
[요청]  POST /scan (Dev1) 또는 POST /scan/mock-bandit (테스트)
    ↓
[분석]  run_tools → vector_search → llm_judge → (escalate?) → store
    ↓
[저장]  PostgreSQL scan_executions / scan_issues / scan_summaries
    ↓
[조회]  GET /insights/* → 통계 반환
```